

Poznámky k předmětu Využití počítačů ve fyzice

Pavel Stránský

5. května 2025

Obsah

1 Úvod	4
1.1 Výběr programovacího jazyka	4
1.2 Výběr vývojového prostředí	5
2 Instalace používaných nástrojů	7
2.1 Instalace Pythonu	7
2.1.1 Instalace doplňujících knihoven	7
2.2 Instalace Visual Studio Code	8
2.2.1 Klávesové zkratky	8
2.2.2 Instalace doplňku pro Python	9
2.3 Instalace Git	9
2.4 Anaconda	10
3 Úvod do používaných nástrojů	11
3.1 Verzovací systém Git	11
3.1.1 Prvotní nastavení	12
3.1.2 Čtyři možné stavy souborů v repozitáři	12
3.1.3 Životní cyklus souborů v repozitáři	13
3.1.3.1 Vytvoření nového repozitáře	13
3.1.3.2 Stav repozitáře	14
3.1.3.3 Vytvoření a první zapsání nového souboru	14
3.1.3.4 Změny v souboru	15
3.1.3.5 Přidání dalšího souboru do projektu	16
3.1.3.6 Zobrazení historie revizí	17
3.1.4 .gitignore	18
3.1.5 Větve	19
3.1.5.1 Vytvoření nové větve	19
3.1.5.2 Výpis větví	19
3.1.5.3 Změny a sloučení větví	19
3.1.5.4 Odstranění větve	20
3.1.6 Návrat k dřívějším verzím	20
3.1.7 Vzdálené repozitáře	20
3.1.8 Tahák	21
3.2 Programovací jazyk Python	22
3.2.1 Vytvoření a spuštění kódu	22
3.2.2 Základy syntaxe Pythonu	23
3.2.3 Pojmenování proměnných a formátování	24
3.2.4 Řady, vektory, matice	25
3.2.5 Grafy	25

3.2.6	Pseudonáhodná čísla	25
3.2.7	Práce se zvukem	26
4	Obyčejné diferenciální rovnice 1. řádu	28
4.1	Pár důležitých pojmů	28
4.2	Eulerova metoda 1. řádu	29
4.3	Eulerova metoda 2. řádu	29
4.4	Runge-Kuttova metoda 4. řádu	29
4.5	Odvození metod	30
5	Soustavy diferenciálních rovnic 1. řádu	34
5.1	Symplektické algoritmy	34
5.2	Shrnutí	38
6	Náhodná procházka	40
7	Hledání minima funkce	43
7.1	Metropolisův algoritmus	43
7.2	Minimalizace pomocí knihovny SciPy	43
7.3	Shrnutí	47
8	Histogram	48
8.1	Základní definice a tvrzení z teorie pravděpodobnosti	48
8.2	Příklady náhodných veličin	50
8.3	Výběr z neznámého rozdělení	51
9	Monte-Carlo metoda	57
9.1	Hit-And-Miss	58
9.1.1	Chyba	58
9.1.2	Použití	59
9.2	Monte-Carlo integrace	60
10	Paralelizace	63
11	LaTeX	67
11.1	Formát T _E Xovského souboru	68
11.2	Struktura T _E Xovského souboru	68
11.2.1	Hlavička	68
11.2.2	Preamble	68
11.2.3	Tělo dokumentu	69
11.3	Další návody a odkazy	69
12	Symbolické výpočty	70
12.1	Mathematica	70
13	Fourierova transformace	72
13.1	Diskrétní Fourierova transformace	72
13.2	Použití Fourierovy transformace	74
13.3	Rychlá Fourierova transformace	78

14 Kafe	79
14.1 Načtení dat	79
14.2 Model	79
14.3 Fitování dat	80
14.4 Vylepšení modelu	81
14.4.1 Neurčitá teplota okolí	81
14.4.2 Model s vyzařováním	81
14.4.3 Další člen Taylorova rozvoje	83
14.5 Shrnutí	83
14.6 Uložení dat	84

1 Úvod

Cílem tohoto cvičení je pohrát si s jednoduchými fyzikálními problémy, vyzkoušet si různé způsoby jejich řešení a získat **zkušenost a odvalu** pro řešení složitějších problémů v budoucnosti. Postupy a kódy, které si budeme ukazovat, nemusejí být ty nejsofistikovanější či neoptimalnější. Půjde nám hlavně o srozumitelnost a o pochopení principu, jak nejběžnější numerické algoritmy fungují, jaký algoritmus zvolit pro konkrétní úlohu, jaké mají probírané algoritmy vlastnosti a na co si při jejich použití dát pozor. Vzhledem k tomu, že prezentované algoritmy budou relativně jednoduché, budeme se snažit o maximální eleganci výsledného programu, přičemž pod elegancí budeme chápat hlavně dobrou **čitelnost a udržitelnost** kódu.

Ve cvičení nebude probíhat systematická výuka žádného programovacího jazyka ani numerických algoritmů, k tomu slouží jiné dedikované přednášky. Od účastníka cvičení se očekává, že již prošel kurzem základů programování a je schopen napsat funkční kód v nějakém aktuálním programovacím jazyce.

1.1 Výběr programovacího jazyka

Většina příkladů cvičení bude demonstrována v programovacím jazyce **Python**. Jeho výhodou je jednoduchost, v dnešní době velká rozšířenost a existence nepřehledného množství pokročilých knihoven, které lze snadno použít. Neznamena to, že se musíte upínat k Pythonu, pokud vám není sympatický nebo pokud ovládáte jiný programovací jazyk a chcete raději pilovat své schopnosti v něm. Každé cvičení bude mít vždy „teoretickou část“, ve které budou algoritmy a techniky vysvětleny bez vazby na konkrétní programovací jazyk. Výsledné programy pak nebudou nijak obsáhlé a až na výjimky (úvod do paralelizace) nebudou vyžadovat při programování žádné speciální finesy,¹ lze je tudíž napsat snad v jakémkoliv dnes používaném jazyce. Pokud se rozhodnete (nyní nebo kdykoliv v budoucnu) pro jiný jazyk než Python, doporučuji dát si pozor na následujících několik bodů, ve kterých se programovací jazyky lišívají:

- *Indexování:* Některé programovací jazyky indexují pole od 0, jiné od 1. Ve dvou a vícerozměrných polích může první index udávat někdy řádek, jindy sloupec.
- *Číselné typy a implicitní přetypování:* Rozdíly jsou zejména u operace dělení / dvou celých čísel, kde v některých jazycích je výsledkem celé číslo, tedy celá část po výsledku dělení, v jiných číslo s desítnou čárkou. Různě také funguje implicitní přetypování, a to zejména mezi číslem a řetězcem (výrazy typu "12.3" + 4).
- *Práce s textovými řetězci a speciálními znaky; formátování:* Různé jazyky používají různé způsoby pro formátování textových řetězců a pro převod číselných hodnot na text. V některých jazycích je textový řetězec jen pole znaků, a je tedy možné k jednotlivým znakům přistupovat pomocí indexů a stejným způsobem je měnit. To přináší komplikace při použití bohatších znakových sad,² proto nejnovější jazyky od možnosti přímé editace textových řetězců upouštějí.
- *Parametry funkcí* mohou být předávány hodnotou nebo odkazem. Pokud jsou předávány odkazem, pak při změně hodnoty proměnné uvnitř funkce dojde ke změně i vně funkce. Podobně může fungovat přiřazení jedné proměnné do druhé.
- *Obor platnosti proměnných:* Některé programovací jazyky mají bloky, ve kterých pokud deklarujeme proměnnou, při opuštění bloku přestane být dostupná. V jiných jazycích může být obor platnosti proměnné například celá funkce. V některých jazycích neexistují globální proměnné.³

¹Například objektově orientované programování, abstraktní typy, asynchronní programování, regulární výrazy atd.

²Například dnes nejčastěji používaná znaková sada UTF-8 může mít jednotlivý znak reprezentovaný jedním až čtyřmi bajty.

³I pokud existují, doporučuje se jejich použití omezit na minimum a používat je jen v opravdu odůvodněných případech.

- *Správa paměti:* Drtivá většina dnešních programovacích jazyků má automatickou správu paměti, kdy nemusíte řešit, kdy a kolik paměti alokujete a že ji musíte uvolnit, když ji přestanete používat. Nízkoúrovňové jazyky (například C/C++) však vyžadují, aby se o správu paměti staral programátor sám.

Pro fyzika je navíc důležité, aby programovací jazyk, který si zvolí, byl silný v následujících bodech:

- *Existence základních numerických knihoven:* Nejčastěji je potřeba numerické řešení diferenciálních rovnic, práce s pseudonáhodnými čísly, diagonalizace matic a fitování parametrů funkcí ze zadaných dat. Užitečná je také rychlá Fourierova transformace a v dnešní době i možnost zvýšení přesnosti čísel s desetinnou čárkou nad standardních 64-bitů. Experimentátoři využijí pokročilé možnosti analýzy a filtrování dat.
- *Možnost tvorby grafů:* Grafická reprezentace výsledků bývá obvykle nejčitelnější a nejsrozumitelnější. V základu se hodí grafy funkcí s jedním parametrem a konturové grafy. V dnešní době je navíc výhodou, pokud program umí vytvořit animace.
- *Možnost spustit program na různých platformách:* Výpočetní clustery a dedikované výpočetní počítače pracují často pod jinými operačními systémy než běžné osobní počítače. Programovací jazyk by měl existovat a být snadno nainstalovatelný zejména na stroje s operačním systémem Linux.
- *Podpora distribuovaných výpočtů a výpočtů ve více vláknech:* Výpočetní síla dnešních počítačů používaných ve vědě spočívá v paralelizaci.

1.2 Výběr vývojového prostředí

Zatímco na volbě programovacího jazyka záleží, co všechno budeme schopni naprogramovat a jak budou výsledné programy rychlé a efektivní, tak to, jak se nám bude dobře a rychle kód psát, záleží na volbě vývojového prostředí. Na funkčnost výsledného programu nemá vliv, zda jsme ho napsali v jednoduchém textovém editoru nebo v sofistikovaném programu s pokročilými možnostmi spouštění, ladění a verzování kódu. Je ale jasné, že to druhé nám významně usnadní práci a sníží pravděpodobnost chyb, které náš program bude nakonec obsahovat.

Jako vývojové prostředí v tomto cvičení bude používáno [Visual Studio Code](#). Tento volně dostupný program lze nainstalovat na všechny neuznávanější operační systémy (Linux, Windows, MacOS). Má nepřehledné možnosti při editaci zdrojových souborů, překladu a ladění snad ve všech známých programovacích jazycích. Bohaté možnosti nastavení umožňují přizpůsobit si práci svým potřebám (například zvýrazňování syntaxe, klávesové zkratky či vzhled prostředí). Pomocí doplňků ho můžete integrovat s dalšími službami, například s verzovacím programem Git nebo vzdálenými repozitáři, čehož také využijeme. Komunita, která toto vývojové prostředí používá a spravuje, je obrovská, což zaručuje dobrou podporu a rychlé přidávání nových funkcí. Je to právě univerzálnost a rozšířenost tohoto vývojového prostředí, díky které padla volba na něj.

Pro Python existuje samozřejmě obrovské množství dalších velmi kvalitních vývojových prostředí. Nejčastěji používané jsou

- [IDLE](#):⁴ Toto jednoduché vývojové prostředí je součástí instalace jazyka Python. Obsahuje základní zvýrazňování syntaxe, má však omezené možnosti co se týče ladění, psaní rozsáhlejších projektů s více zdrojovými soubory nebo integrace s verzovacími programy.
- [Spyder](#) a [PyCharm](#): Vývojová prostředí s pokročilými funkcemi přímo určená pro jazyk Python. Vynikající alternativy pro Visual Studio Code, které v mnoha ohledech dokonce překonávají.
- [Jupyter](#): Jiná koncepce práce s programovacím jazykem. Kód je součástí tzv. *zápisníku (notebook)*, ve kterém může být zároveň strukturovaný a formátovaný text.⁵ Jedná se tedy o

⁴Integrated Development and Learning Environment.

⁵Stejný přístup má například program [Mathematica](#), které se budeme věnovat v sekci [12.1](#).

kombinaci programu a poznámek, kde součástí poznámek jsou přímo výsledky výpočtů (tabulky, grafy atd.). Zápisník lze snadno konvertovat například do formátu LaTeX nebo PDF. Koncepce zápisníku však není příliš vhodná při použití verzovacích programů a při práci na rozsáhlejších projektech v týmu.

Pokud teprve vybíráte, které vývojové prostředí se naučíte, doporučuji zvážit, zda potřebujete následující funkce a vlastnosti a nakolik je dané vývojové prostředí podporuje.

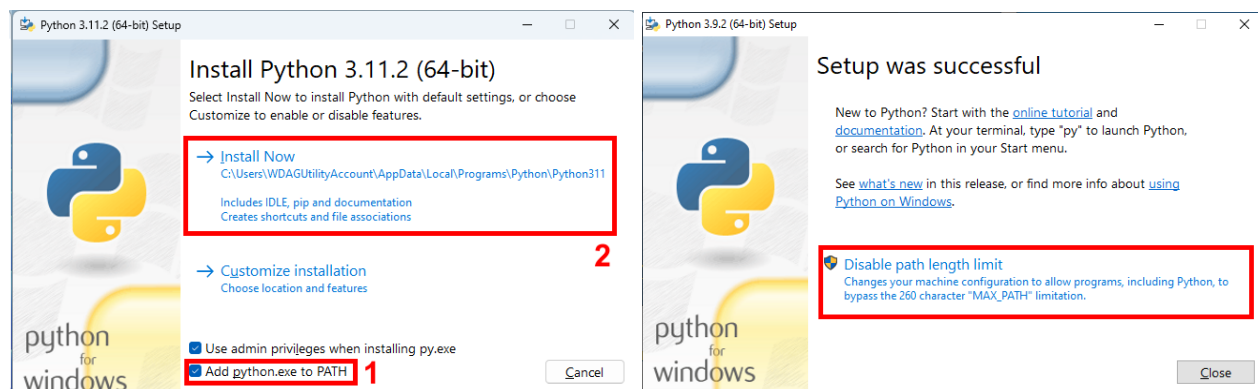
- *Zvýrazňování syntaxe*: Pomáhá čitelnosti a snazší orientaci v kódu.
- *Spouštění a ladění kódu (debugger)*: To zahrnuje vkládání *ladících bodů (breakpoints)*, na kterých se chceme, aby se program zastavil, možnost krokování a výpisu (a často i editace) aktuálních hodnot proměnných či podrobné informace o chybách, pokud k nim při provádění programu dojde, včetně rychlého skoku na příslušná místa kódu.
- *Našeptávání a automatické doplňování*: Zatímco dříve byly názvy proměnných často omezené na malý počet znaků, dnešní programovací jazyky umožňují používat v podstatě libovolnou délku. Není také nutné omezovat se jen na znaky ASCII, nýbrž proměnné a funkce lze pojmenovávat včetně symbolů s diakritikou a písmen z jiných abeced (včetně například řeckých písmen). Klidně delší, ale výstižné pojmenování proměnných se doporučuje i kvůli lepší čitelnosti kódu bez nutnosti obsáhlých a na údržbu náročných komentářů. Automatické dokončování výrazně zrychluje psaní a snižuje množství chyb a překlepů.
- *Integrace s verzovacími systémy*: Pokud budete pracovat na větším týmovém projektu, použití verzovacích systémů se nevyhnete. Práci usnadňuje, když samotné vývojové prostředí umí zapisovat nové verze a porovnávat různé verze téhož souboru s vyznačením provedených změn.

2 Instalace používaných nástrojů

Vzorové příklady k cvičení budou vypracovány v nejnovější verzi programovacího jazyka **Python**. Jako vývojové prostředí bude používáno **Visual Studio Code**. V této sekci si stručně ukážeme, jak programy nainstalovat a jak provést jejich základní nastavení.

2.1 Instalace Pythonu

Instalační soubor pro svůj operační systém stáhnete ze stránky <https://www.python.org/downloads/>. Při instalaci na počítač s Windows doporučuji zvolit „Add Python to PATH“, což zjednoduší práci s Pythonem z příkazové řádky, a na posledním dialogovém okně obrazovce zvolit „Disable path length limit“ (pokud je tato volba k dispozici):



Pro ověření instalace napíšeme v příkazové řádce příkaz `python`.⁶ Tím se spustí REPL⁷ Pythonu, ve které můžeme již psát všechny příkazy programovacího jazyka, které se po zadání ihned provedou a vypíší výsledek. REPL ukončíme zadáním příkazu `exit()`.

2.1.1 Instalace doplňujících knihoven

Samotná instalace Pythonu obsahuje jen minimální množství nejnutnějších knihoven. My budeme využívat ještě následující rozšiřující knihovny:

- **NumPy** (**N**umerical **P**ython: numerická matematika, řady a vícedimenzionální datové typy),
- **SciPy** (**S**cientific **P**ython: algoritmy pro optimalizaci, statistiku, řešení diferenciálních rovnic, lineární algebru, atd.),
- **Matplotlib** (vizualizace, grafy).

K jejich doinstalování slouží modul `pip`. V příkazové řádce napíšeme

```
python -m pip install numpy scipy matplotlib
```

čímž se nainstalují naráz všechny tři knihovny.⁸

Existuje samozřejmě celá řada dalších užitečných a používaných knihoven, jako je například

- **Pandas** pro zpracování a analýzu dat,
- **SymPy** pro symbolické výpočty,
- **TensorFlow**, **scikit-learn** (a další) pro strojové učení,

⁶Na počítačích s Linuxem je příkaz v terminálu `python3`.

⁷**R**ead-**E**valuate-**P**rint-**L**oop.

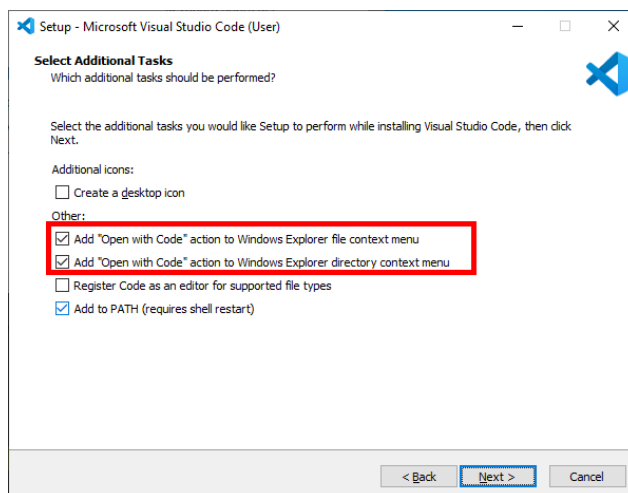
⁸Pokud na počítači s Linuxem uvedený postup nebude fungovat, je potřeba nejprve nainstalovat instalátor `pip` pomocí příkazu `sudo apt install python3-pip`. Pak lze použít buď výše uvedený příkaz, nebo stručnější `pip3 install numpy`.

- **QuTiP** pro simulace kvantových systémů,

a nepřehledné množství vysoce specializovaných knihoven pro konkrétní fyzikální, matematické či statistické úkoly. S nimi se seznámíte ve specializovaných kurzech a přednáškách.

2.2 Instalace Visual Studio Code

Zde projdeme instalaci vývojového prostředí **Visual Studio Code** s doplňkem pro programovací jazyk Python. Instalační soubor stáhnete ze stránky <https://code.visualstudio.com>. Během instalace na počítač s Windows doporučuji zvolit obě možnosti „Add Open with Code action to...“,



což zjednoduší otevírání složek s projekty nebo samostatných souborů pomocí pravého tlačítka myši.

Na operačním systému Linux je nejjednodušší provést instalaci pomocí Snap Store příkazem v terminálu

```
sudo snap install --classic code
```

nebo použít tento [návod](#).

2.2.1 Klávesové zkratky

Ačkoliv veškeré funkce vývojového prostředí Visual Studio Code lze používat pomocí myši, pro seriózní práci je dobré se naučit klávesové zkratky, jelikož významně zrychlují a zefektivňují práci. Základní klávesové zkratky (nový soubor, uložení souboru, hledání v souboru, navigace v souboru, kopírování a vkládání atd.) fungují stejně jako v jiných programech. Kromě toho je ve Visual Studio Code množství klávesových zkratk, které usnadňují editaci, spouštění, ladění a správu kódu. Všechny klávesové zkratky lze zobrazit a upravit podle vlastních preferencí postupným stiskem kláves **Ctrl+K Ctrl+S**.

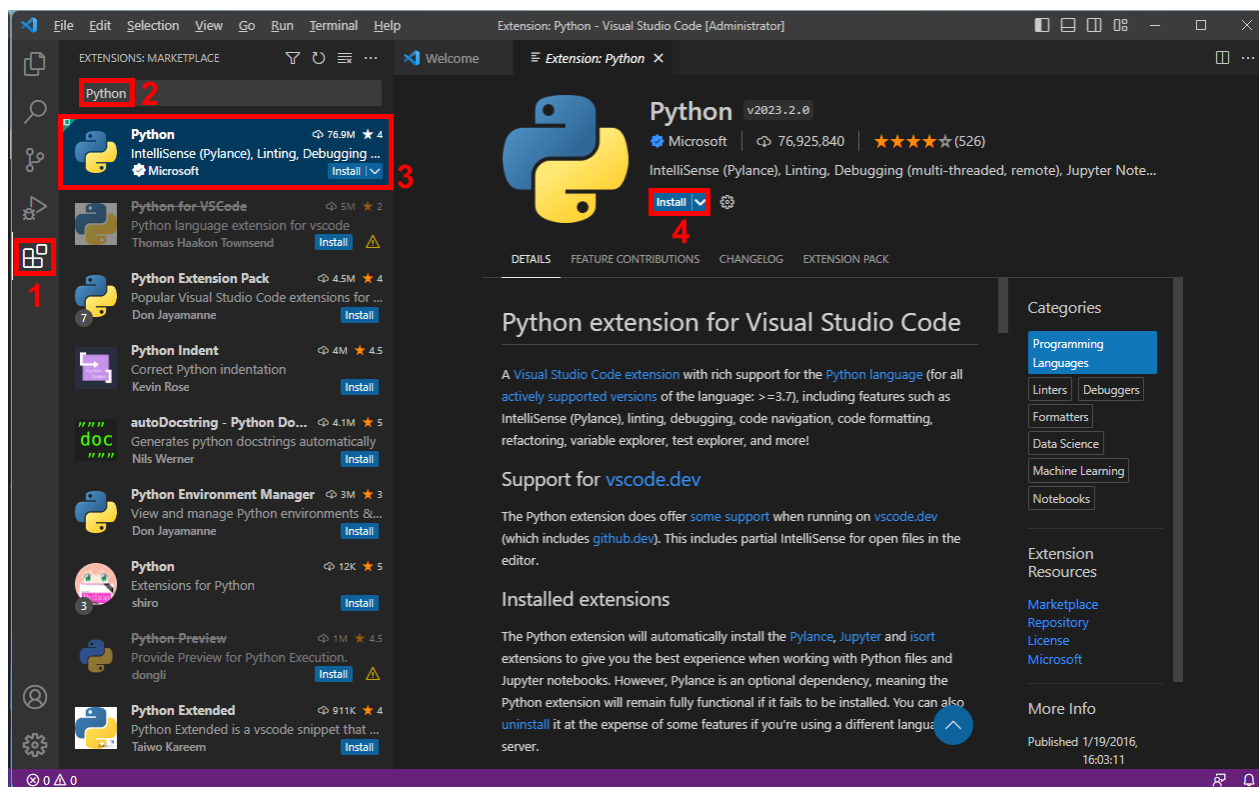
Klávesové zkratky se liší podle použitého operačního systému. Doporučuji vám projít si oficiální „taháky“ s nejdůležitějšími zkratkami pro **Windows**, **Linux** i **MacOS** přímo od tvůrců tohoto vývojového prostředí. Uvedené klávesové zkratky platí pro rozložení klávesnice Angličtina (Spojené státy). Pro jiné rozložení klávesnice nemusejí některé z nich fungovat.

Pro operační systém Windows nejdůležitější zkratky jsou:

- **Alt+Z**: Zalomí dlouhé řádky na konci okna.
- **Alt+←**: Vrátí kurzor na předchozí pozici (například po odskrolování někam do neznáma).
- **Ctrl+/***: Přidá či odstraní komentář řádku.
- **Ctrl+[**, **Tab**: Odsadí označenou část kódu o jednu úroveň.

- **Ctrl+]**, **Shift+Tab**: Zmenší odsazení označené části kódu o jednu úroveň.
- **Ctrl+Mezera**: Zobrazí okénko s nápovědou k danému symbolu.
- **F12**: Posune kurzor na definici daného symbolu (proměnné, funkce, atd.).
- **Shift+F12**: Zobrazí všechny reference na daný symbol.
- **Ctrl+G**: Přejít na řádek s daným číslem.
- **Ctrl+Shift+0**: Zobrazí všechny definované symboly v otevřeném okně.
- **F2**: Přejmenuje symbol (proměnnou, funkci, atd.) v rozsahu jeho platnosti.
- **Shift+Enter**: Spustí označenou část kódu v konzoli.
- **Ctrl+F5**: Spustí otevřený soubor.
- **F5**: Spustí otevřený soubor v režimu ladění.
- **F9**: Přidá ladící bod.

2.2.2 Instalace doplňku pro Python

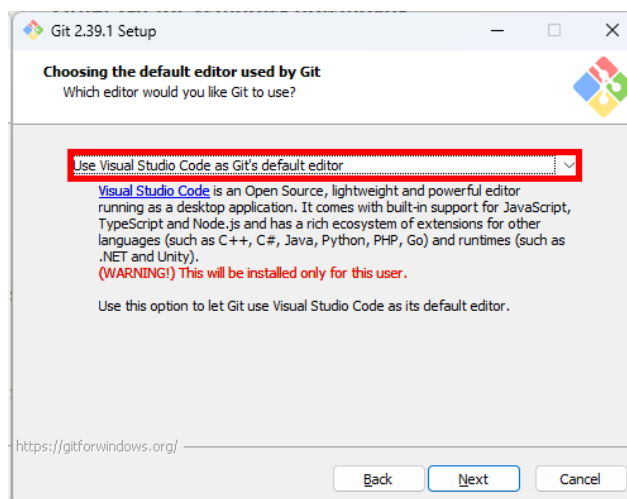


Ke správě doplňků (extensions) se dostanete kliknutím na ikonku 1 nebo stisknutím **Ctrl+Shift+X**. Vyhledáte doplněk **Python** 2 od Microsoftu, vyberete ho 3 a nainstalujete 4.

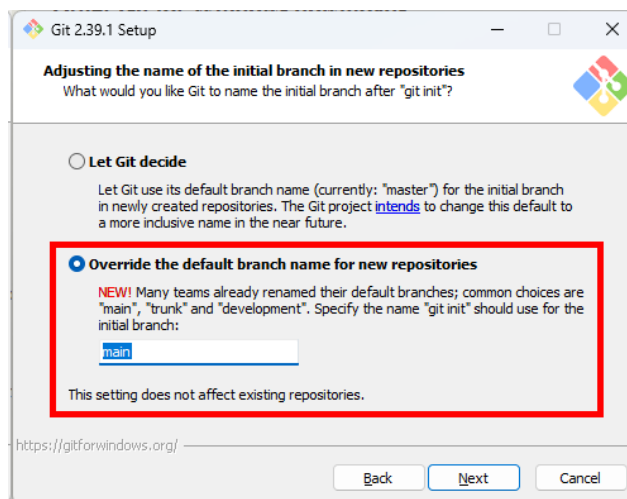
2.3 Instalace Git

Instalační soubor verzovacího systému **Git** stáhnete z webové stránky <https://git-scm.com>. K instalaci Gitu potřebujete administrátorská práva na svém počítači.

V následujícím postupu zobrazuji jen ty snímky obrazovky, na kterých je vhodné vybrat jinou volbu, než jaká je instalátorem standardně nabízena.



Jako editor zvolíme dříve nainstalované Visual Studio Code. Systém Git vyžaduje editor jednak pro povinný komentář každé zapsané změny (commit), jednak pro ošetření kolizí při slučování větví (merge).



Hlavní větev repozitáře se donedávna standardně jmenovala **master**. Vzhledem k negativním konotacím tohoto slova v angličtině se přechází na neutrálnější označení. Nejběžnější je **main**, na které již přešel i populární správce vzdálených repozitářů [GitHub](#). Doporučuji tedy zvolit pojmenování **main**.

Všechna ostatní nastavení, na která se táže instalační program, nechte na doporučené hodnotě. Nastavení lze samozřejmě kdykoliv po instalaci změnit.

2.4 Anaconda

Ačkoliv platforma [Anaconda](#) nebude v tomto kurzu využívána, vzhledem k jejímu rozšíření stojí za to se o ní zmínit. Jedná se o rozsáhlý balík programů zahrnující instalaci Pythonu a velké množství knihoven včetně všech knihoven zmiňovaných v sekci 2.1.1. Balík navíc obsahuje samostatnou instalaci vývojového prostředí Visual Studio Code a několik dalších vývojových prostředí včetně nejznámějších Spyder a Jupyter, které byly zmíněny v sekci 1.2. Výhoda platformy Anaconda a platform podobného typu spočívá v tom, že všechny knihovny jsou vybrány s ohledem na to, aby byly mezi sebou kompatibilní a aby byly zároveň kompatibilní s použitou verzí Pythonu. To je však zároveň i hlavní nevýhodou: verze Pythonu a přiložených knihoven nikdy není ta nejaktuálnější dostupná na trhu, a tedy nelze používat nejnovější funkce a rysy jazyka. Navíc Anaconda bude vždy obsahovat množství knihoven a programů, které nevyužijete, jen budou zabírat místo na disku a mohou zpomalovat chod používaných funkcí.

3 Úvod do používaných nástrojů

V této sekci naleznete základy použití verzovacího systému Git a úvod do syntaxe a idiomatiky jazyka Python. Tato sekce tvoří jakousi stručnou technickou „referenční příručku“, ke které se budou následující části poznámek odkazovat.

3.1 Verzovací systém Git

Každý si patrně už někdy v životě zasteskl, že nemá uložené dřívější verze svých souborů. Změny, které se ukazují být nevhodné či provedené omylem (kočka nepozorovaně přejde po klávesnici a my pak soubor uložíme), kamarád, který z nějakých důvodů chce zrovna tu verzi vašeho programu, kterou jste mu poskytli před rokem, a vy jste mezitím kód zásadně přepsali, to vše jsou důvody k předsevzetí nějakým způsobem důležité milníky v práci na projektu archivovat.

Triviální verzování může spočívat například v ručním ukládání kopií projektu s daty či jinými označeními jednotlivých verzí. Takovýto postup je ale velmi těžkopádný, jelikož v každé kopii musí být uloženy všechny soubory projektu, i pokud se v nich od předchozí verze nic nezměnilo. Obtížně se vyhledává, co přesně se mezi jednotlivými verzemi změnilo a kdo změnu provedl, pokud na projektu pracuje větší tým, přičemž obtížnost roste s velikostí a komplexností projektu. Pro usnadnění uchovávání historie změn proto vznikly verzovací systémy.

Verzovací systém pomáhá udržet historii změn souborů projektu, přičemž ke každému snímku historie se lze vrátit. Vše provádí chytře a v maximální míře automaticky. Systém sám sleduje, v jakých souborech byly provedeny změny. Pokud se jedná o textový soubor, umí porovnat aktuální a libovolnou historickou verzi řádek po řádku a zvýraznit odlišnosti. Umožňuje pracovat s nezávislými vývojovými větvemi (*branches*), ve kterých lze například zkoušet různý přístup k řešení daného problému a mezi kterými lze jednoduše přepínat. Vybrané řešení lze pak snadno začlenit (*merge*) do hlavní vývojové větve.

Soubory projektu a informace o jejich historických změnách se nazývá souhrnně *repozitář*. V něm se uchovávají nejen verze souborů, ale také údaje o tom, kdo a kdy změny provedl. To předurčuje verzovací systémy pro efektivní správu týmových projektů, kdy každý člen týmu pracuje na určité části projektu a své změny následně do projektu včleňuje.

My se zaměříme na verzovací systém [Git](#). Ten patří mezi *distribuované* verzovací systémy,⁹ což znamená, že každý uživatel má na svém počítači celý obsah repozitáře a pouze ve chvílích, kdy uzná za vhodné nebo kdy je k tomu příležitost, může své změny synchronizovat se *vzdáleným repozitářem*, ve kterém se shromažďují změny od všech členů týmu, začleňují se do hlavní vývojové větve projektu a hlídají se případné kolize. Výhody tohoto přístupu oproti centralizovaným verzovacím systémům¹⁰ jsou následující:

1. Práce na projektu nevyžaduje připojení k nějakému centrálnímu serveru s repozitářem, a tedy můžete pracovat offline klidně někde v džungli nebo na Marsu.
2. Není potřeba spravovat zvlášť server s repozitářem a zvlášť klientské počítače. Vše je na jednom místě.
3. Při práci na týmovém projektu není případná porucha počítače spojená se ztrátou dat žádná katastrofa, protože ostatní kolegové mají celou kopii repozitáře na svých počítačích.

Git nejefektivněji funguje na textové soubory, ale zvládne verzovat i soubory binární (například obrázky).

Git je v dnešní době jeden z nejpoblárnějších verzovacích systémů. Pod jedho správou jsou vyvíjeny i velké projekty, například samo [Visual Studio Code](#). Tento projekt je navíc otevřený (open source), což znamená, že kdokoli, tedy i vy nebo já, má přístup k celému repozitáři, tedy ke všem

⁹Další distribuované verzovací systémy jsou například [Mercurial](#), [Bazaar](#) či [Darcs](#).

¹⁰Nejznámější reprezentant centralizovaného verzovacího systému je [Subversion](#).

zdrojovým kódům a k jejich kompletní historii. Může se do práce na projektu zapojit, přispět k jeho vývoji a začít psát historii sám.

Program Git pochází z prostředí Linuxu a je navržený tak, aby se s ním dalo pracovat z příkazové řádky (terminálu) pomocí jednoduchých textových příkazů. Tímto způsobem lze používat všechny dostupné funkce Gitu. My se s nejdůležitějšími funkcemi seznámíme právě pomocí textových příkazů, protože na nich se nejlépe naučí, jak tento Git funguje a jaké jsou jeho možnosti. Jakmile si člověk ujasní principy verzování pomocí Gitu, může využít některý z množství nástrojů a doplňků pro různé programy, které práci s repozitáři usnadní a zrychlí. Obsluha Git repozitářů je integrována i do vývojového prostředí Visual Studio Code, ve kterém navíc existuje výborný doplněk [GitLens](#).

Na stránkách projektu Git najdete [podrobný interaktivní návod](#) ke všem funkcím verzovacího systému (a to částečně i v [češtině](#)).

3.1.1 Prvotní nastavení

Git nelze používat do té doby, než jsou nastaveny základní informace o uživateli. To se provede pomocí příkazů v příkazové řádce (terminálu)

```
git config --global user.name "..."  
git config --global user.email "..."
```

přičemž za ... doplníte své jméno (přezdívkou) a email. Těmito údaji se podepisují všechny vámi zapsané změny v repozitáři. Pokud tedy spolupracujete na projektu s jinými lidmi, je dobré zvolit takové údaje, pomocí kterých vás kolegové snadno identifikují a úspěšně kontaktují.

Výpis všech nastavení získáte příkazem

```
git config --list
```

Uvidíte, že v mezi nastaveními je i název hlavní větve repozitáře (`init.defaultbranch=main`) a cesta k nastavenému textovému editoru (v našem případě Visual Studio Code).

Všechna nastavení a práci s příkazem `config` najdete v tomto [podrobném návodu](#) nebo zadáním příkazu

```
git help config
```

3.1.2 Čtyři možné stavy souborů v repozitáři

1. *Nesledovaný (untracked)*: Systém Git historii tohoto souboru neuchovává. Každý nově vytvořený soubor je v tomto stavu. Speciální podskupinu tvoří *ignorované (ignored)* soubory, což bývají v drtivé většině pomocné soubory nebo soubory s citlivými údaji, například přístupovými tokeny nebo přímo hesly. Seznam ignorovaných souborů se zapisuje do speciálního souboru `.gitignore`, o kterém pojednává sekce [3.1.4](#).
2. *Připravený k zapsání (staged)*: Soubor je systémem Git sledován (verzován). Od předchozího zapsání v něm došlo ke změnám (nebo byl nově vytvořen) a k zapsání byl připraven pomocí příkazu

```
git add soubor
```

Hromadný příkaz

```
git add *
```

připraví k zapsání všechny změněné soubory a všechny nové soubory kromě těch, které jsou uvedeny v souboru `.gitignore` (viz sekce [3.1.4](#)).

Přehled změn v souborech připravených k zapsání oproti naposledy uložené verzi dá příkaz

```
git diff --staged
```

Verze souborů se změnami připravenými k zapsání jsou uloženy mimo pracovní adresář: při dodatečných modifikacích souboru se k těmto verzím souboru lze vrátit příkazem

```
git restore --staged soubor
```

(pozor, tímto příkazem se nenávratně ztratí všechny změny v souboru, které byly provedeny od jeho poslední přípravy k zapsání).

3. *Změněný (modified)*: Soubor je systémem Git sledován (verzován). Od předchozího zapsání v něm došlo ke změnám, avšak soubor ještě nebyl označen k zapsání. Přehled změn v souborech oproti naposledy uložené verzi či oproti verzi připravené k zapsání dá příkaz

```
git diff
```

4. *Zapsaný (committed)*: Soubor je systémem Git sledován (verzován) a jeho historie je zaznamenána v repozitáři. Jedná se o verzi souboru, ke které je vždy možné se vrátit. Zapsání (commit) všech souborů připravených k zapsání (staged) se provede příkazem

```
git commit [m "popis revize"]
```

kde "popis revize" je povinné označení ukládaného historického snímku. Pokud se neuvede část v hranatých závorkách, systém Git spustí textový editor zvolený při instalaci a v něm otevře soubor, do kterého název revize a případně nějaké podrobnější komentáře uvedete. Po uložení souboru a uzavření textového editoru se provede zapsání.

Každé zapsání ukládá úplný obraz celého repozitáře a lze se k němu kdykoliv vrátit.

3.1.3 Životní cyklus souborů v repozitáři

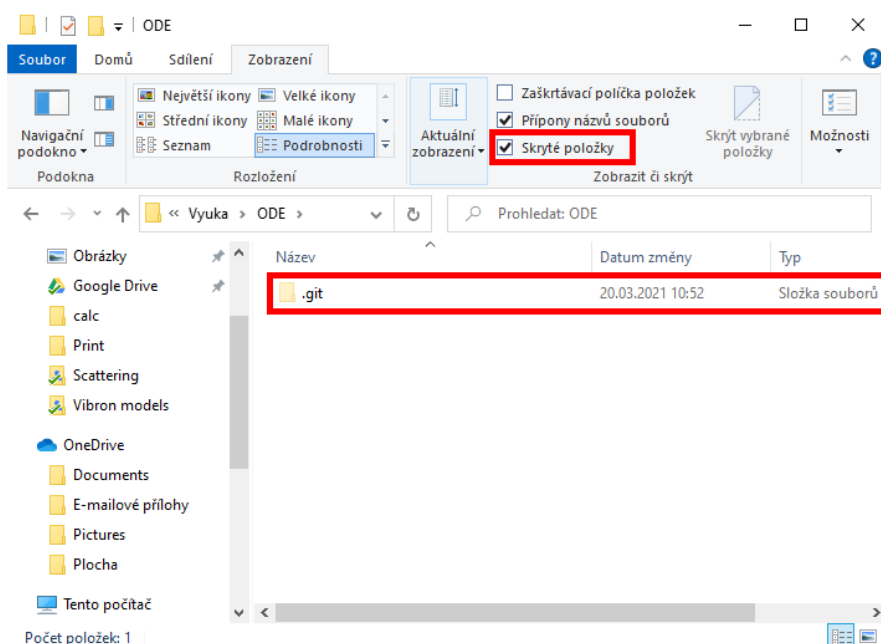
Práci s repozitářem budeme demonstrovat na příkladu projektu pro integraci diferenciálních rovnic ze sekce 4.

3.1.3.1 Vytvoření nového repozitáře

Ve složce, jejíž historii chceme ukládat, se repozitář vytvoří jednoduchým příkazem

```
git init
```

Existenci repozitáře lze vždy odhalit pomocí skryté podsložky s názvem `.git`:



V této složce je uložena veškerá zapsaná historie vašeho projektu od jeho vytvoření. Pokud tuto složku smažete, přijdete tím o „paměť“ systému Git pro daný projekt, avšak aktuální soubory projektu zůstanou tak, jak jsou. Pokud naopak přesunete nebo přepokopírujete složku s projektem včetně podsložky `.git` kamkoliv jinam, klidně na jiný počítač, kopírujete zároveň celou historii projektu. Nedoporučuji jakkoliv měnit soubory v této skryté složce, protože tím můžete nevratně narušit integritu repozitáře a systém Git přestane být schopen repozitář používat.

3.1.3.2 Stav repozitáře

Aktuální stav repozitáře kdykoliv zjistíte pomocí

```
git status
```

Příkaz vypíše název aktuální větve a stav všech souborů v projektu. Pro čerstvě vytvořený repozitář se dozvíte něco takového:

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

3.1.3.3 Vytvoření a první zapsání nového souboru

Ve složce s projektem otevřeme Visual Studio Code (napsáním code do příkazové řádky nebo stiskem pravého tlačítka myši nad otevřenou složkou a výběrem „Open with Code“), vytvoříme nový soubor a uložíme ho jako `ode.py`. Příkaz `git status` ukáže

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  ode.py

nothing added to commit but untracked files present (use "git add" to track)
```

Systém Git si je vědom toho, že si uživatel nemusí pamatovat všechny příkazy, a proto se snaží práci usnadnit a napovídá, které další akce lze v další fázi použít a jak.

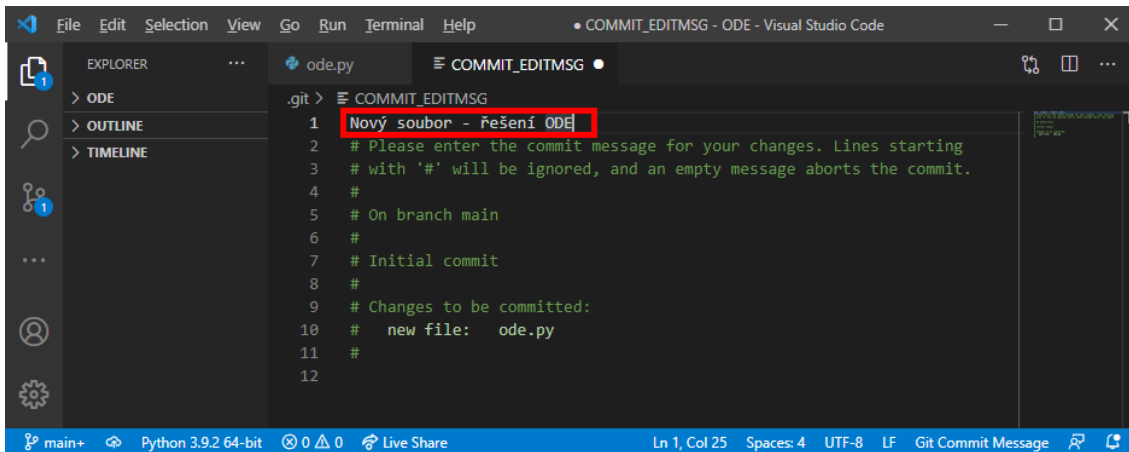
Příkazem `git add *` se nový soubor připraví k zapsání. Stav repozitáře se změní takto:

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
  new file:   ode.py
```

Zapsání se provede příkazem `git commit`. Otevře se editor (nový soubor ve Visual Studiu Code), do kterého uvedeme stručný a výstižný popis změn:



Po uložení a zavření souboru se vytvoří první snímek historie našeho projektu. V příkazové řádce vidíme stručný souhrn, co se zapsalo.

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git commit
hint: Waiting for your editor to close the file...
[main (root-commit) 1b69421] Nový soubor - řešení ODE
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 ode.py

D:\Pavel\Documents\Fyzika\Vyuka\ODE>git status
On branch main
nothing to commit, working tree clean
```

3.1.3.4 Změny v souboru

Do souboru `ode.py` vepíšeme první část kódu, kterým bude řešení jedné diferenciální rovnice Eulerovou metodou 1. řádu.

```
def euler_1(model, y, t, dt):
    y1 = y + model(y, t) * dt
    t1 = t + dt
    return y1, t1

def ode_solve(model, initial_condition, integrator=runge_kutta_4, dt=0.1, maxt=10):
    y = initial_condition          # Initial conditions
    ys = [y]                       # List with results

    t = 0                          # Actual time
    ts = [t]                       # List with times

    while t < maxt:
        y, t = integrator(model, y, t, dt) # Step

        ys.append(y)                # Store position
        ts.append(t)                # Store time

    return ys, ts

def relaxation(y, t):
    return -y

ys, ts = ode_solve(relaxation, 1)
```

Po spuštění kód vyřeší diferenciální rovnici relaxace (17) a v proměnných `ys` a `ts` vrátí seznam y -ových hodnot a odpovídajících časových okamžiků t .

Kód funguje a dává smysluplné výsledky ($y(t)$ klesá s rostoucím časem k nule), zapíšeme tedy změny do „historie“. Stav repozitáře je

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ode.py

no changes added to commit (use "git add" and/or "git commit -a")
```

Soubor je ve stavu změněný (modified). Před zapsáním je nutné jeho stav změnit na k zapsání (staged) pomocí příkazu `git add *`. Lze také použít zkrácený příkaz

```
git commit -a
```

který všechny změněné soubory převede do stavu k zapsání a rovnou zapsání provede. Změnu označím popisem „Eulerova metoda 1. řádu“.

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git commit -a
hint: Waiting for your editor to close the file...
[main 509b08b] Eulerova metoda 1. řádu
1 file changed, 27 insertions(+)
```

Dobrá programátorská praxe je zapisovat vždy jen takový kód, který lze spustit, tj. který neobsahuje žádné syntaktické chyby.

3.1.3.5 Přidání dalšího souboru do projektu

V programu na řešení diferenciálních rovnic se bude hodit oddělit univerzální výpočetní funkce od funkcí specifických pro daný model. Vytvoříme tedy nový soubor **relaxation.py**, do kterého přesuneme kód

```
import ode

def relaxation(y, t):
    return -y

ys, ts = ode.ode_solve(relaxation, 1)
```

Nesmíme zapomenout náš „modul“ naimportovat pomocí příkazu `import ode`. Program lze spustit a funguje, je tedy dobrý čas tyto změny zapsat. Nejdříve se ale podíváme na stav repozitáře:

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ode.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/
        relaxation.py

no changes added to commit (use "git add" and/or "git commit -a")
```

Soubor **ode.py** se změnil, soubor **relaxation.py** byl vytvořen a zatím je ve stavu nesledován. Navíc se ještě vytvořila celá složka **__pycache__**.¹¹ Jedná se o pomocnou složku, kterou verzovat nechceme. Toho se docílí buď tím, že ji ponecháme v nesledovaném stavu (untracked) a začneme sledovat jen nový soubor. Tento postup však znemožní používání hromadných příkazů `git add *` či `git commit -a`. Gitu lze speciálně naznačit, jaké soubory a jaké složky jsou jen pomocné, a má je tedy ignorovat. Za tím účelem se vytvoří speciální soubor pojmenovaný **.gitignore**, do kterého zapíšeme následující pravidlo

¹¹Pro urychlení provádění celého programu ukládá do tohoto adresáře interpret Pythonu speciálně předpřipravené používané moduly.

__pycache__/

a uložíme. Stav repozitáře se změní následovně:

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ode.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        relaxation.py

no changes added to commit (use "git add" and/or "git commit -a")
```

Pomocná složka už není uvedena mezi nesledovanými soubory a Git ji ignoruje. Můžeme již připravit k zapsání všechny nové a změněné soubory pomocí hromadného příkazu `git add *` a následně zapsat všechny změn pomocí `git commit`, přičemž změnu pojmenuji „Oddělení řešitele ODE a modelu“.

3.1.3.6 Zobrazení historie revizí

Historie zobrazíme příkazem

`git log`

```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git log
commit 419e936eac9715111090af19b7e347b31a12ff29 (HEAD -> main)
Author: Pavel Stránský <pcfyzika@pavelstransky.cz>
Date:   Tue Mar 30 10:42:13 2021 +0200

    Oddělení řešitele ODE a modelu

commit 509b08b5551c9b48362b7c31690e04f1947c5f6b
Author: Pavel Stránský <pcfyzika@pavelstransky.cz>
Date:   Tue Mar 30 10:38:36 2021 +0200

    Eulerova metoda 1. řádu

commit 1b694213d728693b000e269a82c972f26f4214c4
Author: Pavel Stránský <pcfyzika@pavelstransky.cz>
Date:   Tue Mar 30 10:35:05 2021 +0200

    Nový soubor - řešení ODE
```

Vypsání seznam obsahuje všechny údaje o jednotlivých snímcích historie (zapsáních), tj. datum a čas, autora a jeho e-mail a hash kód podepisující zapsání.

Příkaz `log` má velké možnosti, jak formátovat a filtrovat informace o zapsáních, přičemž všechny jsou uvedeny v [referenční příručce](#). Nejdůležitější jsou

- `git log --oneline` naformátuje vše hezčím způsobem (každé zapsání na jednom řádku) a z hashe zobrazí jen prvních 7 znaků, což stačí k identifikaci jednotlivých zapsání,
- `git log -2` vypíše jen poslední dva záznamy,
- `git log --stat` vypíše statistiky o jednotlivých verzích (počty změn a výpis změněných souborů),
- `git log --graph` zobrazí výsledek „graficky“, což se hodí v případě více větví (větvě budou vysvětleny v sekci [3.1.5](#)),
- `git log --follow ode.py` vypíše všechny historické změny v souboru `ode.py`,
- `git log --since "2 years 1 day 3 minutes ago"`

- `git log --until "2019-01-15"` vypíše všechny změny od, resp. do uvedeného časového údaje,
- `git log --author="Pavel"` vypíše všechny změny, které zapsal uživatel Pavel,
- `git log --grep="Euler"` vypíše všechny změny, které obsahují v popisu slovo „Euler“.

Uvedené filtry lze samozřejmě libovolně kombinovat.

3.1.4 .gitignore

Překladače programovacích jazyků často vytvářejí v adresáři projektu dočasné pomocné soubory, které nechcete, aby se staly součástí repozitáře (tyto soubory nenesou žádnou relevantní informaci, navíc mohou na různých počítačích vypadat jinak podle toho, jaký překladač či jaké vývojové prostředí zrovna použijete). Abyste mohli používat příkazy pro hromadné sledování či zapisování souborů `git add *` či `git commit -a`, musíte GITu naznačit, jaké soubory má ignorovat. K tomu slouží soubor `.gitignore`.

Každé pravidlo v souboru `.gitignore` zabírá jeden řádek. Řádek, který začíná znakem `#`, je ignorován a může sloužit například jako komentář. Příklady jednotlivých řádků:

- `tajne.txt`
Ignoruje soubor s názvem `tajne.txt` (může obsahovat třeba přihlašovací údaje k nějaké službě a ty rozhodně nechceme sdílet ani archivovat; nezapomeňte, že co je jednou zapsané v repozitáři, z něj až na výjimky nelze odstranit).
- `*.log`
Ignoruje všechny soubory s příponou `log`,
- `!important.log`
avšak neignoruje soubor `important.log`.
- `*.[oa]`
Ignoruje všechny soubory s příponou `o` nebo `a`.
- `temp/`
Ignoruje všechny soubory v podsložce `temp`.
- `doc/**/*.pdf`
Ignoruje všechny soubory s příponou `pdf` v podsložce `doc` a ve všech jejích podsložkách. Neignoruje však soubory s příponou `pdf` v kořenové složce projektu.

Další příklady použití souboru `.gitignore` jsou například [na této stránce](#).

Pokud na GitHubu zakládáte nový projekt, můžete upřesnit, jaký programovací jazyk budete používat a GitHub automaticky vytvoří optimální soubor `.gitignore`.

Úloha 3.1: Podívejte se do souboru `.gitignore` v repozitáři k těmto zápiskům. Zatímco Python si téměř žádné pomocné soubory nevytváří, L^AT_EX jich generuje požehnaně. Proto je seznam ignorovaných souborů celkem dlouhý.

3.1.5 Větve

Verzovací systém Git umožňuje pracovat paralelně na několika různých verzích kódu a mezi nimi dokáže efektivně přepínat. Větvení se hodí v případě, že chceme vyzkoušet více různých přístupů k řešení úlohy. Pro každý nápad vytvoříme vlastní větev úprav, což dovolí řešení porovnávat a nakonec vybrat to, které vyhovuje nejvíce. Každou větev můžeme dále větvit. Změny v perspektivních větvích lze následně začleňovat do hlavní větve, neperspektivní větve lze nechat být nebo odstranit.

Častá programátorská praxe je taková, že v hlavní větvi¹² se nachází vždy jen odladěná a otestovaná verze programu. Pro každou změnu v projektu se vytvoří nová větev. Na té se může pracovat hodinu, ale třeba několik měsíců. V této „vývojové“ větvi se změny provedou, odladí, pošlou testerům, a teprve po důkladném otestování se začlení do hlavní větve. Mezitím je stále k dispozici funkční program hlavní větve, který lze bez obav používat k ostrému provozu.

3.1.5.1 Vytvoření nové větve

Budeme pokračovat ve vývoji programu pro řešení obyčejných diferenciálních rovnic. Předpokládejme, že řešení pro jednu rovnici máme hotové a chceme ho rozšířit tak, aby fungovalo i na soustavu libovolného počtu diferenciálních rovnic 1. řádu, kterým se věnuje sekce 5. K tomu vytvoříme novou větev `odesystem`

```
git branch odesystem
```

a přepneme se na ni příkazem¹³

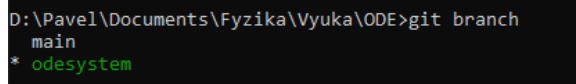
```
git switch odesystem
```

Zpět na hlavní větev se lze vrátit příkazem `git switch main`.¹⁴

3.1.5.2 Výpis větví

Seznam všech větví se vypíše příkazem

```
git branch
```



```
D:\Pavel\Documents\Fyzika\Vyuka\ODE>git branch
main
* odesystem
```

Aktivní větev je vyznačena hvězdičkou i barevně.

Další užitečná upřesnění výpisu větví jsou

- `git branch --merged` vypíše jen větve spojené s aktuální větví,
- `git branch --no-merged` vypíše jen větve nespojené s aktuální větví.

3.1.5.3 Změny a sloučení větví

Poté, co provedeme a odladíme změny související s rozšířením programu na výpočet soustavy diferenciálních rovnic ve větvi `odesystem`, můžeme tyto změny začlenit do hlavní větve `main`. K tomu slouží příkaz

```
git merge odesystem
```

spuštěný v hlavní větvi. Pokud během slučování nedojde k žádným konfliktům, vše proběhne hladce. Může se však stát, že byly provedeny změny na stejném místě jednoho souboru v obou větvích (konflikt). V tom případě Git označí konflikty v souborech a jejich stav změní na modifikovaný. Pak je potřeba tyto soubory projít, vybrat z konfliktních verzí jednu, případně kód upravit tak, aby odrazil obě konfliktní změny, a zapsat příkazem `git commit -a`. Tím je slučování dokončeno.

¹²Hlavní větev se donedávna pojmenovávala `master`, dnes se spíše prosazuje pojmenování `main`.

¹³Vytvoření nové větve a přepnutí se do ní lze jedním příkazem `git switch -c odesystem`.

¹⁴Git umožňuje přepínání mezi větvemi také pomocí příkazu `git checkout`.

3.1.5.4 Odstranění větve

Větev odstraníme prostým

```
git branch -d odesystem
```

Odstraňovat větev má smysl jen v případě, kdy jsme přesvědčeni, že změny v ní provedené byla slepá cesta, nebo pokud provedené změny byly začleněny do jiné větve.

3.1.6 Návrat k dřívějším verzím

Pozor, při neopatrném použití těchto příkazů hrozí ztráta dat! Pokud si budete následující příkazy zkoušet, doporučuji vám zazálohovat si celou složku s projektem včetně repozitáře (složky `.git`).

- `git commit --amend` umožňuje změnit popis posledního zapsání.
- `git restore soubor` vrátí změny uvedeného souboru na poslední zapsanou verzi (lze použít i hromadné `git restore *`¹⁵).
- `git reset HEAD~1` zruší poslední¹⁶ zapsání, avšak soubory ve složce projektu ponechá tak, jak jsou.¹⁷
- `git reset --hard HEAD~1` zruší poslední zapsání a soubory projektu uvede do stavu, v jakém byly na začátku této poslední revize. Všechny provedené změny jsou nenávratně ztraceny. Pokud byl nějaký soubor v poslední revizi vytvořen, je smazán. Toto je nejnebezpečnější příkaz, proto používejte jen v opravdu odůvodněných případech.
- `git reset 946cb25` se vrátí na zapsání, jehož hash začíná uvedeným číslem.
- `git reset ORIG_HEAD` se vrátí na tu zapsanou verzi, na které jsme byli bezprostředně před použitím posledního příkazu `git reset`.
- `git checkout -b oldrev 946cb25` vytvoří novou větev nazvanou „oldrev“, ve které budou soubory ve stavu, kdy bylo provedeno zapsání s uvedeným hashem (hash zjistíme například pomocí `git log --oneline` popsaného v sekci 3.1.3.6).
- `git checkout 946cb25` vrátí se do tohoto zapsání, ale ponechá historii tak, jak je.¹⁸ K návratu zpět stačí použít příkaz pro přepnutí do hlavní větve `git switch main`.

3.1.7 Vzdálené repozitáře

Verzovací systém GIT umí kromě sledování a uchovávání historie změn souborů vašeho projektu i koordinovat změny, které provádí na projektu více řešitelů (pracovní tým). K tomu slouží vzdálené repozitáře. Mezi nejrozšířenější systémy patří

- **GitHub**: Na něm si každý může zdarma zřídit vlastní repozitáře a používat je například k synchronizaci svého projektu mezi různými počítači, pro sdílení vlastních výtvorů s komunitou nebo právě pro spolupráci ve vlastním týmu. Vlastníkem firmy vyvíjející tento systém je Microsoft, díky čemuž je základní práce s ním integrována do vývojového prostředí Visual Studio Code.
- **GitLab**: Open source řešení, základní verze rovněž zdarma, k pokročilé verzi má MFF UK licenci.

¹⁵`git restore *` se chová podobně jako `git reset --hard`.

¹⁶Číslo udává, o kolik revizí se vracíme zpět.

¹⁷HEAD vlastně značí ukazatel na poslední zapsání.

¹⁸Tzv. detached HEAD state.

Oba systémy disponují webovým rozhraním, ze kterého lze projekty jednoduše spravovat, a rovněž desktopovými aplikacemi, které umožňují pracovat s lokálními repozitáři pomocí jednoduchého rozhraní. Pro GitHub existuje například [GitHub Desktop](#). Projekty mohou být buď soukromé (přístup k nim máte pouze vy či ti, kterým pošlete pozvánku), nebo veřejné (přístup má zcela kdokoli).

Tyto zápisky a vzorové ukázky kódů jsou veřejně na GitHubu a můžete k nim dostat na adrese <https://github.com/PavelStransky/PCInPhysics>. K dispozici je samozřejmě celý repozitář s historií se všemi commity a se všemi vývojovými větvemi. Repozitář můžete stáhnout buď z uvedené webové stránky (zelené tlačítko **Clone or download**), nebo pomocí programu `git`.

- `git clone https://github.com/PavelStransky/PCInPhysics`

V aktuální složce vytvoří podsložku `PCInPhysics` a do ní stáhne celý repozitář. V případě těchto zápisků se jedná o tento soubor v $\text{\LaTeX}$ u, výsledné PDF, EPS či PNG verze všech obrázků a všechny zdrojové kódy.

- `git remote -v`

V adresáři s lokálním repozitářem ukáže, na jaký vzdálený repozitář je navázán. Stejně jako základní větev se standardně jmenuje `main`, vzdálený repozitář se standardně jmenuje `origin`. Vy můžete mít na jeden projekt navázáno více vzdálených repozitářů, každý pak samozřejmě musíte pojmenovat jinak.

- `git remote add origin https://github.com/Uzivatel/VzdalenyRepozitar`

Přidá do vašeho projektu odkaz na vzdálený repozitář umístěný na GitHubu.

- `git remote show origin`

Zobrazí informace o vzdáleném repozitáři.

- `git pull`

Do adresáře s lokálním repozitářem stáhne aktuální verzi aktuální větve. Pokud máte lokálně rozpracované změny, stažení se nepovede. Pokud máte uložené změny (`commit`), `git` se automaticky pokusí vaše změny sloučit se změnami v globálním repozitáři (`merge`).

- `git push origin master`

Do vzdáleného repozitáře `origin` zapíše vaši větev `master`. Vzdálený repozitář může být nastaven tak, že vaše změny musí ještě někdo schválit.

- `git push`

Zkrácený zápis, pokud jste dříve nastavili pomocí příkazu `git push -set-upstream origin master` název lokální větve a příslušného vzdáleného repozitáře.

Další informace najdete například v [dokumentaci](#).

Úloha 3.2: *Stáhněte si do svých počítačů (naklonujte si) z GitHubu repozitář s poznámkami k tomuto cvičení. V budoucnu si pomocí příkazu `git pull` stahujte aktuální verze. Můžete si vytvořit pracovní větev poznámek a v ní si s kódy hrát. V hlavní větvi `main` se vám uchová originální verze ze vzdáleného repozitáře.*

Úloha 3.3: *Vytvořte si účet na GitHubu, vytvořte prázdný projekt a navažte si ho s dříve na cvičení vytvořeným lokálním repozitářem pomocí příkazů `git remote add` (plný příkaz výše). Následně si do vzdáleného repozitáře nahrajte lokální repozitář pomocí příkazu `git push`.*

3.1.8 Tahák

Pro snadnou orientaci v použití Gitu bez nutnosti pamatovat si všechny příkazy je dostupný [Git Cheat Sheet](#) nebo [tento interaktivní web](#).

3.2 Programovací jazyk Python

Python je v dnešní době velmi populární programovací jazyk, který pronikl do spousty rozmanitých odvětví, a to zejména díky bohatosti a pokročilosti knihoven, které jsou vyvíjeny komunitou a jsou díky tomu aktuální a odladěné (či průběžně odladované). Python existuje na mnoha platformách od osobních počítačů po mikrokontroléry a dá se v něm naprogramovat téměř cokoliv: pokročilé vědecké výpočty používající nejnovější numerické algoritmy, dobře vypadající grafy, statistická analýza, analýza velkých dat a strojové učení, ale také webové služby, okénkové programy či editace obrázků a videí. Je běžné, že i komerční programy obsahují Python coby skriptovací jazyk, čímž umožňují uživateli používat své vlastní kódy „uvnitř“ komerčního produktu a integrovat různé programy mezi sebou.¹⁹

Python lze charakterizovat jako jazyk:

- *Interpretovaný*, což znamená, že provádění programů probíhá přímo ze zdrojového kódu.²⁰ Ke spuštění kódu je tedy potřeba mít nainstalovaný interpret, což jsme učinili v sekci 2.1. Pokud kód obsahuje syntaktickou chybu, je odhalena až ve chvíli, kdy se na ni interpret při provádění kódu narazí. U interpretovaných jazyků se neztrácí čas překladem do strojového kódu a je pro ně přirozené dynamické typování proměnných. Tím, že interpret čte text kódu příkaz po příkazu, je provádění programů pomalejší, ale vzhledem k tomu, že velká část časově náročných funkcí a knihoven bývá napsána v rychlejších programovacích jazycích, není to zásadní nevýhoda. Kvůli skutečnosti, že Python se obvykle používá jako mezičlánek spojující efektivně napsané funkce různých knihoven, setkáte se také s označením jazyk *skriptovací*.

Opakem interpretovaných jazyků jsou jazyky kompilované, přičemž kompilace se provádí buď přímo do strojového kódu daného procesoru²¹ nebo do mezikódu. Mezikód ke spuštění vyžaduje dodatečnou kompilaci, která však může být vysoce optimalizovaná přímo pro danou hardwarovou konfiguraci použitého počítače.^{22,23}

- *Dynamicky typovaný*, což znamená, že proměnná nemusí mít předem daný typ a typ proměnné se může za běhu programu dokonce měnit.
- *S automatickou správou paměti*, takže programátor se nemusí starat o alokování a uvolňování paměti. Python při inicializaci proměnné paměť alokuje automaticky a ve chvíli, kdy proměnnou přestaneme používat, paměť uvolní.²⁴
- V Pythonu lze pythonovským způsobem používat tři základní programovací paradigmat: *procedurální*, *objektové* a *funkcionální*.

Python klade důraz na jednoduchost, stručnost a čitelnost kódu. Filosofie programovacího jazyka je shrnuta v [Zenu Pythonu](#),²⁵ se kterým vám doporučuji se seznámit.

3.2.1 Vytvoření a spuštění kódu

Ve Visual Studio Code vytvoříme nový soubor a uložíme si ho s příponou *.py. Podle této přípony VS Code pozná, že se jedná o pythonovský kód a použije k práci s ním správný doplněk.

Hotový kód lze spustit ve VS Code jedním z následujících tří způsobů.

1. *Kliknutím na šipku na nástrojové liště*: Tímto způsobem se spustí celý soubor kódu.

¹⁹Jako příklad poslouží nejnovější verze programu [Mathematica](#), [Origin](#) či [SAS](#).

²⁰Mezi další známé interpretované jazyky patří například JavaScript nebo PHP.

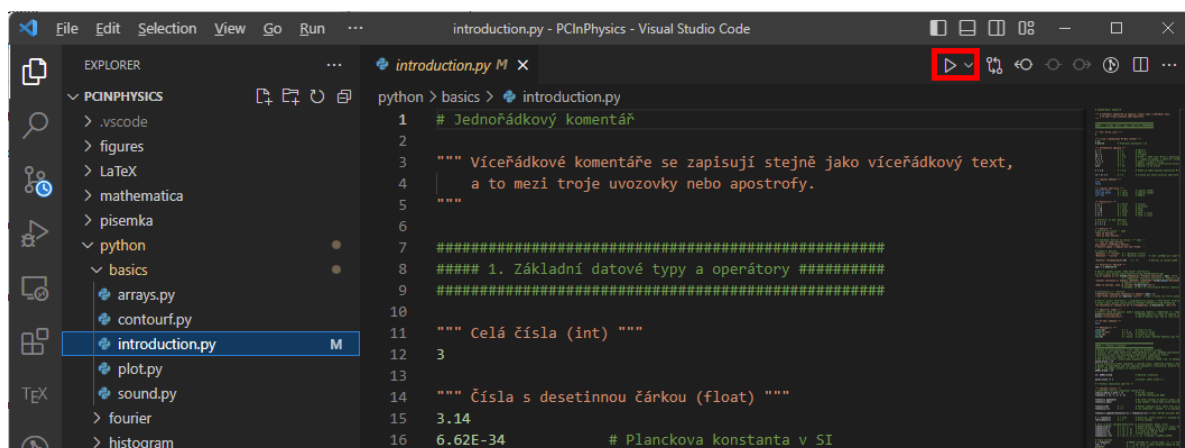
²¹Například C/C++, Pascal nebo Fortran.

²²Zde se jedná například o jazyky Java, C# a celý .NET framework nebo Julia.

²³Pro Python existuje knihovna [Numba](#), která také umožňuje překlad vybraných částí programu do strojového kódu pomocí LLVM překladače, a tím provádění programu výrazně zrychlit.

²⁴Algoritmus se nazývá *Garbage collection*.

²⁵Zen se také vypíše, pokud do svého kódu zadáte `import this`.



2. *Stiskem F5*: Kód se spustí v režimu ladění (debugger). Zastaví se na každém kontrolním bodě (breakpoint, který se vloží na vybranou řádku kódu klávesou F9) nebo na každé chybě. Pro pokračování provádění kódu stačí stisknout buď znovu F5, nebo F10 či F11 pro jeden krok.
3. *Označením části kódu a stiskem Shift+Enter*: V okně **TERMINAL** spustí REPL Pythonu a v něm označenou část kódu. REPL se po provedení kódu nezavře, takže v něm lze psát dodatečné příkazy. Pro ukončení REPL do něj stačí napsat `exit()` nebo v něm stisknout **Ctrl+Z** a potvrdit nebo ukončit okno terminálu kliknutím na ikonku popelnice.

3.2.2 Základy syntaxe Pythonu

Soubor se vzorovými příklady najdete v repozitáři ke cvičení: [introduction.py](#).²⁶ Doporučuji vám si ho stáhnout nebo jeho obsah přepísnout a důsledně si ho řádek po řádku projít a spouštět nejlépe pomocí označení a stiskem **Ctrl+Enter**, jak bylo popsáno v předchozí sekci. Některé části kódu odkazují na proměnné zavedené v dřívější části souboru, proto kód procházejte postupně od začátku do konce.

Z vzorového kódu bych vypíchl následující body:

1. *Základní datové typy a operátory*: Python obsahuje jen dva základní číselné typy: `int` a `float`. Jejich použití je podobné jako v jiných programovacích jazycích.
2. *Formátování řetězců*: Python umožňuje několik různých způsobů, jak formátovat řetězce (vkládat do nich čísla či jiné proměnné). Doporučuji se seznámit s velmi čitelným způsobem pomocí `f`-řetězce. Více informací k formátování řetězců naleznete v tomto výborném [návodu](#).
3. *Proměnné a kolekce*: Důležité standardní kolekce jsou *seznam* (list) `[...]` a *slovník* (dictionary) `{...}`. Dále existuje typ *n-tice* (tuple) `(...)` a *množina* (set) `{...}`. O jaký typ kolekce se jedná poznáte podle typu závorek, kterými je uzavřena. Python nabízí bohaté možnosti indexování prvků kolekcí.

Python nemá typ „řada“ (jedno či vícerozměrný soubor hodnot stejných typů). Ten je implementován až v rozšiřujících knihovnách, například v knihovně **numpy**, které je věnována sekce [3.2.4](#).

4. *Podmínky a cykly*: Příkaz uvozující blok kódu vždy končí znakem `„:“`. Každý blok je definován svým odsazením, které musí být na každém řádku stejné (tj. musí obsahovat stejný počet odsazujících znaků, mezi které patří buď mezera nebo tabulátor). Konvence je používat k odsazení [4 mezery](#).

Cykly jsou možné pouze přes iterovatelné objekty. Pro cyklus přes přirozená čísla (indexy) musíme vytvořit odpovídající iterovatelný objekt příkazem `range`. V drtivé většině případů se lze při programování v Pythonu indexům zcela vyhnout.

²⁶Vzorový soubor je inspirován příkladem [Nauč se Python v Y minutách](#).

5. *Funkce*: Python umožňuje velkou variabilitu co se týče argumentů funkce a návratových hodnot díky své funkci automatického sbalení a rozbalení kolekcí. Funkce se navíc chová jako objekt, lze ji tedy přiřadit jakémukoli proměnné. To je jeden z prvků funkcionálního programování.

Funkcionální programování také obsahuje koncept anonymní funkce, což je jednoduchá funkce definovaná na jednom řádku, která nemá vlastní jméno. V Pythonu se vytváří pomocí klíčového slova `lambda`.

6. *Moduly*: Při programování je důležité vhodně strukturovat kód. Python obsahuje jednoduchý koncept modulů, přičemž každý soubor s příponou `*.py` lze použít jako modul. Modul je vlastně speciální typ objektu.

Dobře se seznámte se způsoby, jak modul načíst a používat, jelikož většina funkcí Pythonu se nachází právě v modulech. Jedná se například o matematické funkce v modulu `math` nebo rozšiřující knihovny `numpy`, `scipy` a `matplotlib`.

7. *Třídy*: Python umožňuje objektově orientované programování. Práce s třídami se však v mnohém liší od striktně objektově orientovaných programovacích jazyků, jakými jsou například C++, C# nebo Java. Důležitý rozdíl je například v přístupnosti atributů (všechny atributy v Pythonu jsou veřejné). Rovněž dědičnost a dědění metod se chová odlišně. Dále je nutné pamatovat na to, že každá metoda třídy musí mít v deklaraci jako první argument odkaz na instanci, se kterou je volána (konvenčně se označuje `self`). Ve vzorovém kódu je k objektovému programování opravdu jen to nejnutnější minimum. Pokud chcete v Pythonu využívat objekty seriózně, doporučuji pročíst si odpovídající [kapitulu v manuálu](#).

Uvedený kód s příklady obsahuje jen ty struktury jazyka Python, které budeme používat. V budoucích cvičeních se ještě seznámíte se *správou kontextu* (klíčové slovo `with`). Python umožňuje navíc

- ošetření chyb pomocí *výjimek* (klíčová slova `raise`, `try`, `except`, `finally`),
- tvorbu *generátorů* (klíčové slovo `yield`),
- *asynchronní programování*, korutiny a úkoly (klíčová slova `await`, `async`)
- či tvorbu a použití *dekorátorů*.

Pro plné ovládnutí jazyka vám doporučuji se v budoucnu s těmito koncepty seznámit, a to buď na dedikovaných přednáškách, z tutoriálů a návodů na webu, nebo studiem cizích kódů.

Na oficiálních stránkách Pythonu najdete [tutoriál k nejnovější verzi programovacího jazyka](#).

3.2.3 Pojmenování proměnných a formátování

Formátování kódu Pythonu je celkem volné. Povinné je dodržet jen správné odsazení bloků. Pro snazší čitelnost kódu byla nicméně vytvořena řada doporučení, která najdete souhrnně na stránce [PEP 8](#).²⁷ Python obsahuje dokonce knihovnu `autopep8`, která váš zdrojový soubor podle těchto pravidel naformátuje. Automatické formátování podle PEP 8 lze nastavit i ve [Visual Studio Code](#).

Pro pojmenování proměnných existuje jediné závazné pravidlo, které zní, že indentifikátor se nesmí shodovat s žádným z [35 klíčových slov](#) jazyka. Kromě toho jsou ve výše uvedeném dokumentu další nezávazná pravidla k pojmenování proměnných:²⁸

- *Proměnné a funkce* se doporučuje pojmenovávat malými písmeny a jednotlivá slova spojovat podtržítky, tzv. underscore style (například `pocet_bodu`).

²⁷PEP = Python Enhancement Proposal

²⁸Čitelnosti různých stylů pojmenování proměnných se věnují i seriózní vědecké články, například <http://www.cs.kent.edu/~jmaletic/papers/ICPC2010-CamelCaseUnderScoreClouds.pdf>.

- *Konstanty* se doporučuje pojmenovávat velkými písmeny a jednotlivá slova spojovat podtržítky (například `PLANCKOVA_KONSTANTA`).
- *Třídy* se doporučuje pojmenovávat tak, že jednotlivá slova názvu začínají velkým písmenem a mezi nimi není žádný znak, tzv. Pascal style (například `PostovniAdresa`).
- *Moduly* se doporučuje pojmenovávat krátkými výstižnými názvy složenými jen z malých písmen.
- Kvůli čitelnosti je dobré se vyhnout jednopísmenným označením `l`, `I` a `O`, jelikož takto označené proměnné mohou být snadno zaměněny s čísly `1` a `0`.

Pokud vám vyhovuje jiný styl pojmenovávání, lze ho použít. Je však dobré být v pojmenovávání konzistentní napříč celým projektem.

Častá otázka je, zda proměnné označovat *česky* či *anglicky*. Jelikož nikdy nevíte, kdo v dnešním propojeném světě bude chtít váš kód použít a třeba i upravit pro své potřeby, doporučuji používat anglická pojmenování.

Snadná čitelnost kódu je podpořena i správným psaním *komentářů*. Předně se dnes doporučuje vycházet z předpokladu, že dobře napsaný kód je čitelný i bez komentářů. Čitelnost totiž zaručují zejména vhodně a výstižně označené proměnné a správně navržená struktura kódu. Komentovat je dobré jen ty části kódu, kde se používá nějaký ne všeobecně známý trik nebo postup. Nadbytek komentářů je velmi těžké udržovat při úpravách a může vést i k tomu, že po několika změnách kódu nebudou komentáře v souladu s tím, co kód vykonává.²⁹

Naopak doporučené je nešetřit popisem, co dělají jednotlivé funkce, jaké jsou jejich argumenty a co vracejí na výstupu. K tomu slouží komentář typu *docstring*, který je vysvětlený v souboru `introduction.py` v sekci o funkcích.³⁰ Použití *docstringu* usnadní i tvorbu a správu doprovodné dokumentace k celému projektu. Existují dokonce nástroje, které vám z těchto komentářů vytvoří strukturované webové stránky, například nástroj `Sphinx`.

3.2.4 Řady, vektory, matice

Jak bylo zmíněno v předchozí sekci, definice jazyka Python neobsahuje typ *řada*, tj. jedno či vícerozměrné pole hodnot stejného typu. Tento typ a operace s ním jsou implementovány v knihovně `numpy`. Ta není v holé instalaci Pythonu obsažena, je nutné ji doinstalovat podle návodu v sekci 2.1.1. Základy práce s řadami jsou vysvětleny v souboru `arrays.py` v repozitáři.

3.2.5 Grafy

K vykreslení grafů se nejčastěji používá knihovna `Matplotlib`. V základní instalaci Pythonu není obsažena, je nutné ji doinstalovat podle návodu v sekci 2.1.1. Jednoduchý příklad vykreslení grafů je v souboru `plot.py` v repozitáři.

3.2.6 Pseudonáhodná čísla

V Pythonu lze pseudonáhodná čísla generovat pomocí několika knihoven.

- Pro základní použití je k dispozici knihovna `random`. Z ní nejdůležitější funkce jsou tyto:
 - `random()`: Pseudonáhodné číslo x rovnoměrně vybrané z intervalu $x \in \langle 0; 1 \rangle$.
 - `uniform(a, b)`: pseudonáhodné číslo x rovnoměrně vybrané z intervalu $x \in \langle a; b \rangle$.

²⁹A common fallacy is to assume authors of incomprehensible code will somehow be able to express themselves lucidly and clearly in comments [1].

³⁰Stejně jako v mluvených jazycích se zlepšujete čtením literatury, v programovacích jazycích pomáhá čtení cizích kódů. Pro příklad použití komentářů se koukněte třeba na zdrojový soubor funkce `optimize` z knihovny `scipy`.

- `gauss(mu, sigma)`: pseudonáhodné číslo z Gaussovského rozdělení se střední hodnotou `mu` a směrodatnou odchylkou `sigma`.
 - `randint(a, b)`: pseudonáhodné celé číslo d z intervalu $a \leq d < b$.
 - `seed(s)`: nastaví počáteční násadu generátoru podle parametru `s` (pro jednu konkrétní násadu bude generátor dávat stejnou sekvenci čísel). Parametr může být jakéhokoliv typu, tedy číslo, řetězec atd. Pokud se parametr neuvede, použije se jako ná sada aktuální systémový čas, a tudíž bude sekvence pseudonáhodných čísel při každém spuštění programu jiná.
 - `choice(m)`: vybere pseudonáhodně element ze seznamu `m`.
 - `shuffle(m)`: promíchá elementy v seznamu `m`.
- Ke generování řad či vícerozměrných objektů, jejichž elementy jsou náhodná čísla, lze použít i knihovnu **numpy**, viz sekce 3.2.4 a soubor s příklady [numpy.py](#).
 - Pro pokročilejší použití je knihovna `numpy.random`. Ta umožňuje zvolit vlastní generátor pseudonáhodných čísel, generovat čísla z celé řady **statistických rozdělení** a generovat naráz celé vektory či matice.
 - `generator = default_rng()`: Inicializuje standardní generátor pseudonáhodných čísel.
 - `generator = Generator(PCG64())`: Inicializuje specifický generátor pseudonáhodných čísel (v tomto případě PCG-64, což je O’Neillův **permutační kongruenční generátor**).
 - `generator.random(size=10)`: vektor délky 10 s elementy z rovnoměrného rozdělení na intervalu $\langle 0; 1 \rangle$.
 - `generator.normal(size=10)`: vektor délky 10 s elementy z normálního Gaussova rozdělení se střední hodnotou 0 a směrodatnou odchylkou 1.
 - `generator.normal(loc=1, scale=2, size=(10,10))`: matice rozměru 10×10 s elementy z normálního Gaussova rozdělení se střední hodnotou 1 a směrodatnou odchylkou 2.

3.2.7 Práce se zvukem

V Pythonu existuje velké množství knihoven, které umějí pracovat se zvukem. Pro naše účely postačí jednoduchá knihovna [sounddevice](#).³¹ Pokud ji nemáte nainstalovanou, doinstalujete ji příkazem

<code>python -m pip install sounddevice</code>	Windows
<code>pip install sounddevice</code>	Linux, Mac

Z této knihovny využijeme následující funkce:

- `play(data, fs)` přehraje časovou řadu uloženou v poli `data` při vzorkovací frekvenci `fs`. Jsou-li elementy přehrávané řady typu `float`, jejich hodnoty musejí ležet v rozmezí $h_j \in \langle -1, 1 \rangle$, jinak bude zvuk přebuzený a zkreslený. Vzorkovací frekvence zvukových souborů bývají nejčastěji násobky či podíly 44100 Hz nebo 48000 Hz.³²
- Přehrávání probíhá asynchronně. Chceme-li počkat, než bude přehrávání dokončeno, použijeme funkci `wait()`.
- `stop()` okamžitě zastaví přehrávání.

³¹Další knihovny pro přehrávání a nahrávání zvuku jsou uvedeny na <https://realpython.com/playing-and-recording-sound-python>.

³²44100 Hz je vzorkovací frekvence používaná na CD, 48000 Hz na DVD.

- `rec(numsamples, samplerate=fs, channels=2)` nahraje časovou řadu s `numsamples` hodnotami. `channels=1` pro mono signál, `channels=2` pro stereo signál.

Jelikož zde budeme pracovat se zvukovými soubory typu WAV, budeme potřebovat ještě knihovnu [soundfile](#) k jejich načtení.

V [repozitáři](#) je ukázkový soubor [sound.py](#), který načte zadaný WAV soubor a přehraje ho.

4 Obyčejné diferenciální rovnice 1. řádu

V této části se budeme věnovat řešení jedné diferenciální rovnice prvního řádu,

$$\frac{dy}{dt} = f(y, t) \quad (1)$$

s počáteční podmínkou

$$y(t_0) = y_0. \quad (2)$$

Zde $y = y(t)$ je hledaná funkce a t je nezávisle proměnná.

Numerické řešení diferenciální rovnice spočívá v nahrazení infinitezimálních přírůstků přírůstky konečnými:

$$\frac{\Delta y}{\Delta t} = \phi(y, t) \quad (3)$$

kde ϕ je funkce, která udává směr, podél kterého se při numerickém řešení vydáme. Volbá této funkce je klíčová a záleží na ní, jak přesné řešení dostaneme a jak rychle ho dostaneme.

4.1 Pár důležitých pojmů

- **Explicitní algoritmy:** K výpočtu hodnoty funkce y_{i+1} se vyžadují pouze hodnoty z aktuálních a minulých kroků, tj. y_i, y_{i-1} , atd.
- **Jednokrokové algoritmy:** K výpočtu hodnoty funkce y_{i+1} v následujícím kroku vyžadují pouze znalost hodnoty funkce v aktuálním kroku y_i . Rozepsáním (3) dostaneme

$$y_{i+1} = y_i + \underbrace{\phi(y_i, t_i)}_{\phi_i} \Delta t, \quad (4)$$

přičemž počáteční hodnota y_0 je dána počáteční podmínkou. My se omezíme pouze na tyto algoritmy.

- **Lokální diskretizační chyba:**

$$\mathcal{L} = y(t + \Delta t) - y(t) - \phi(y(t), t)\Delta t, \quad (5)$$

kde $y(t)$ udává přesné řešení v čase t .

- **Akumulovaná diskretizační chyba:**

$$\epsilon_i = y_i - y(t_i) \quad (6)$$

- **Řád metody:** Metoda je p -tého řádu, pokud

$$\mathcal{L}(\Delta t) = \mathcal{O}(\Delta t^{p+1}). \quad (7)$$

- **Kontrola chyby řešení:** Chybu numerického řešení diferenciální rovnice lze zmenšit 1) menším krokem, 2) lepší metodou (metodou vyššího řádu). Menší krok však znamená vyšší výpočetní čas. Sofistikované metody proto průběžně mění velikost kroku: když se funkce mění pomalu, krok prodlouží, když se mění rychle, krok zkrátí (tzv. **metody s adaptivním krokem**). Tím se docílí vysoké přesnosti při co nejmenším výpočetním čase.

4.2 Eulerova metoda 1. řádu

$$\phi_i = f(y_i, t_i), \quad (8)$$

tj. krok do y_{i+1} děláme vždy ve směru tečny v bodě y_i .

- Nejjednodušší metoda integrace diferenciálních rovnic.
- Chyba je obrovská, k dosažení přesných hodnot je potřeba velmi malého kroku, což znamená dlouhý výpočetní čas.

4.3 Eulerova metoda 2. řádu

$$\begin{aligned} k_1 &= f(y_i, t_i) \\ k_2 &= f(y_i + k_1 \Delta t, t + \Delta t) \\ \phi_i &= \frac{1}{2} (k_1 + k_2), \end{aligned} \quad (9)$$

tj. uděláme jednoduchý Eulerův krok ve směru k_1 , spočítáme derivaci k_2 po tomto kroku a vyrazíme z bodu y_i ve směru, který je průměrem obou směrů (doporučuji si nakreslit obrázek).

Ekvivalentní je udělat „Eulerův půlkrok“ a vyrazit z bodu y_i ve směru derivace spočtené po tomto půlkroku:

$$\begin{aligned} k'_1 &= f(y_i, t_i) \\ k'_2 &= f\left(y_i + k'_1 \frac{\Delta t}{2}, t + \frac{\Delta t}{2}\right) \\ \phi_i &= k'_2 \end{aligned} \quad (10)$$

4.4 Runge-Kuttova metoda 4. řádu

$$\begin{aligned} k_1 &= f(y_i, t_i) \\ k_2 &= f\left(y_i + k_1 \frac{\Delta t}{2}, t + \frac{\Delta t}{2}\right) \\ k_3 &= f\left(y_i + k_2 \frac{\Delta t}{2}, t + \frac{\Delta t}{2}\right) \\ k_4 &= f(y_i + k_3 \Delta t, t + \Delta t) \\ \phi_i &= \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4) \end{aligned} \quad (11)$$

- Jedna z nejčastěji používaných metod.
- Vysoká rychlost a přesnost při relativní jednoduchosti.
- Existují i Runge-Kuttovy metody vyššího řádu p , avšak vyžadují výpočet více než p dílčích derivací k_j . Obecně platí, že metoda řádu $p \leq 4$ vyžaduje p derivací, metoda řádu $5 \leq p \leq 7$ vyžaduje $p + 1$ derivací a metoda řádu $p = 8, 9$ vyžaduje $p + 2$ derivací.

4.5 Odvození metod

Metody se odvozují na základě rozvoje do Taylorovy řady. Volně řečeno platí, že kolik členů Taylorova rozvoje se vezme, takový bude řád metody p .

- *Eulerova metoda 1. řádu.* Odvození je triviální:

$$y(t + \Delta t) \approx y(t) + \underbrace{y'(t)}_{f(y,t)} \Delta t, \quad (12)$$

takže

$$y_{i+1} = y_i + f(y_i, t_i) \Delta t. \quad (13)$$

- *Eulerova metoda 2. řádu.*

$$\begin{aligned} y(t + \Delta t) &\approx y(t) + y'(t) \Delta t + \frac{1}{2} \underbrace{y''(t)}_{\approx \frac{y'(t+\Delta t) - y'(t)}{\Delta t}} \Delta t^2 \\ &\approx y(t) + f(y, t) \Delta t + \frac{1}{2} [f(\underbrace{y(t + \Delta t)}_{\approx y(t) + y'(t) \Delta t}, t + \Delta t) - f(y, t)] \Delta t \\ &\approx y(t) + \frac{1}{2} [f(y, t) + f(y(t) + f(y, t) \Delta t, t + \Delta t)] \Delta t, \end{aligned} \quad (14)$$

a odtud

$$y_{i+1} = y_i + \underbrace{\frac{1}{2} [f(y_i, t_i) + f(y_i + f(y_i, t_i) \Delta t, t_{i+1})]}_{\phi_i} \Delta t, \quad (15)$$

což je jen jinak napsané vztahy (9).

Druhá Eulerova metoda 2. řádu (10) se obdrží stejným způsobem, jen při aproximaci druhé derivace se použije poloviční krok:

$$y''(t) \approx \frac{y'\left(t + \frac{\Delta t}{2}\right) - y'(t)}{\frac{\Delta t}{2}}. \quad (16)$$

Úloha 4.1: *Dokončete odvození vztahů (10).*

Analogicky se postupuje při odvození metod vyššího řádu. Jeho složitost však narůstá exponenciálně s řádem metody. U Eulerovy metody 2. řádu se navíc ukázalo, že lze odvodit několik stejně dobrých vztahů. Tato volnost s řádem metody narůstá.

Úloha 4.2: *Naprogramujte Eulerovu metodu 1. a 2. řádu a Runge-Kuttovu metodu. Vyřešte numericky diferenciální relaxační rovnici³³*

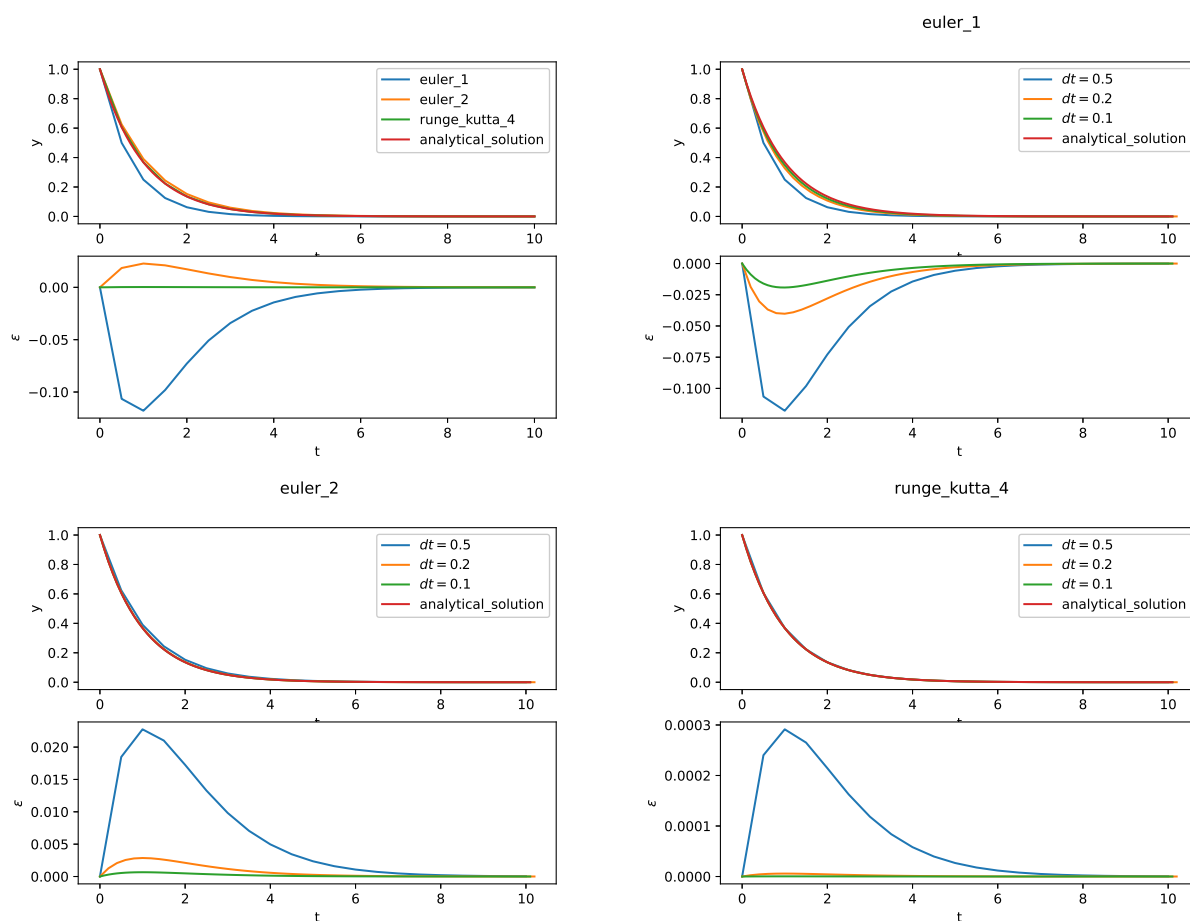
$$\frac{dy}{dt} = -y \quad (17)$$

s počátečními podmínkami $y_0 = 1$ (analytickým řešením je funkce e^{-t}). Integrační krok Δt ponechte jako volný parametr. Nakreslete grafy řešení $y(t)$ pro rozdílné hodnoty integračních kroků, například $\Delta t = 0.01$ a $\Delta t = 0.1$ pro čas $t \in \langle 0; 10 \rangle$.

Řešení 4.2: *Jedno možné řešení v souborech `ode.py` (integrační metoda), `graphs.py` (vykreslování grafů) a `relaxation.py` (hlavní soubor, který obsahuje definici modelu a volá všechny výpočetní a vykreslovací funkce). Na řešení této úlohy je také ukázán základ práce s větvemi v sekci 3.1.5.*

³³Rovnice popisuje například Newtonův zákon chlazení tělesa, aktuální množství prvku při radioaktivním rozpadu nebo různé populační modely.

- `ode.py`: Modul napsaný maximálně obecně, aby ho bylo možné použít na řešení libovolné diferenciální rovnice, a také soustav diferenciálních rovnic probíraných v sekci 5.
 - `euler_1`: Integrační krok Eulerovy metody 1. řádu.
 - `euler_2`: Integrační krok Eulerovy metody 2. řádu.
 - `runge_kutta_4`: Integrační krok Runge-Kuttovy metody 4. řádu.
 - `ode_solve`: Integruje diferenciální rovnici danou pravou stranou prvního parametru `model` s počáteční podmínkou předanou parametrem `initial_condition`. Integrační metodu lze specifikovat parametrem `integrator`, což je vlastně funkce $\phi(y_i, t_i)$ z rovnice (4). Délku kroku udává parametr `dt`. Funkce vrací pole hodnot řešení rovnice (soustavy rovnic) v jednotlivých časech a pole časů.
 - `scipy_ode_solve`: Integruje diferenciální rovnici pomocí funkce `odeint` z knihovny `scipy.integrate`. Pozor, parametr `dt` zde neznačí integrační krok, nýbrž časový krok výsledného pole. Funkce `odeint` používá sofistikovaný řešitel diferenciálních rovnic s proměnným krokem. Pro podrobnosti můžete mrknout na [dokumentaci](#) k této funkci.



Obrázek 1: Integrace relaxační diferenciální rovnice (17) různými metodami a s různými časovými kroky.

- `graphs.py`: Vykresluje grafy srovnávající různé metody nebo různé kroky výpočtu.
 - `plot_solution_error`: Vykreslí graf řešení diferenciální rovnice. Parametr `analytical_solution` je odkaz na přesné řešení dané diferenciální rovnice. Je-li zadán, vykreslí se do grafu dva panely: jeden s hodnotami numerického řešení, druhý s rozdílem řešení numerického a přesného ϵ (akumulovaná diskretizační chyba dle rovnice (6)).

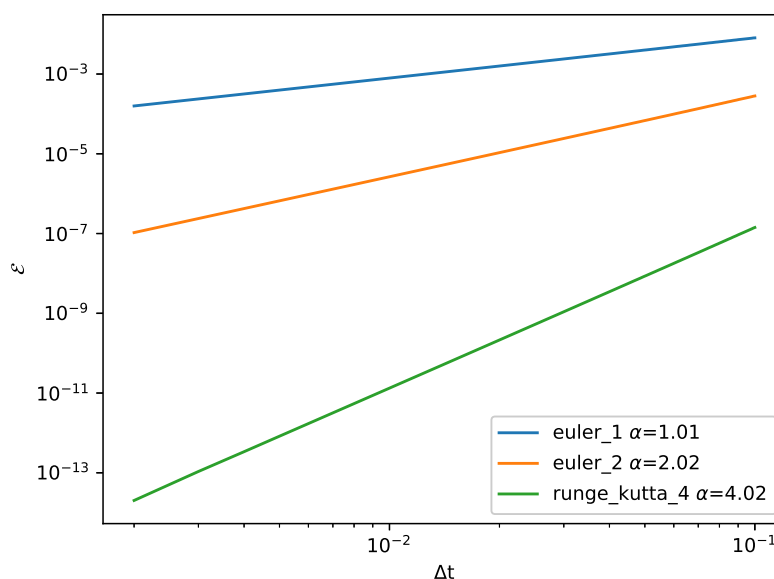
- `plot_compare_methods`: Vykreslí do jednoho obrázku řešení zadané diferenciální rovnice různými metodami. Z důvodu efektivnějšího kódu se v této funkci používá trochu složitější práce s panely, než jak je uvedeno ve vzorovém souboru `plot.py`.
- `plot_compare_steps`: Vykreslí do jednoho obrázku řešení zadané diferenciální rovnice s různými integračními kroky.
- `plot_cumulative_errors`: Řešení úlohy 4.3.
- `relaxation.py`: Soubor, který využívá integračních funkcí z modulu `ode.py` a vykreslovacích funkcí z modulu `graphs.py` pro řešení diferenciální rovnice pro relaxaci.
 - Vyřeší diferenciální rovnici různými metodami, řešení nakreslí do jednoho grafu a porovná s teoretickou funkcí.
 - Vyřeší diferenciální rovnici s různým časovým krokem, řešení nakreslí do jednoho grafu a porovná s teoretickou funkcí.
 - Nakreslí graf k úloze 4.3.

Příslušné grafy jsou zobrazeny na obrázku 1. Je vidět, že chyba opravdu klesá s řádem metody a že čím je metoda vyššího řádu, tím rychleji chyba klesá se zmenšujícím se krokem.

Úloha 4.3: Rozšiřte kód tak, aby počítal průměrnou kumulovanou chybu

$$\mathcal{E} = \sqrt{\frac{1}{n} \sum_{i=0}^{n-1} (y_i - y(t_i))^2}, \quad (18)$$

kde $y(t)$ je analytické řešení diferenciální rovnice. Nakreslete závislost $\mathcal{E}(\Delta t)$ pro $\Delta t \in (0.002; 0.1)$ a pro různé metody. Jelikož očekáváme mocninnou závislost dle (7), kde exponent je tím větší, čím větší je řád metody, je výhodné graf $\mathcal{E}(\Delta t)$ kreslit v log-log měřítku. V Pythonu použijete místo `plot(...)` funkci `loglog(...)` z knihovny `matplotlib.pyplot`. Ověřte, že získané křivky jsou v souladu s řády použitých metod.



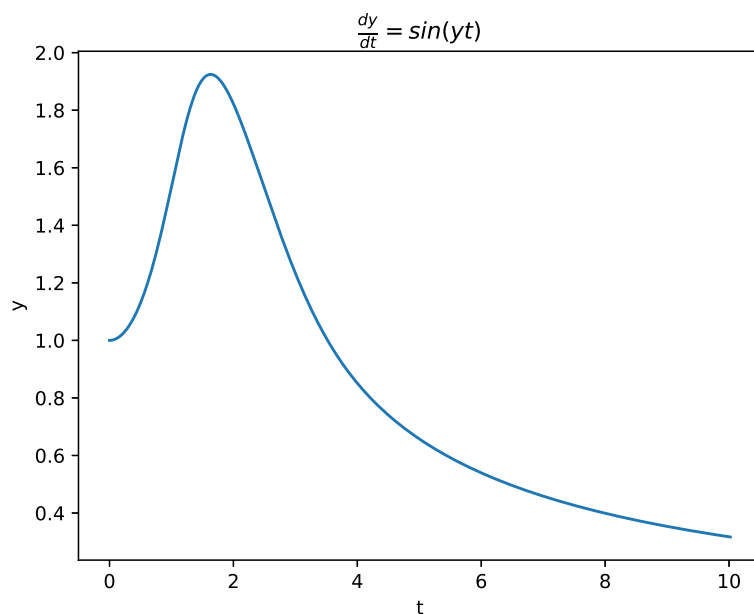
Obrázek 2: Závislost průměrné kumulované chyby (18) na délce kroku Δt vypočítaná a vykreslená pomocí funkce pro relaxační diferenciální rovnici.

Řešení 4.3: Řešení této úlohy vykreslí funkce `plot_cumulative_error` z modulu `graphs.py`. Graf je zobrazen na obrázku 2. Body log-log grafu byla proložena přímkou pomocí funkce lineární regrese `linregress` z knihovny `scipy.stats`. Nafitovaná směrnice α je uvedena v obrázku.

Úloha 4.4: Pomocí naprogramovaných metod vyřešte nelineární diferenciální rovnici

$$\frac{dy}{dt} = \sin(ty) \quad (19)$$

s počáteční podmínkou $y_0 = 1$, $t_0 = 0$ a vykreslete graf jejího řešení.



Obrázek 3: Řešení diferenciální rovnice (19) s počáteční podmínkou $y(0) = 1$. Použitá metoda: Runge-Kutta, časový krok $\Delta t = 0.02$.

Řešení 4.4: Řešení je v souboru `sinyt.py`. Graf vypočítané funkce je na obrázku 3.

5 Soustavy diferenciálních rovnic 1. řádu

Každou obyčejnou diferenciální rovnici n -tého řádu lineární v nejvyšší derivaci lze převést na soustavu n obyčejných diferenciálních rovnic prvního řádu ve tvaru

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(\mathbf{y}, t), \quad (20)$$

kde $\mathbf{y} = \mathbf{y}(t)$ je vektor hledaných funkcí a $\mathbf{y}(t_0) = \mathbf{y}_0$ vektor počáteční podmínky.

Příklad 5.1: Pohybovou rovnici

$$Ma = F(x), \quad (21)$$

kde M je hmotnost pohybujícího se tělesa, $x = x(t)$ jeho poloha a $a = a(t) = d^2x/dt^2$ zrychlení převeďme na dvě diferenciální rovnice prvního řádu zavedením rychlosti $v = v(t) = dx/dt$.³⁴

$$\frac{d}{dt} \begin{pmatrix} x \\ v \end{pmatrix} = \begin{pmatrix} v \\ \frac{1}{M}F(x) \end{pmatrix}, \quad (22)$$

tj. vektor funkce pravých stran podle rovnice (20) je

$$\mathbf{f}(\mathbf{y}, t) = \begin{pmatrix} v \\ \frac{1}{M}F(x) \end{pmatrix} \quad (23)$$

kde $\mathbf{y} \equiv \begin{pmatrix} x \\ v \end{pmatrix}$.

Příklad 5.2: Pohybová rovnice pro harmonický oscilátor (matematické kyvadlo s malou výchylkou) při volbě jednotek $M = \Omega = 1$, kde M je hmotnost kmitající částice a Ω její rychlost, zní

$$a = \frac{d^2x}{dt^2} = -x \quad \Longleftrightarrow \quad \frac{d}{dt} \begin{pmatrix} x \\ v \end{pmatrix} = \begin{pmatrix} v \\ -x \end{pmatrix} \quad (24)$$

Úloha 5.1: Převeďte na soustavu obyčejných diferenciálních rovnic prvního řádu rovnici třetího řádu pro Hiemenzův tok

$$x''' + xx'' - x'^2 + 1 = 0. \quad (25)$$

Řešení 5.1: Hledaná soustava diferenciálních rovnic je

$$\frac{d}{dt} \begin{pmatrix} x \\ v \\ a \end{pmatrix} = \begin{pmatrix} v \\ a \\ -xa + v^2 - 1 \end{pmatrix}. \quad (26)$$

Drtivá většina knihoven a algoritmů pro integraci obyčejných diferenciálních rovnic počítá s rovnicemi ve tvaru (20).

5.1 Symplektické algoritmy

Jedná se o speciální algoritmy navržené pro řešení pohybových diferenciálních rovnic. Od běžných algoritmů je odlišuje to, že zachovávají objem fázového prostoru, a tedy i energii (zatímco u obecných algoritmů se energie s integračním časem mění a většinou roste).

V praxi se ze symplektických algoritmů používá nejčastěji *Verletův algoritmus*. Pro diferenciální rovnici 2. řádu ve tvaru

$$M \frac{d^2x}{dt^2} = F(x), \quad (27)$$

³⁴Tato soustava rovnic odpovídá Hamiltonovým pohybovým rovnicím, se kterými se seznámíte v přednášce Teoretická mechanika.

což je pohybová rovnice pro jeden hmotný bod o hmotnosti M , na který působí časově neproměnná síla $F(x)$, má Verletův algoritmus tvar

$$\begin{aligned}x_{i+1} &= x_i + v_i \Delta t + \frac{1}{2} a_i \Delta t^2, \\v_{i+1} &= v_i + \frac{1}{2} (a_{i+1} + a_i) \Delta t,\end{aligned}\tag{28}$$

kde $a_i \equiv F(x_i)/M$ je zrychlení částice.

Verletův algoritmus se používá nejčastěji v molekulární dynamice k simulaci pohybu velkého množství vzájemně interagujících částic. Jedná se o velmi rychlou a efektivní metodu.

Řád Verletova algoritmu je $p = 2$. Symplektické algoritmy s vyšším řádem existují, avšak v praxi se nepoužívají.

Úloha 5.2: Rozšiřte kód naprogramovaný v úkolu 4.2 tak, aby fungoval pro libovolně velkou soustavu obyčejných diferenciálních rovnic. Využijte k tomu funkce pro práci s řadami z knihovny `numpy` popsané v sekci 3.2.4. Vyřešte diferenciální rovnici harmonického oscilátoru

$$\frac{d^2x}{dt^2} = -x\tag{29}$$

s počátečními podmínkami $x_0 = 0$, $x'_0 \equiv v_0 = 1$, $t_0 = 0$ a porovnejte řešení různými metodami s analytickým řešením $x(t) = \sin t$. Jako časový krok volte například $\Delta t = 0.1$ a $\Delta t = 0.01$ a počítejte na časovém intervalu $t \in \langle 0; 30 \rangle$.

Řešení 5.2: Metody jsou naprogramovány v souborech `ode.py` (integrační metoda), `graphs.py` (vykreslování grafů) a jsou podrobně diskutovány v řešení úlohy 4.2. Soubor `ho.py` obsahuje pravé strany diferenciálních rovnic harmonického oscilátoru (funkce `ho`) a volá všechny výpočetní a vykreslovací funkce. Navíc obsahuje následující funkce:

- `ho_solution`: Přesné řešení pohybové rovnice harmonického oscilátoru s počáteční podmínkou $x_0 = 0$, $x'_0 = 1$, $t_0 = 0$, jímž je funkce $x(t) = \sin t$.
- `ho_energy`: Pro vstupní polohu a rychlost (nebo pole poloh a rychlostí) vrátí energii harmonického oscilátoru.

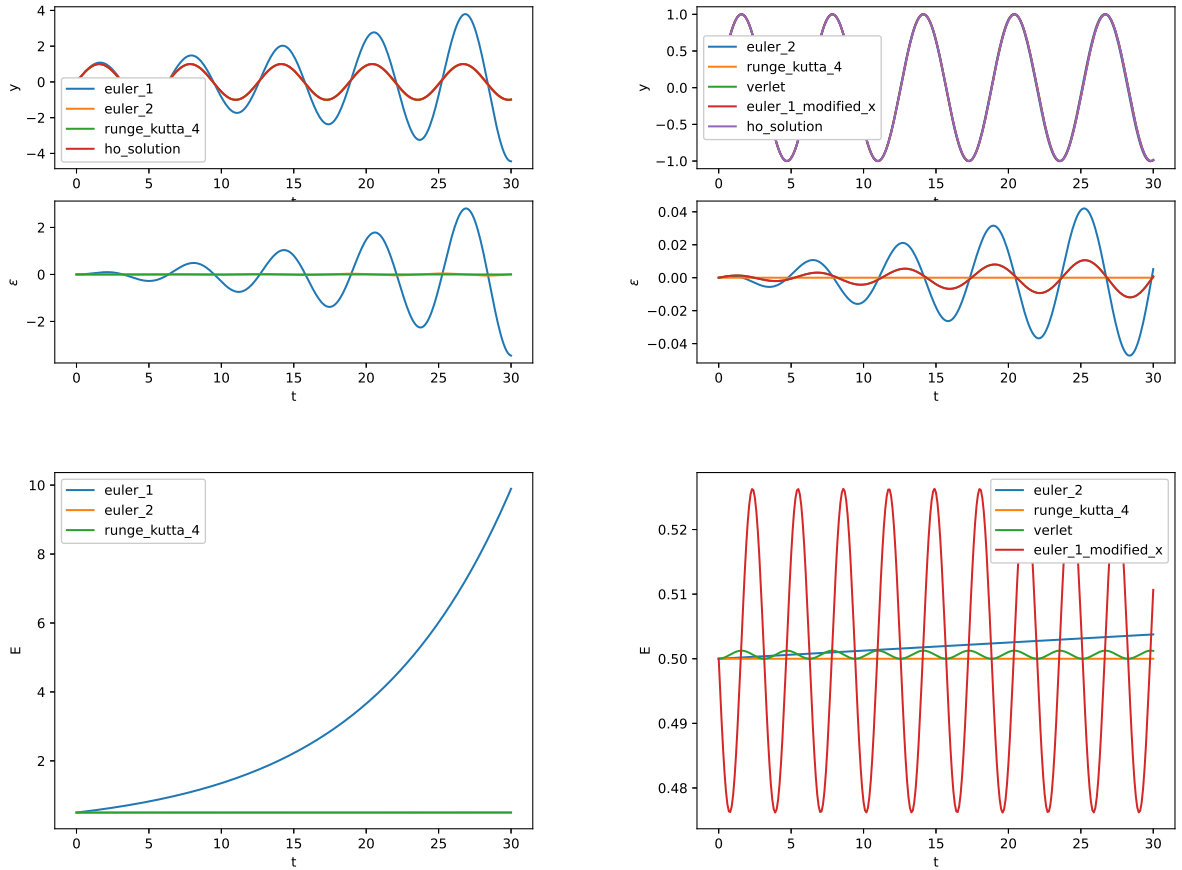
Příslušné grafy jsou zobrazeny na obrázku 4.

Úloha 5.3: Naprogramujte Verletův algoritmus (28). Ukažte, že zatímco při použití Eulerovy metody nebo Runge-Kuttovy metody energie systému v průběhu výpočtu roste, Verletův algoritmus energii zachovává. Energie bezrozměrného harmonického oscilátoru (29) je dána vzorcem

$$E = \frac{1}{2} (x^2 + v^2).\tag{30}$$

Řešení 5.3: Symplektický Verletův algoritmus je naprogramován v souboru `symplectic.py`. Toto vzorové řešení bude fungovat jen pro jednu pohybovou rovnici, tj. pro jednu diferenciální rovnici původně druhého řádu přepsanou na dvě diferenciální rovnice prvního řádu, přičemž první rovnice musí být pro souřadnici, druhá pro rychlost. Řešení lze samozřejmě rozšířit na více pohybových rovnic.

V souboru `symplectic.py` je i funkce `plot_energy`, která vykreslí graf závislosti $E(t)$. Energie je znázorněna v panelech na třetím řádku na obrázku 4. Harmonický oscilátor je konzervativní systém (zachovává energii), pozorovaná rostoucí energie je způsobena nepřesností integračních metod. U symplektických algoritmů energie slabě osciluje okolo střední hodnoty, která však i pro velmi dlouhé časy zůstává na přesné hodnotě $E = \frac{1}{2}$.



Obrázek 4: Integrace diferenciální rovnice harmonického oscilátoru (29) různými metodami. Časový krok je $\Delta t = 0.1$. *Levý sloupec:* všechny metody. *Pravý sloupec:* Bez eulerovy metody 1. řádu a včetně symplektických metod. *1. řádek:* hodnoty $y(t)$. *2. řádek:* Akumulované diskretizační chyby dle (6). Pro Eulerovu metodu 1. řádu je obalová křivka divergence numerického od analytického řešení očividně exponenciální v čase. *3. řádek:* energie (30). Pro Eulerovy metody energie roste. Energie se mění i pro Runge-Kuttovu metodu (pro tento systém energie s časem klesá), avšak změna je řádově menší než pro ostatní metody, a tudíž není na grafech při daném měřítku svislé osy vidět. Naopak pro Verletův algoritmus a pro „předbíhající“ Eulerovu metodu energie osciluje okolo počáteční energie $E = \frac{1}{2}$.

Úloha 5.4: Eulerovu metodu 1. řádu lze pro soustavy dvou diferenciálních rovnic 1. řádu vylepšit následující záměnou:

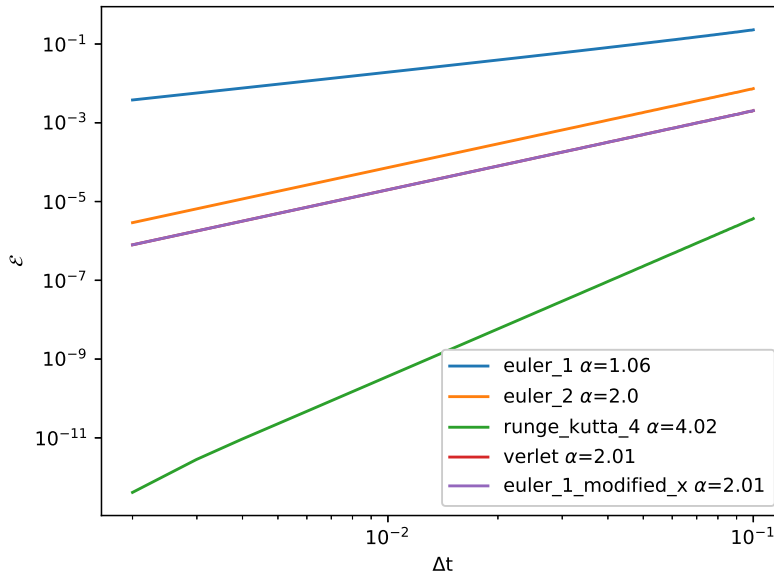
$$\begin{aligned} x_{i+1} &= x_i + v_i \Delta t \\ v_{i+1} &= v_i - x_i \Delta t \end{aligned} \quad \longrightarrow \quad \begin{aligned} x_{i+1} &= x_i + v_i \Delta t \\ v_{i+1} &= v_i - x_{i+1} \Delta t \end{aligned} \quad (31)$$

(vypočítáme x_{i+1} a tuto hodnotu použijeme namísto hodnoty x_i pro výpočet rychlosti v_{i+1}). Naprogramujte tuto metodu a pomocí výsledků úlohy 4.3 ukažte, že pro harmonický oscilátor se jedná o metodu 2. řádu.

Řešení 5.4: „Vylepšené“ Eulerovy metody jsou naprogramovány v souboru `symplectic.py`. Metoda s předbíhající se souřadnicí je označena `euler_1_modified_x`, metoda s předbíhající rychlostí pak `euler_1_modified_v`. Obě metody dávají velmi podobné výsledky, v grafech je proto uvedena jen první z nich.

Řešení diferenciální rovnice harmonického oscilátoru je znázorněno v pravém sloupci obrázku 4. Je vidět, že se přesnosti týče, vylepšená Eulerova metoda je srovnatelná s Verletovou metodou. Energie však osciluje trochu víc.

Řád metody lze určit na základě obrázku 5. Jak u Verletovy metody, tak u vylepšené Eulerovy



Obrázek 5: Závislost průměrné kumulované chyby (18) na délce kroku Δt vypočítaná a vykreslená pro soustavu diferenciálních rovnic pro harmonický oscilátor. Zobrazeny jsou i symplektické metody.

metody klesá průměrná kumulovaná chyba s druhou mocninou délky časového kroku δt , jedná se tedy o metody 2. řádu.

Úloha 5.5: Pohrajte si s řešením rovnice pro klesající exponenciálu

$$\frac{d^2x}{dt^2} = x \quad (32)$$

s počátečními podmínkami $x_0 = 1$, $x'_0 = -1$. Přesvědčte se, že Verletova metoda a vylepšená Eulerova metoda z předchozího úkolu jsou nestabilní — pro tuto rovnici v relativně krátkém čase začnou řešení exponenciálně divergovat.

Řešení 5.5: Řešení rovnice (32) je v souboru `exp.py`. Soustava rovnic je analogická k soustavě harmonického oscilátoru — liší se pouze znaménkem. Systém popsany touto rovnicí není konzervativní — nelze nadefinovat zachovávající se veličinu, která by měla význam energie.

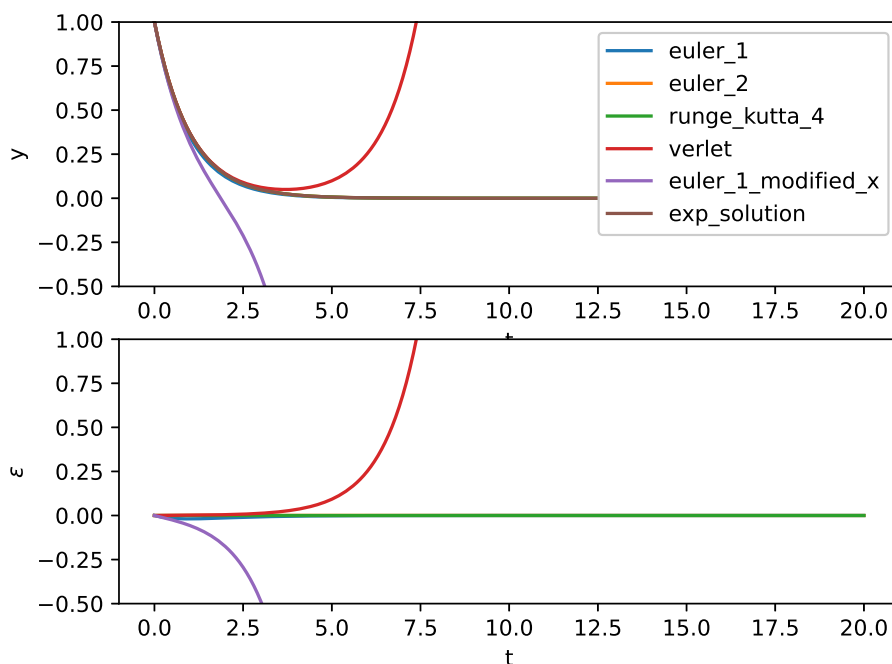
Z obrázku 6 je vidět, že symplektické algoritmy jsou nestabilní, a tudíž nejsou na tento typ úlohy vhodné, což je pochopitelné, protože jsou navrženy pouze pro energii zachovávající systémy. Příčinu nestability lze nahlédnout i z obecného řešení rovnice (32),

$$y(t) = A e^t + B e^{-t}, \quad (33)$$

přičemž my speciálními počátečními podmínkami vybíráme pouze exponenciálně klesající řešení. Symplektické algoritmy však v určité chvíli „překmitnou“ na exponenciálně rostoucí řešení, což způsobí pozorovanou divergovat.

Úloha 5.6: Vyřešte nelineární soustavu tří diferenciálních rovnic pro jednoduchý Lorenzův model vedení tepla v atmosféře

$$\begin{aligned} \frac{dx}{dt} &= \sigma(y - x), \\ \frac{dy}{dt} &= x(\rho - z) - y, \\ \frac{dz}{dt} &= xy - \beta z \end{aligned} \quad (34)$$



Obrázek 6: Totéž jako v obrázku 4, avšak pro exponenciálně klesající systém daný rovnicí (32). Symplektické algoritmy jsou nestabilní.

s hodnotami parametrů $\sigma = 10$, $\rho = 28$ a $\beta = 8/3$, počátečními podmínkami $(x_0, y_0, z_0) = (1, 1, 1)$ (na počátečních podmínkách zase tolik nezáleží), s krokem $\Delta t = 0.01$ a na časovém intervalu $t \in \langle 0, 100 \rangle$. Vykreslete graf $z(x)$. Výsledná křivka je slavný *Lorenzův podivný atraktor* ve tvaru motýlích křídel, který zpopularizoval teorii klasického chaosu.

Řešení 5.6: Řešení Lorenzova systému diferenciálních rovnic je jednoduchou aplikací kódu naprogramovaného v úloze 5.2 a lze ho nalézt v souboru `lorenz.py`. Příslušnou projekci trojrozměrné trajektorie do 2D roviny (x, z) lze nalézt na obrázku 7.

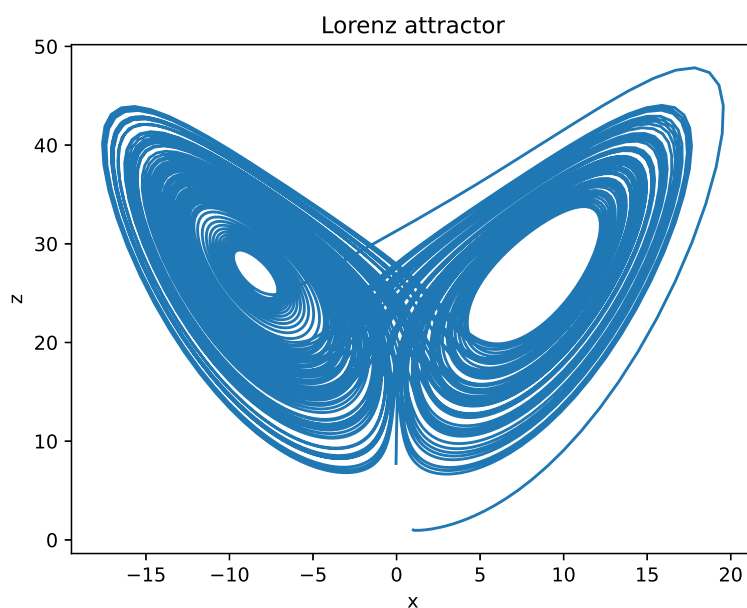
Úloha 5.7: Vyřešte numericky pohybovou rovnici pro pohyb tlumeného a buzeného kyvadla

$$\frac{d^2\theta}{dt^2} + \eta \frac{d\theta}{dt} + \omega^2 \sin \theta = A \sin \omega t \quad (35)$$

s počátečními podmínkami $\theta_0 = 0$, $\dot{\theta}_0 = 1$ a hodnotami parametrů $\eta = 0.5$, $\omega = 1$, $A = 1$ na časovém intervalu $t \in \langle 0, 500 \rangle$. Pro frekvenci buzení volte tři různé hodnoty $\omega_0 = 0.2, 0.4, 0.8$. Otestujte stabilitu řešení volbou dvou různých kroků, např. $\Delta t = 0.01, \Delta t = 0.005$.

5.2 Shrnutí

- Řešitelé obyčejných diferenciálních rovnic převážně pracují se soustavami diferenciálních rovnic prvního řádu. Na tento tvar není obtížné diferenciální rovnici vyššího řádu převést.
- Nejčastěji se používají jednokrokové metody, jejichž hlavní výhoda je v možnosti jednoduše dle potřeby měnit délku kroku (metody s adaptivním krokem).
- Přesnost řešení závisí na řádu metody p a na délce integračního kroku Δt . Čím je řád metody vyšší, tím rychleji klesá chyba se zmenšujícím se krokem. V praxi, pokud nechcete svěřit svůj problém černé skřínce ve formě nějaké hotové knihovny, se velmi často používá Runge-Kuttova metoda 4. řádu, která je jednoduchá na implementaci, je stabilní a rychlá.



Obrázek 7: Lorenzův systém (34) integrovaný pomocí Runge-Kuttovy metody 4. řádu na časovém intervalu $t \in \langle 0; 100 \rangle$ s krokem $\Delta t = 0.01$.

- Symplektické metody, z nichž nejběžnější je Verletova metoda, jsou výhodné k modelování fyzikálních systémů zachovávajících energii. Pro nekonzervativní systémy nejsou vhodné.

A hlavně, nyní již umíte jen pomocí sčítání a násobení vypočítat a nakreslit průběh goniometrické funkce sinus.

6 Náhodná procházka

Náhodná procházka je jeden ze základních prostředků, jak simulovat velké množství nejen fyzikálních procesů (například pohyb Brownovské částice, fluktuace akciového trhu, cestu opilce z hospody atd.). V dalších cvičeních si ukážeme, jak se pomocí náhodné procházky dá jednoduše hledat minimum funkcí (a to i funkcí více proměnných).

Algoritmus pro náhodnou procházku je následující: v každém časovém kroku uděláme krok v d -rozměrném prostoru $\mathbf{y}_i \rightarrow \mathbf{y}_{i+1}$ takovým způsobem, aby pravděpodobnost pohybu do všech směrů byla stejná. Délka kroku s se volí buď náhodná, nebo konstantní.

K simulování náhodných procesů slouží algoritmy generující pseudonáhodná čísla, což jsou čísla, která mají statistické vlastnosti blízké vlastnostem skutečných náhodných čísel, avšak jsou počítána jednoduchými deterministickými algoritmy. Náhodná čísla se generují od počáteční tzv. *násady* (seed), kterou lze explicitně zadat, a tím posloupnost náhodných čísel přesně zreprodukovat (díky tomu, že generující algoritmy jsou deterministické). Pokud násada není explicitně zadána, knihovny pro generování pseudonáhodných čísel většinou volí systémový čas, takže při každém spuštění programu dostáváme posloupnost odlišnou.

Základní funkce a postupy pro generování náhodných čísel v Pythonu jsou uvedeny v sekci 3.2.6.

Úloha 6.1: *Naprogramujte náhodnou procházku ve 2D rovině. Délku kroku volte konstantní $s = 1$, směr volte náhodně. Začněte z bodu $(x, y) = (0, 0)$ a procházku ukončete poté, co opustíte oblast tvaru čtverce o hraně délky $2a$. Uchovávejte celou procházku v poli či seznamu. Nakonec trajektorii vykreslete do grafu.*

Řešení 6.1: *Řešení je naprogramováno v souboru 2d.py.*

- `random_direction_2d` vrátí náhodný směr ve 2D rovině [generuje náhodný úhel ϕ , směr je dán jednotkovým vektorem se složkami $(\cos \phi, \sin \phi)$].
- `random_walk_2d` vykreslí do grafu náhodnou procházku s `num_steps` kroky omezenou ve čtverci rozměru `2*box_size×2*box_size` a náhodnou procházku vrátí.
- `random_walk_2d_interactive` generuje náhodnou procházku a vykresluje ji do grafu krok po kroku. Musí být zapnutý interaktivní mód vykreslování `plt.ion()` a v prostředí Spyder vypnuto použití inline grafů příkazem `%matplotlib auto` v konzoli REPL. Tato funkce implementuje i cyklické okrajové podmínky.

Vzorový kód je napsán takovým způsobem, aby mohl být přímočaře rozšířen pro vícerozměrnou náhodnou procházku.

Úloha 6.2: *Upravte náhodnou procházku tak, aby počítala s cyklickými okrajovými podmínkami. To znamená, že pokud opustíte oblast čtverce jednou jeho stranou, objevíte se na straně protilehlé (jako kdybyste okraje čtverce zavinuli a slepili). Výpočet ukončete poté, co uděláte n kroků.*

Řešení 6.2: *Cyklické okrajové podmínky jsou naprogramovány ve funkci `random_walk_2d_interactive` v souboru 2d.py. Stejně je lze naprogramovat i do funkce `random_walk_2d`, výsledný graf však nebude vypadat příliš hezky.*

Úloha 6.3: *Zamyslete se nad tím, jak byste realizovali náhodnou procházku s konstantní délkou kroku v $d > 2$ rozměrech, a zkuste své řešení naprogramovat. Zásadní je dodržet požadavek, aby pohyb do jakéhokoliv směru nastával se stejnou pravděpodobností (esence úlohy tedy spočívá v generování náhodného směru v d -rozměrném prostoru).*

Řešení 6.3: *Zatímco pro směr ve 2D rovině stačí náhodě generovat jeden úhel (předchozí úloha), vícerozměrné úlohy jsou komplikovanější. Přímé rozšíření 2D případu do 3D (či do vyšších dimenzí)*

za generování více úhlů a použití (hyper-)sférických souřadnic k cíli nevede — takto otrocky generované směry upřednostňují okolí pólů před rovníkem (rozmyslete). K úspěšnému generování náhodného kroku je nutné využít jeden z následujících algoritmů:

1. Hyperkoule vepsaná v hyperkrychli.

- (a) Nagenerujeme bod v hyperkrychli o hraně délky 2, tj. generujeme vektor $\mathbf{v}$ s d složkami, přičemž každá složka je náhodné číslo z rovnoměrného rozdělení $(-1, 1)$.
- (b) Zkontrolujeme, zda bod leží uvnitř vepsané jednotkové koule například tak, že spočítáme jeho normu $v = |\mathbf{v}|$ a porovnáme, zda $n \leq 1$.
- (c) Pokud ne, opakujeme postup od začátku. Pokud ano, nagenеровaný bod promítneme na jednotkovou kouli (jinými slovy vektor $\mathbf{v}$ nanormujeme) a získaný jednotkový vektor $\hat{\mathbf{v}} \equiv \mathbf{v}/v$ udává hledaný směr.

Tato metoda je nesmírně neefektivní, pokud je dimenze d vysoká, poněvadž v tom případě většina nagenеровaných bodů leží vně vepsané hyperkoule a je zahozena. Poměr celkového počtu nagenеровaných bodů ku úspěšným zásahům uvnitřku hyperkoule lze snadno spočítat. Objem hyperkrychle o hraně délky 2 je

$$V_d^{(\text{krychle})} = 2^d, \quad (36)$$

objem vepsané hyperkoule o poloměru 1 je

$$V_d^{(\text{koule})} = \frac{\pi^{\frac{d}{2}}}{\Gamma\left(\frac{d}{2} + 1\right)}, \quad (37)$$

kde Γ je Eulerova gama funkce.³⁵ Vzájemný poměr

$$\eta_d \equiv \frac{V_d^{(\text{krychle})}}{V_d^{(\text{koule})}} = \left(\frac{2}{\sqrt{\pi}}\right)^d \Gamma\left(\frac{d}{2} + 1\right) \quad (39)$$

udává, kolik bodů musíme průměrně nagenеровovat, abychom se trefili do hyperkoule (reciproká hodnota $1/\eta_d$ určuje pravděpodobnost, že se do hyperkoule trefíme). Zatímco pro $d = 3$ je $\eta_3 \approx 1.91$, pro $d = 10$ již $\eta_{10} \approx 401$, tj. pro nalezení jednoho náhodného směru v desetirozměrném prostoru musíme nagenеровovat v průměru přes 4000 náhodných čísel. Z posledního vztahu je vidět, že s rostoucí dimenzí roste η_d exponenciálně.

Z tohoto výpočtu také vyplývá, že pokud bychom nezahazovali body ležící mimo hyperkouli, pak bychom ve výsledné procházce výrazně upřednostňovali pohyb podél diagonál, a to tím více, čím vyšší je dimenzionalita procházky (u $d = 10$ bychom podél diagonál vyrazili s více než 99% pravděpodobností).

2. Náhodný Gaussovský vektor.

- (a) Nagenerujeme vektor $\mathbf{n}$ s d složkami, přičemž každá složka je číslo z normálního Gaussovského rozdělení $N(0, 1)$.
- (b) Vektor nanormujeme a získáme hledaný náhodný směr $\hat{\mathbf{n}} \equiv \mathbf{n}/n$.

Tato metoda je mnohem přímočařejší než předchozí, předpokladem je jen mít k dispozici generátor čísel vybraných z normálního rozdělení.

Důkaz, že tato metoda dává opravdu náhodný směr v d dimenzích, a další informace o metodě se nalézají v článcích [2, 3].

³⁵Gama funkce je rozšíření faktoriálu pro neceločíselné (obecně pro komplexní) hodnoty. Pro $n \in \mathbb{N}$ platí

$$\Gamma(n) = (n - 1)!. \quad (38)$$

3. Speciální případ $d = 3$ (náhodný let).

- (a) Generujeme dvě náhodná čísla $\xi_{1,2}$ z rovnoměrného rozdělení na intervalu $(0; 1)$.
 (b) Sférické úhly jednotkového směru jsou pak

$$\begin{aligned}\phi &= 2\pi\xi_1, \\ \theta &= \arccos(1 - 2\xi_2),\end{aligned}\tag{40}$$

takže hledaný jednotkový vektor $\hat{\mathbf{n}}$ do náhodného směru má komponenty

$$\begin{aligned}\hat{n}_x &= \sin\theta \cos\phi = \sqrt{1 - (1 - 2\xi_2)^2} \cos 2\pi\xi_1, \\ \hat{n}_y &= \sin\theta \sin\phi = \sqrt{1 - (1 - 2\xi_2)^2} \sin 2\pi\xi_1, \\ \hat{n}_z &= \cos\theta = 1 - 2\xi_2.\end{aligned}\tag{41}$$

Ve více rozměrech je tento přístup prakticky nerealizovatelný (vede na problém inverzních funkcí k funkcím daným řadou goniometrických funkcí).

Náhodná procházka v d -rozměrném prostoru je naprogramována v souboru `nd.py`. Funkce `random_direction` generuje směr pomocí 1. metody, funkce `random_direction_gaussian` pomocí 2. metody. Spočítejte si náhodnou procházku pro $d = 10$ oběma metodami. Uvidíte, že i pro takto relativně „malou“ dimenzi je rozdíl ve výpočetních časech je dramatický.

7 Hledání minima funkce

Náhodnou procházku lze úspěšně použít k hledání minima funkce obecně více proměnných. Představte si funkci dvou proměnných jako zvlněnou „krajinu“ v noci. Potřebujete se vrátit k chatě, která se nachází pod vámi hluboko v úkolí. Je tma a nevidíte jakým směrem se vydat. Zkusíte tedy udělat náhodný krok a pokud povede dolů, vykročíte. Pokud by však krok vedl nahoru, zůstanete na místě a zkusíte nový směr.

Jednoduchá náhodná procházka funguje dobře pro funkce s jedním minimem. V obecném případě ale může mít funkce více lokálních minim a právě uvedený algoritmus skončí náhodně v jednom z nich, ze kterého se již nedokáže dostat ven. Přitom rozhodně nemusí jít o minimum nejhlubší (globální).

Hledání globálního minima funkce mnoha proměnných je obecně velmi komplexní problém. Dva nejjednodušší postupy, kterými můžeme vylepšit stávající metodu pomocí náhodné procházky, jsou následující:

- Provedeme několik náhodných procházek, které obecně dojdou do různých lokálních minim. Následně porovnáme konečné funkční hodnoty a vybereme to minimum, které má hodnotu nejnižší.
- Provedeme jednu náhodnou procházku doplněnou o *Metropolisův algoritmus*.

7.1 Metropolisův algoritmus

Metropolisův algoritmus rozšiřuje náhodnou procházku o konečnou teplotu. Je inspirován termodynamickým Boltzmannových rozdělením energie: máme tepelnou energii, díky které můžeme při náhodné procházce s určitou pravděpodobností udělat krok i „do kopce“, avšak čím je kopec strmější, tím bude pravděpodobnost takového kroku menší.

Předpokládejme, že jsme na vrstevnici s funkční hodnotou f a nová funkční hodnota po provedení kroku náhodné procházky by byla $f_{\text{nová}} > f$. Při minimalizaci pomocí obyčejné náhodné procházky bychom tento krok neprovedli. V Metropolisově algoritmu krok provedeme s pravděpodobností

$$p = e^{\frac{f - f_{\text{nová}}}{T}}, \quad (42)$$

kde T je parametr, který má roli „teploty“: pokud $T = 0$, žádný tepelný pohyb neexistuje, krok do kopce nikdy neprovedeme a vracíme se tak k obyčejné minimalizaci. Pokud $T \rightarrow \infty$, uděláme krok do kopce s pravděpodobností $p = 1$, což znamená, že tepelný pohyb zcela převládá, my se pohybujeme zcela náhodně a potenciál pod sebou vůbec necítíme.

V praxi je největší umění zvolit správnou hodnotu teploty. Pokud zvolíme teplotu nízkou, skončíme v lokálním minimu a už se z něj nedostaneme, pokud naopak příliš vysokou, budeme chaoticky procházet krajinou naší funkce a žádné minimum nenajdeme. Dobrá volba je začít spíš s vyšší teplotou a teplotu postupně snižovat. Jakmile se ocitneme zaseklí v nějakém minimu, můžeme teplotu zase trochu zvýšit a tím vyzkoušet, zda se nepřesuneme do nějakého minima hlubšího.

7.2 Minimalizace pomocí knihovny SciPy

Python obsahuje funkci pro hledání minima `minimize` v knihovně `scipy.optimize`.

Úloha 7.1: Rozšířte svůj program z minulého cvičení pro náhodnou procházku tak, aby hledal minimum funkce dvou proměnných $f(x, y)$. Otestujte svůj program pro kvadratickou funkci

$$f(x, y) = x^2 + y^2 \quad (43)$$

a pro Rosenbrockovu funkci

$$g(x, y) = (a - x)^2 + b(y - x^2)^2 \quad (44)$$

vypadající jako velmi pozvolná klesající hluboké údolí ve tvaru paraboly. Tato funkce se používá k testování rychlosti a efektivity minimalizačních algoritmů. Její minimum se nachází v bodě (a, a^2) a hodnoty parametrů nejčastěji se volí $a = 1, b = 100$.

Implementujte vhodným způsobem ukončení náhodné procházky, tj. okamžik, kdy jste již dorazili do minima funkce.

Řešení 7.1: Vzorový kód naleznete v souboru `minimize.py` a vychází z náhodné vícerozměrné procházky z modulu `nd.py` (z tohoto modulu kód využívá funkci `random_direction_gaussian`). K minimalizaci jsou v kódu dvě různé metody:

- **minimize:** Hledá minimum funkce `function` z počátečního bodu daného parametrem `initial_condition`. Pokud tento parametr není specifikován, zvolí se počáteční bod náhodně z `dimension`-rozměrné hyperkrychle se středem v počátku a délkou hrany `2*initial_condition_box`. Výpočet je ukončen, pokud se kódu `max_failed_steps`-krát po sobě nepodaří udělat úspěšný krok (krok směrem k menší funkční hodnotě). Každý náhodný krok má konstantní délku danou parametrem `step_size`.
- **minimize_adaptive:** Předchozí metoda hledání minima má chybu $\Delta x_i \approx \text{step_size}$. Pro zmenšení chyby je v ní nutné zmenšit délku kroku náhodné procházky. Pokud tak učiníme, výpočetní čas T se výrazně prodlouží (desetkrát za každý řád zpřesnění výsledku, tj. $T \propto 1/\text{step_size}$). Mnohem vhodnější je začít s velkým krokem a krok zmenšovat postupně. Tak postupuje tato funkce: začne s krokem `initial_step_size` a pokaždé, když se jí nepodaří `max_failed_steps`-krát provést úspěšný krok, zmenší délku kroku na polovinu. Výpočet probíhá do chvíle, dokud je délka kroku větší než `final_step_size`. Konečná chyba je tedy $\Delta x_i \approx \text{final_step_size}$ a doba výpočtu roste pouze logaritmicky se zmenšováním této chyby, tj. $T \propto 1/\log \text{final_step_size}$.

Obě funkce vracejí řadu s celou náhodnou procházkou. Nalezené minimum je tedy v posledním bodě této řady.

Minimalizace konkrétních funkcí $f(x, y)$ a $g(x, y)$ ze zadání úlohy je provedena v souboru `min_functions.py`.

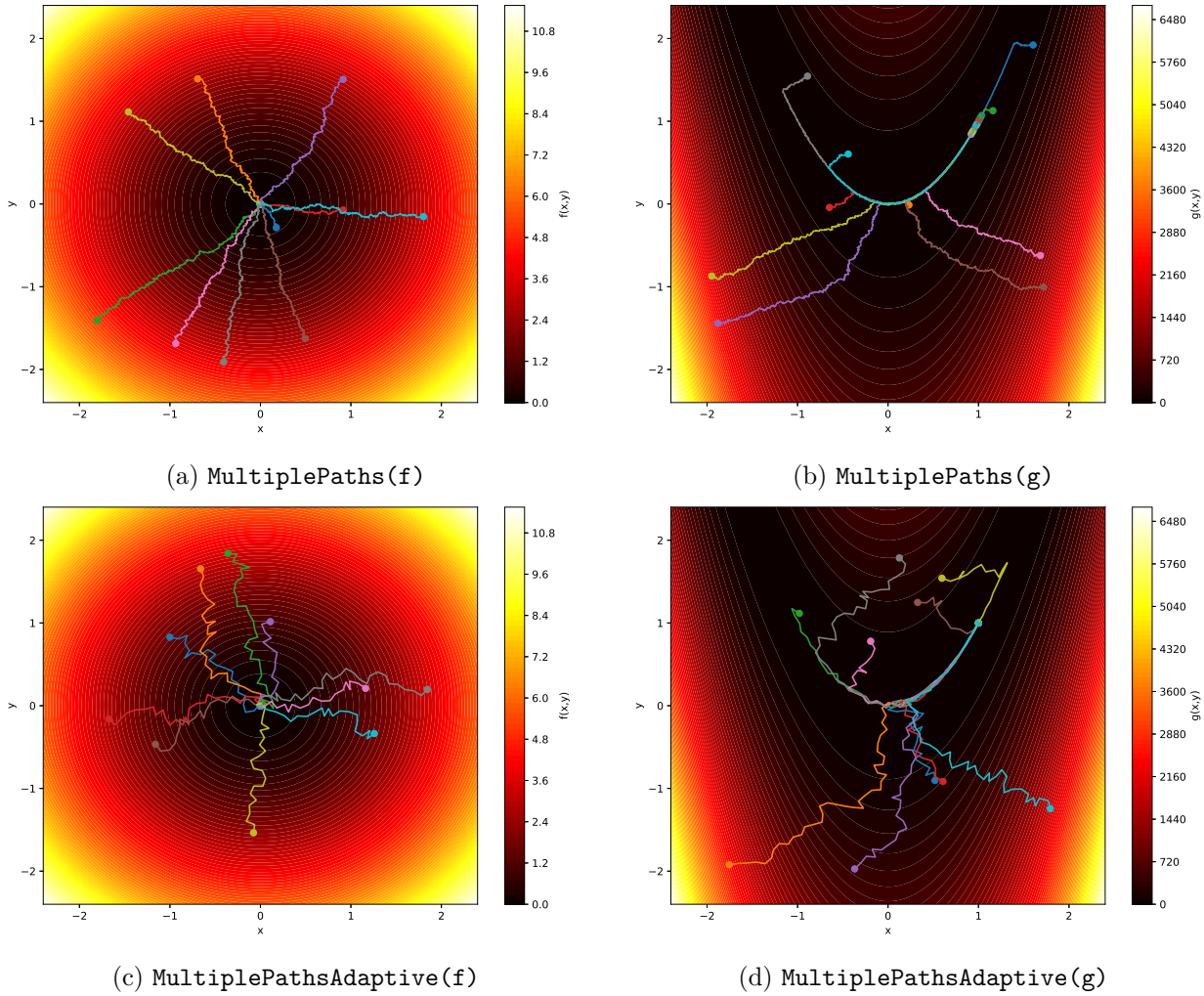
Úloha 7.2: Náhodnou procházku zakreslete jako čáru do grafu společně s konturovým grafem potenciálu. Návod na nakreslení konturového grafu v Pythonu pomocí funkce `matplotlib.pyplot.contourf` naleznete v souboru `contourf.py`.

Řešení 7.2: Vzorový kód je v souboru `min_functions.py` a využívá jako modul `minimize.py`. Obsahuje následující funkce:

- **show_graph:** Vykreslí konturový graf funkce `function` a do něj zakreslí křivky (náhodné procházky) z parametru `paths`. Pokud tento parametr není specifikován, vykreslí pouze konturový graf s funkcí. Meze funkce pro vykreslení jsou dány parametrem `box_size`.
- **multiple_paths:** Vypočítá celkem `num_paths` náhodně vybraných jednoduchých minimalizačních procházek a vykreslí je v grafu společně s konturovým grafem minimalizované funkce. Parametrem `method` lze určit minimalizaci s pevným krokem nebo s adaptivním krokem.
- **multiple_paths:**

Výsledky pro funkce f a g ze zadání jsou zobrazeny na obrázku 8.

Úloha 7.3: Použijte kód pro vícerozměrnou náhodnou procházku a najděte pomocí něho minimum



Obrázek 8: Minimalizace kvadratické funkce (43) a Rosenbrockovy funkce (44) pomocí náhodné procházky a deseti trajektorií náhodně vybraných ze čtverce `initial_box_size = 2`. 1. řádek: Minimalizace metodou `minimize` s délkou kroku `step_size = 0.01` a kritériem ukončení `max_failed_steps = 100`. Chyba určení minima je $\Delta x, y \approx 0.01$. V případě Rosenbrockova minima je chyba velká: Rosenbrockovo „údolí“ je velmi pozvolné podél a strmé napříč, což způsobuje, že trajektorie končí v širokém okolí skutečného minima $(x_{\min}, y_{\min}) = (1, 1)$. Počet kroků, než je výpočet ukončen, je zde ≈ 1000 . 2. řádek: Minimalizace vylepšeným algoritmem s postupně se zmenšujícím krokem. Na počátku je krok velký, `initial_step_size = 0.1`, ale postupně se zmešuje až k `final_step_size = 10^{-6}`. To stačí i pro přesné nalezení minima Rosenbrockovy funkce. Počet nutných kroků zde je ≈ 3000 .

funkce čtyř proměnných

$$\begin{aligned}
 h(s, t, u, v) = & \frac{1}{4} (s^2 + t^2 + u^2 + v^2) \\
 & - \frac{1}{2} \left[(s^2 + t^2) (2 - s^2 - t^2 - u^2 - v^2) + (su - tv)^2 \right] \\
 & + \frac{s}{2} \sqrt{2 - s^2 - t^2 - u^2 - v^2}.
 \end{aligned} \tag{45}$$

Řešení 7.3: Volání funkce `minimize.minimize_adaptive(h, dimension=4)` vypočítá minimum v bodě

$$\begin{aligned}
 s_{\min} & \approx -0.913 \\
 t_{\min} = u_{\min} = v_{\min} & \approx 0.000 \\
 h_{\min} \equiv h(s_{\min}, t_{\min}, u_{\min}, v_{\min}) & \approx -0.771.
 \end{aligned} \tag{46}$$

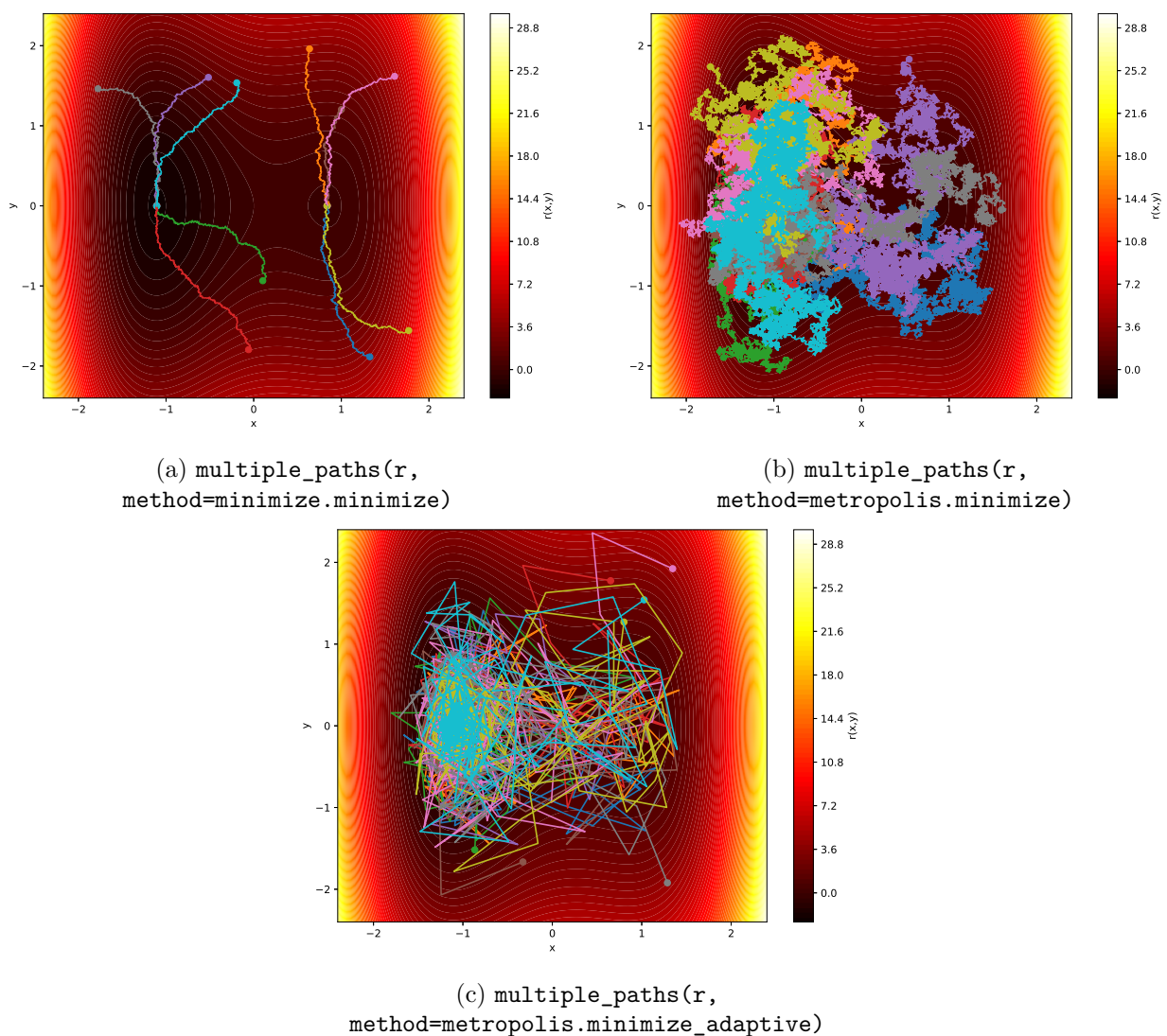
V případě minimalizace této funkce je nutné ošetřit odmocninu, pod kterou se může během náhodné procházky objevit záporné číslo, což způsobí konec výpočtu s chybou. V kódu je to vyřešeno tak, že v případě kroku do této „nedovolené oblasti“ vrátí funkce `h` hodnotu ∞ (v Pythonu `float("inf")`), která je větší než všechna možná jiná čísla, a tím tento krok zakáže. Důležitá podmínka však je, že počáteční bod náhodné procházky musí ležet v definičním oboru funkce `h`.

Úloha 7.4: Naprogramujte Metropolisův algoritmus a odladte ho na případu funkce

$$r(x, y) = x^4 - 2x^2 + x + y^2. \quad (47)$$

Tato funkce má dvě lokální minima (jedná se o vzorovou funkci ze souboru `contourf.py`).

Řešení 7.4: Pro ukázkou, že standardní minimalizace pomocí náhodné procházky vede náhodně do různých lokálních minim, slouží obrázek 9(a).



Obrázek 9: Porovnání různých metod minimalizace dvoujámové funkce (47). (a) Minimalizace obyčejnou náhodnou procházkou, která dokonverguje do náhodného lokálního minima. Následně je možné vybrat to minimum, které je hlubší. (b) Metropolisův algoritmus s konstantní teplotou $T = 1$. Kvůli vysoké teplotě dráha výrazně fluktuuje, avšak postupně dokonverguje k okolí hlubšího globálního minima. (c) Metropolisův algoritmus s postupně se zmenšujícím krokem a teplotou. Zde do globálního minima postupně zkonvergují všechny trajektorie. Počet použitých kroků je 2000.

Metropolisův algoritmus je naprogramován v souboru `metropolis.py`.

- **minimize:** Minimalizuje funkci `function` se zadanou teplotou `temperature` a s fixní délkou kroku `step_size`. Výpočet je ukončen přesně po `max_steps` krocích. Kritérium ukončení výpočtu z metody bez teploty zde nebude fungovat, jelikož konečná teplota způsobuje neustálý „tepelný pohyb“, nikdy tedy nedojde k úplnému zastavení, ani pokud dosáhneme minima funkce.

Výsledek je zobrazen na obrázku 9(b). Je vidět, že oproti případu bez teploty se dostaneme častěji k okolí globálního minima, avšak důsledkem celkem vysoké teploty nalezená poloha výrazně fluktuuje. Navíc, a to zde není zobrazeno, výsledek velmi silně závisí na kombinaci teploty a délky kroku.

- **minimize_adaptive:** Vylepšení spočívající v postupném zmenšování teploty a zároveň kroku. Výpočet začne s krokem `initial_step_size` a teplotou `initial_temperature` a po každých `num_steps_change` krocích dělu kroku `i` teplotu vydělí dvěma. Výpočet je ukončen po dosažení délky kroku `final_step_size`. Příklad realizovaných procházek pro funkci (47) je na obrázku 9(c).

Úloha 7.5: Prostudujte dokumentaci k funkci `minimize` a vytvořte kód, který tuto funkci využije k najetí minima všech doposud studovaných funkcí dvou a více proměnných. Pokud programujete v jiném programovacím jazyku, nalezněte odpovídající minimalizační funkci či knihovnu a použijte ji.

Řešení 7.5: Knihovní funkci voláme příkazem `minimize(f, initialCondition)`, kde parametr `initialCondition` je n -tice udávající počáteční bod, ze kterého minimalizační procedura vystartuje.

V případě funkce s více lokálními minimy tato knihovní funkce nenajde nutně minimum globální. Lze však pomocí volitelného parametru `method` vybrat metodu minimalizace, která bude úspěšnější. Pokročilá varianta Metropolisova algoritmu je v knihovně `scipy.optimize` naprogramována pod názvem `basinhopping`.

7.3 Shrnutí

- Jeden z nejjednodušších algoritmů na hledání minima (maxima) funkce je pomocí náhodné procházky. K její implementaci stačí mít generátor náhodných čísel.
- Úspěch algoritmů založených na náhodné procházce je podmíněn tím, že každý krok procházky musí vést do libovolného směru se stejnou pravděpodobností. Při procházce v rovině toho lze docílit náhodným generováním úhlu. Ve vícerozměrné procházce je nejvýhodnější použít algoritmus založený na Gaussovském náhodném vektoru (za předpokladu, že umíme generovat čísla z Gaussovského rozdělení; jeden jednoduchý takový generátor bude obsahem dalších cvičení).
- Obecná funkce může mít více lokálních minim, přičemž náhodná procházka nás zavede do jednoho z nich, které nemusí být nejhlubší (globální). K nalezení globálního minima lze využít Metropolisův algoritmus, který k náhodné procházce přidá tepelný pohyb, a tím umožní „vyskočit“ z mělkého lokálního minima.

Metropolisův algoritmus se využívá i pro jiné termodynamické úlohy, například k modelování spinových systémů při konečné teplotě, čímž lze studovat fázový přechod feromagnet $\leftrightarrow$ paramagnet.

8 Histogram

V tomto cvičení budeme pokračovat s náhodnými čísly, se kterými již pracovali při programování náhodné procházky v 6. sekci. Zde se podíváme hlouběji na jejich vlastnosti, naučíme se zobrazit hustotu pravděpodobnosti jejich rozdělení (histogram), vytvoříme triviální generátor náhodných čísel s Gaussovským normálním rozdělením a generátor čísel s libovolným rozdělením zadaným hustotou pravděpodobnosti nebo distribuční funkcí.

Histogram je jeden z klíčových objektů v mnoha oblastech fyziky, kde se pracuje s náhodnými veličinami. To je v podstatě celá kvantová mechanika, a tudíž obory jako je atomová, jaderná, či subjaderná fyzika. S náhodnými veličinami se setkáte samozřejmě také v klasické statistické fyzice, ale také například v meteorologii či dalších oborech. Stojí proto za to se s histogramem seznámit podrobně.

8.1 Základní definice a tvrzení z teorie pravděpodobnosti

V následujícím textu budeme značit X spojitou náhodnou veličinu³⁶ s hodnotami v intervalu $x \in \langle a, b \rangle$,³⁷ kde může být $a = -\infty$ a $b = \infty$. Důležité pojmy a vztahy pro nás budou:

- **Hustota pravděpodobnosti $\rho(x)$:** Pravděpodobnost, že náhodná veličina X bude nabývat hodnoty z intervalu $\langle x_1, x_2 \rangle \subset \langle a, b \rangle$, je

$$\Pr[x_1 \leq X \leq x_2] = \int_{x_1}^{x_2} \rho(x) dx. \quad (48)$$

Hustota pravděpodobnosti je normalizovaná na definičním oboru,

$$\int_a^b \rho(x) dx = 1 \quad (49)$$

(pravděpodobnost, že bude náhodná veličina nabývat libovolné ze svých povolených hodnot, je 1 = jistý jev). Hustotu pravděpodobnosti lze vždy rozšířit na celou množinu $\mathbb{R}$, pokud dodefinujeme $\rho(x) = 0$ pro $x < a$ a $x > b$.

- **Distribuční funkce (kumulovaná hustota pravděpodobnosti) $F(x)$:** Neklesající spojitá funkce s oborem hodnot $\langle 0, 1 \rangle$ daná integrálem hustoty pravděpodobnosti³⁸

$$F(x) = \int_a^x \rho(x') dx'. \quad (51)$$

Platí tedy díky normalizaci (49)

$$\begin{aligned} F(a) &= 0, \\ F(b) &= 1. \end{aligned}$$

³⁶V teorii pravděpodobnosti se náhodné veličiny značí obvykle velkým písmenem.

³⁷Náhodná veličina X je ve skutečnosti velmi abstraktní objekt. Obecně se definuje na měřitelném prostoru $(\mathcal{X}, \mathcal{A}, \mu)$, kde $\mathcal{X}$ je množina možných hodnot náhodné veličiny X , $\mathcal{A}$ je σ -algebra nad množinou $\mathcal{X}$ (neprázdný systém množin uzavřený na spočetné sjednocení a obsahující prázdnou množinu a množinu $\mathcal{X}$) a μ je míra množiny $\mathcal{M} \subset \mathcal{X}$ (nezáporná σ -aditivní množinová funkce nulová pro prázdnou množinu a jednotková pro celou množinu $\mathcal{X}$). Tato definice v sobě zahrnuje jak náhodné veličiny s diskrétními možnými hodnotami (jako je například hod kostkou), tak náhodné veličiny se spojitými možnými hodnotami, kterým se věnujeme v této sekci.

³⁸Nebo obráceně, hustota pravděpodobnosti je derivace distribuční funkce,

$$\rho(x) = \frac{dF}{dx}. \quad (50)$$

Rozšíříme-li obor hodnot náhodné veličiny na všechna reálná čísla stejným způsobem, jako jsme naznačili u hustoty pravděpodobnosti, platí navíc

$$\begin{aligned} F(x < a) &= 0, \\ F(x > b) &= 1. \end{aligned}$$

Pravděpodobnost, že náhodná veličina X bude nabývat hodnoty z intervalu $\langle x_1, x_2 \rangle$, je pak

$$\Pr[x_1 \leq X \leq x_2] = F(x_2) - F(x_1). \quad (52)$$

- **Střední hodnota:**³⁹

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x\rho(x)dx. \quad (53)$$

- **Rozptyl:**

$$\sigma_X^2 = \mathbb{E}[X^2] - \mathbb{E}[X]^2 = \int_{-\infty}^{\infty} x^2\rho(x)dx - \left[\int_{-\infty}^{\infty} x\rho(x)dx \right]^2. \quad (54)$$

- **Výběrová střední hodnota:** Pokud máme soubor N hodnot náhodné veličiny X , které označíme $\{x_1, x_2, \dots, x_N\}$ (výběr), pak výběrová střední hodnota je dána aritmetickým průměrem,

$$\bar{X} = \frac{1}{N} \sum_{j=1}^N x_j. \quad (55)$$

Čím mohutnější máme výběr, tím lépe výběrová střední hodnota aproximuje střední hodnotu,

$$\bar{X} \xrightarrow{N \rightarrow \infty} \mathbb{E}[X]. \quad (56)$$

- **Histogram:** Graf (obvykle sloupcový), který aproximuje distribuční funkci náhodné veličiny X na základě hodnot výběru $\mathcal{V} = \{x_j, j = 1, \dots, N\}$. Graf se skládá z $H \ll N$ intervalů (sloupců) obvykle konstantní šířky pokrývající obor hodnot náhodné veličiny $\langle a, b \rangle$, přičemž výška sloupce na konkrétním intervalu je rovna počtu hodnot z výběru $\mathcal{V}$, které do intervalu padnou. Pokud histogram správně nanormujeme, získáme (poněkud zubatou) aproximaci distribuční funkce. Aproximace bude tím lepší, čím větší bude výběr a čím větší bude počet intervalů histogramu.
- **Nezávislé náhodné veličiny:** Dvě náhodné veličiny X a Y jsou nezávislé, pokud jedna neovlivňuje druhou. Sdružená hustota pravděpodobnosti nezávislých náhodných veličin je dána součinem dílčích hustot pravděpodobnosti,

$$\rho_{X,Y}(x, y) = \rho_X(x)\rho_Y(y). \quad (57)$$

Například věk a výška náhodně vybrané osoby nejsou nezávislé veličiny (pro děti bude rozdělení jejich výšek jiné než pro dospělé), zatímco věk osoby a její krevní skupina nezávislé veličiny jsou.

- **Součet dvou náhodných veličin:** Pokud máme náhodnou veličinu X s hustotou pravděpodobnosti $\rho_X(x)$ a náhodnou veličinu Y s hustotou pravděpodobnosti $\rho_Y(y)$, přičemž obě náhodné veličiny jsou nezávislé, pak náhodná veličina

$$Z = X + Y \quad (58)$$

bude mít hustotu pravděpodobnosti ρ_Z danou *konvolucí* hustot ρ_X a ρ_Y ,

$$\rho_Z(z) = \int_{-\infty}^{\infty} \rho_X(\xi)\rho_Y(z - \xi)d\xi. \quad (59)$$

³⁹Expectation value

Střední hodnota a rozptyl náhodné veličiny Z jsou dány součtem

$$\begin{aligned} E[Z] &= E[X] + E[Y], \\ \sigma_Z^2 &= \sigma_X^2 + \sigma_Y^2. \end{aligned} \quad (60)$$

- **Centrální limitní věta:** Je-li náhodná veličina Y daná součtem M vzájemně nezávislých náhodných veličin $X^{(1)}, X^{(2)}, \dots, X^{(M)}$ se shodným rozdělením s hustotou pravděpodobnosti $\rho(x) = \rho_{X^{(j)}}(x)$, jehož střední hodnota je $\mu \equiv E[X^{(j)}] < \infty$ a $\sigma^2 \equiv \sigma_{X^{(j)}}^2 < \infty$, $j = 1, \dots, M$, pak

$$Y \sim N(M\mu, M\sigma^2), \quad (61)$$

kde $N(\mu, \sigma^2)$ je Gaussovo normální rozdělení se střední hodnotou μ a rozptylem σ^2 . Zcela ekvivalentně lze zavést náhodnou veličinu U jako přeškálovanou veličinu Y a psát

$$U \equiv \frac{Y - M\mu}{\sqrt{M\sigma^2}} \xrightarrow{M \rightarrow \infty} N(0, 1). \quad (62)$$

Hustota pravděpodobnosti normálního rozdělení je dána vzorcem (66).

8.2 Příklady náhodných veličin

- **Rovnoměrné rozdělení $R(a, b)$ na intervalu $\langle a, b \rangle$:**

$$\rho_R(x) = \frac{1}{b-a} = \text{konst.} \quad (63)$$

$$E[R] = \frac{a+b}{2} \quad (64)$$

$$\sigma_R^2 = \frac{(b-a)^2}{12}. \quad (65)$$

- **Gaussovo normální rozdělení $N(\mu, \sigma^2)$:**

$$\rho_N(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (66)$$

$$E[N] = \mu \quad (67)$$

$$\sigma_N^2 = \sigma^2. \quad (68)$$

- **Poissonovo rozdělení:** Diskrétní rozdělení udávající počet nezávislých jevů k v zadaném intervalu (například počet lidí, které potkáte na mostě cestou z Holešovic do Troji, počet kapek, které vám za nějakou časovou jednotku spadne na hlavu, počet rozpadů radioaktivního prvku ve vzorku za jednotku času atd.). Rozdělení pravděpodobnosti je

$$P_k = \frac{\lambda^k}{k!} e^{-\lambda}, \quad (69)$$

$$E[P] = \lambda, \quad (70)$$

$$\sigma_P^2 = \lambda, \quad (71)$$

přičemž parametr λ udává zároveň střední hodnotu a zároveň rozptyl rozdělení.

Úloha 8.1: Dokažte vztahy (64)–(65) a (70)–(71).

Řešení 8.1: *K dokázání (64)–(65) vyjdeme z definičních vztahů pro střední hodnotu a rozptyl (53)–(54):*

$$E[R] = \int_a^b \frac{x}{b-a} dx = \frac{1}{b-a} \left[\frac{x^2}{2} \right]_a^b = \frac{1}{b-a} \frac{b^2 - a^2}{2} = \frac{a+b}{2}, \quad (72)$$

$$\begin{aligned} \sigma_R^2 &= \int_a^b \frac{x^2}{b-a} dx - \left(\frac{a+b}{2} \right)^2 = \frac{1}{b-a} \frac{b^3 - a^3}{3} - \frac{(a+b)^2}{4} \\ &= \frac{a^2 + ab + b^2}{3} - \frac{a^2 + 2ab + b^2}{4} = \frac{a^2 - 2ab + b^2}{12} = \frac{(b-a)^2}{12}. \end{aligned} \quad (73)$$

Poissonovo rozdělení je diskrétní, je tudíž nutné použít diskrétní analogii vztahů (53)–(54) (nahrazení integrálů sumami):

$$E[P] = \sum_{k=0}^{\infty} k P_k = \sum_{k=1}^{\infty} \frac{\lambda^k}{(k-1)!} e^{-\lambda} = \left| \begin{array}{c} \text{substitute} \\ l = k-1 \end{array} \right| = \sum_{l=0}^{\infty} \frac{\lambda^{l+1}}{l!} e^{-\lambda} = \lambda, \quad (74)$$

$$\begin{aligned} \sigma_P^2 &= \sum_{k=0}^{\infty} k^2 P_k - \lambda^2 = \sum_{k=1}^{\infty} \frac{k \lambda^k}{(k-1)!} e^{-\lambda} - \lambda^2 \\ &= \sum_{k=1}^{\infty} \frac{(k-1) \lambda^k}{(k-1)!} e^{-\lambda} + \sum_{k=1}^{\infty} \frac{\lambda^k}{(k-1)!} e^{-\lambda} - \lambda^2 \\ &= \sum_{k=2}^{\infty} \frac{\lambda^k}{(k-2)!} e^{-\lambda} + \lambda - \lambda^2 = \lambda^2 + \lambda - \lambda^2 = \lambda. \end{aligned} \quad (75)$$

8.3 Výběr z neznámého rozdělení

V praxi se můžeme setkat se situací, kdy máme zadanou hustotu pravděpodobnosti či distribuční funkci nějakého komplikovaného rozdělení a chceme generovat výběr z tohoto rozdělení.

- **Známe-li hustotu pravděpodobnosti rozdělení $\rho(x)$:** Nejjednodušší metoda v tomto případě je vepsat funkci $\rho(x)$ do obdélníku $\langle a, b \rangle \times \langle c, d \rangle$,⁴⁰ nagenarovat číslo rovnoměrně z tohoto obdélníku, a pokud padne pod křivku $\rho(x)$, vezmeme ho, v opačném případě ho zahodíme (tzv. *hit-and-miss metoda*, se kterou se ještě potkáme u metody Monte-Carlo).
- **Známe-li distribuční funkci $F(x)$:** V tomto případě využijeme skutečnosti, že obor hodnot distribuční funkce je $F(x) \in \langle 0, 1 \rangle$. Stačí tedy generovat náhodné číslo y z rovnoměrného rozdělení na intervalu $\langle 0, 1 \rangle$ a číslo

$$x = F^{-1}(y), \quad (76)$$

kde F^{-1} je inverzní funkce k F , bude z rozdělení s danou F . Pokud neznáme inverzní funkci, řešíme numericky rovnici

$$F(x) = y, \quad (77)$$

která však s ohledem na vlastnosti distribuční funkce popsané v sekci 8.1 má téměř vždycky jedno a pouze jedno řešení.

Matematictěji zapsáno: je-li $R = R(0, 1)$ náhodná veličina s rovnoměrným rozdělením na intervalu $\langle 0, 1 \rangle$, pak

$$X = F^{-1}(R) \quad (78)$$

je náhodná veličina s rozdělením daným distribuční funkcí F .

⁴⁰Hustota pravděpodobnosti bývá obvykle definovaná na neomezeném intervalu. Pak je nutné určitou část funkce $\rho(x)$ oříznout. To funguje dobře v případě náhodných veličin s rychle ubývajícím hustotou pravděpodobnosti, jako je například Gaussovo rozdělení $N(0, 1)$, kde při oříznutí $x \in \langle -3\sigma, 3\sigma \rangle = \langle -3, 3 \rangle$ zanedbáme jen 3% možných hodnot. V případě dlouhodobých rozdělení, jako je například lognormální rozdělení nebo Gamma rozdělení, musíme obdélník volit velmi dlouhý, čímž se tato metoda stává velmi neefektivní.

Úloha 8.2: Naprogramujte funkci pro výpočet histogramu: na vstupu bude pole hodnot (výběr z nějakého rozdělení) a počet intervalů histogramu; na výstupu bude pole, jehož každý prvek bude odpovídat jednomu intervalu histogramu a ponese počet hodnot, které do tohoto intervalu padnou ze vstupního pole. Nejeфекtivnějším algoritmu stačí jeden průchod vstupním polem (jeden cyklus). Pokuste se na něj přijít.

Výstupní pole funkce vykreslete jako čárový graf.⁴¹

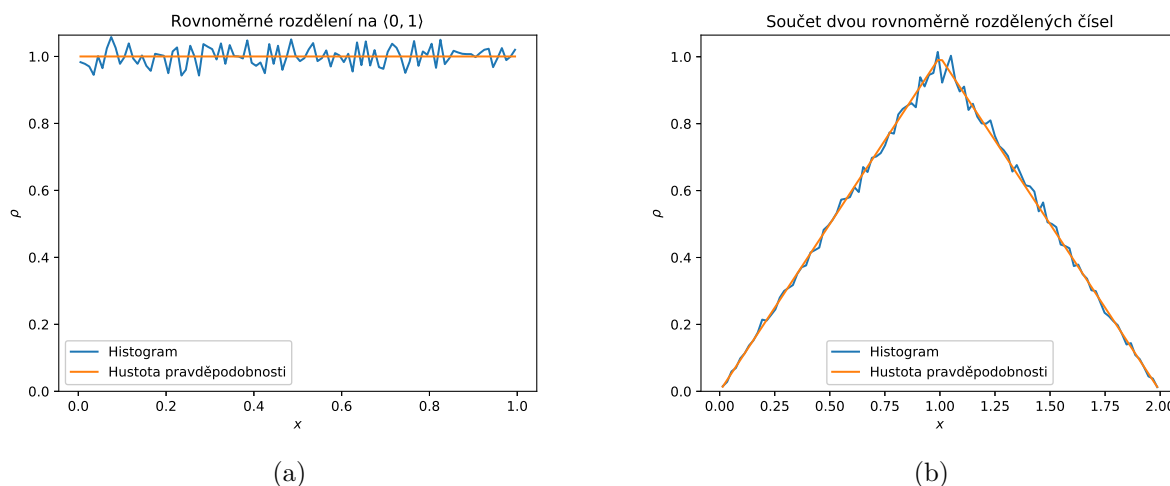
Řešení 8.2: Vzorový výpočet histogramu je naprogramován v souboru `histogram.py` ve funkci `histogram`. Klíčové jsou dva řádky:

```
index = int((d - min_value) / bin_width)
histogram[index] += 1
```

Hodnota, kterou chceme zařadit do příslušného okénka histogramu, se nachází v proměnné `d`. Z ní spočítáme celočíselný index v intervalu mezi $0 \leq \text{index} < \text{num_bins}$ a pak v poli `histogram` na příslušném indexu přidáme jedničku. Histogramované pole hodnot `data` tedy stačí procházet jen jednou, algoritmus má tudíž časovou náročnost $\mathcal{O}(N)$. Nutným předpokladem pro použití tohoto algoritmu je, že všechna okénka histogramu musejí mít stejnou šířku. Funkce `histogram` pak vrací pole s x -ovými hodnotami (středů histogramů) a pole s hotovým histogramem. Pro snazší porovnání s hustotami pravděpodobnosti lze navíc nastavit parametr `normalize=True`, výsledný histogram pak bude splňovat

$$\sum_{i=1}^{\text{num_bins}} h_i w_i = 1, \quad (79)$$

kde h_i je hodnota histogramu v i -tém okénku a $w_i \equiv \text{bin_width}$ šířka okénka.



Obrázek 10: Srovnání histogramu získaného z $n = 10^5$ hodnot vybraných z daného rozdělení a porovnání s teoretickou hustotou pravděpodobnosti (a) rovnoměrného rozdělení (63) ($a = 0, b = 1$) a (b) součtu dvou rovnoměrných rozdělení (81). Počet intervalů histogramu je v obou případech $N = 100$.

Příklady histogramů a jejich vykreslení do grafu jsou v řešení následujícího úkolu a na obrázku 10.

Úloha 8.3: Otestujte funkci z předchozího úkolu na následujících vstupních datech:

1. Výběr z rovnoměrného rozdělení na intervalu $\langle 0, 1 \rangle$ (v Pythonu generované pomocí `random()` z knihovny `random`, resp. pomocí `generator.random()` z knihovny `numpy.random`, jak je shrnuto v sekci 3.2.6).

⁴¹Knihovna `matplotlib` obsahuje funkce na přímé vykreslení sloupcového histogramu, například `hist`.

2. Výběr ze **součtu dvou** rovnoměrných rozdělení na intervalu $\langle 0, 1 \rangle$. Hustota pravděpodobnosti výsledného rozdělení je dána konvolucí (59). Vypočítejte analyticky pomocí tohoto vzorce, jak bude hustota pravděpodobnosti vypadat, a porovnejte se získaným histogramem.
3. Výběr ze **součtu** M rovnoměrných rozdělení na intervalu $\langle 0, 1 \rangle$. Přesvědčte se, že již pro celkem malé M platí centrální limitní věta (61) a výsledné rozdělení se blíží normálnímu rozdělení $N(\mu, \sigma)$. Jaká bude střední hodnota μ a rozptyl σ tohoto rozdělení?

Pro pěkné grafy volte alespoň $N = 10000$ (počet prvků výběru) a $H = 100$ (počet intervalů histogramu).

Řešení 8.3: Testování funkce `histogram` pro různá rozdělení je v souboru `distributions.py`.

1. Hustota pravděpodobnosti rovnoměrného rozdělení je zobrazena funkcí `uniform`. Výsledný graf je na obrázku 10.
2. Hustota pravděpodobnosti součtu dvou rovnoměrných rozdělení je podle (59)

$$\rho_2(z) = \int_0^1 \rho_1(\xi) \rho_1(z - \xi) d\xi = \int \rho_1(z - \xi) d\xi, \quad (80)$$

kde $\rho_1(\xi) = 1$ pro $0 \leq \xi \leq 1$ a $\rho_1(\xi) = 0$ jinde, čímž se v integrálu zbavíme faktoru $\rho_1(\xi)$, neboť je rovný 1 na celém integračním intervalu. Integraci nyní rozdělíme na čtyři intervaly:

$$\begin{aligned} z < 0 : & \quad \rho_2(z) = 0, \\ 0 < z < 1 : & \quad \rho_2(z) = \int_0^z d\xi = z, \\ 1 < z < 2 : & \quad \rho_2(z) = \int_{z-1}^z d\xi = 1 - (1 - z) = 2 - z, \\ z > 2 : & \quad \rho_2(z) = 0. \end{aligned}$$

To lze zapsat zjednodušeně jako

$$\rho_2(z) = \begin{cases} 1 - |1 - z| & 0 < z < 2, \\ 0 & \text{jinak.} \end{cases} \quad (81)$$

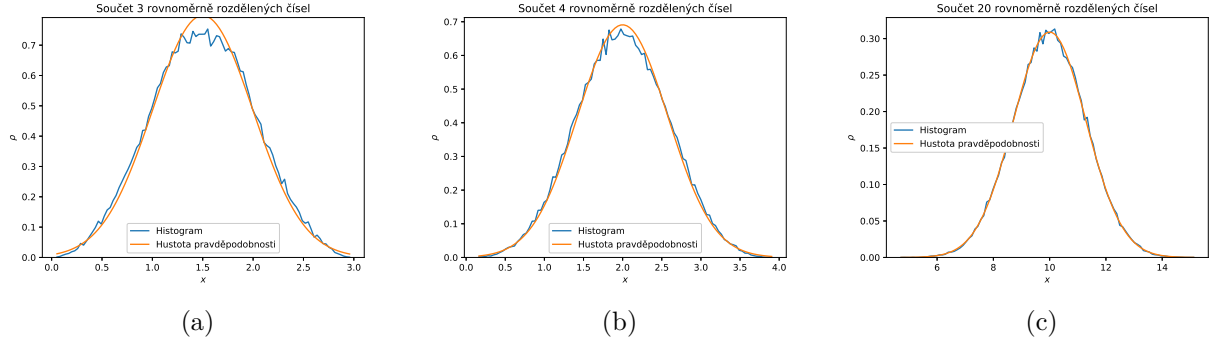
Rozdělení má trojúhelníkový tvar, což odráží skutečnost, že při součtu dvou rovnoměrně rozdělených čísel na intervalu $\langle 0, 1 \rangle$ je mnohem více možností, jak realizovat součet okolo 1, než součet na krajích intervalu (stejně jako při hodu dvěma kostkami je největší pravděpodobnost součtu rovna 7, jak vědí všichni hráči deskové hry *Osadníci z Katanu* a je větší než pravděpodobnost součtu 12, neboť součet 7 můžeme realizovat šesti způsoby z 36 celkových možných výsledků hodu dvěma kostkami, zatímco součet 12 jen jedním způsobem). Hustota pravděpodobnosti se počítá funkcí `sum_2_uniform` a výsledný graf je zobrazen na obrázku 10.

3. Výpočet hustoty pravděpodobnosti součtu více rovnoměrných rozdělení je naprogramován ve funkci `sum_m_uniform` a porovnání se správně nanormovanou Gaussovou je na obrázku 11.

Úloha 8.4: Na základě centrální limitní věty (62) vytvořte jednoduchý generátor čísel s normálním Gaussovským rozdělením $N(0, 1)$. Jaké je optimální hodnota m , abychom získali dostatečně přesnou aproximaci normálního rozdělení, a přitom použili co nejméně algebraických operací?

Řešení 8.4: Ideální počet sečtených čísel z rovnoměrného rozdělení pro získání velmi dobré aproximace čísla z normálního Gaussova rozdělení je $M = 12$, neboť pro tuto hodnotu:

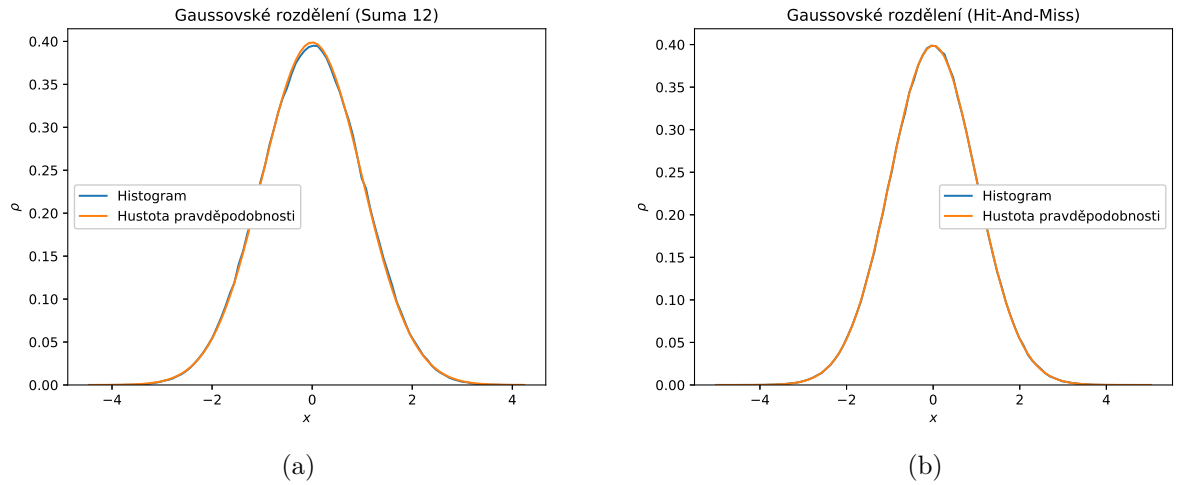
- Rozptyl výsledného rozdělení je $\sigma = 1$ díky vztahům (62) a (65).



Obrázek 11: Srovnání histogramu získaného z $N = 10^5$ hodnot daných součtem (a) $M = 3$, (b) $M = 4$ a (c) $M = 20$ rovnoměrně rozdělených náhodných čísel, s Gaussovskou hustotou pravděpodobnosti. Již pro $M = 3$ je shoda velmi dobrá a centrální limitní věta (61) je přibližně splněna. V případě $M = 20$ je již rozdíl od Gaussovského rozdělení prakticky nepozorovatelný. Počet intervalů histogramu je ve všech případech $H = 100$.

- *Hodnota nagenерованého čísla je v intervalu $(0, 12)$, se střední hodnotou 6. Pokud tuto střední hodnotu odečteme dle vzorce (62), obdržíme rozdělení se střední hodnotou 0 a zahrnující interval 6σ , který pokrývá 99.9999998% Gaussovského rozdělení.*

Generátor je naprogramován v souboru `gaussian.py`, funkce `generator_clt`. Srovnání generátoru s odpovídající Gaussovskou je na obrázku 12(a).



Obrázek 12: Srovnání jednoduchých generátorů Gaussovsky rozdělených náhodných čísel. (a) Generátor založený na použití Centrální limitní věty (62): Gaussovské náhodné číslo je získáno jako součet $m = 12$ rovnoměrně rozdělených náhodných čísel. (b) Generátor založený na hit-and-miss metodě použité na hustotu pravděpodobnosti Gaussovského rozdělení. V obou případech je pro zobrazovaný histogram použito $n = 10^6$ náhodných čísel a počet intervalů histogramu je $N = 100$. U generátoru (a) je pozorována drobná odchylka v okolí maxima hustoty pravděpodobnosti, u generátoru (b) žádná odchylka pozorována není. Nutno zdůraznit, že generátor (a) je zhruba o řád rychlejší než generátor (b), jak je ukázáno v úloze 8.6.

Úloha 8.5: *Nakreslete histogram pro rozdělení hodnot v jednotlivých intervalech histogramů z úlohy 8.3. Jaké očekáváte statistické rozdělení v tomto případě?*

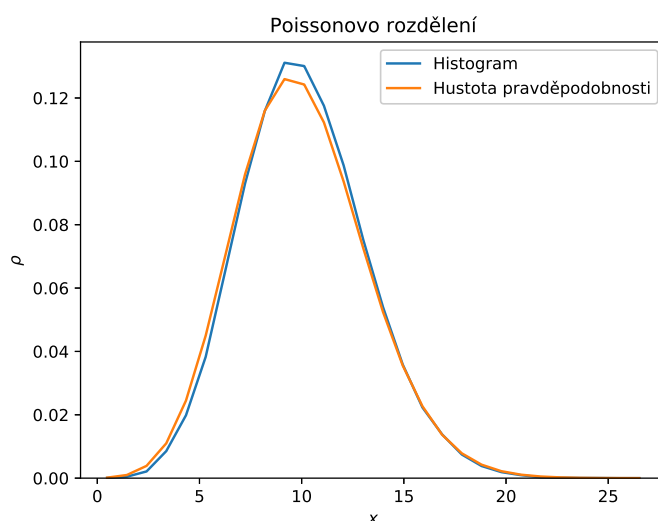
Řešení 8.5: *Počet hodnot v jednotlivých intervalech splňuje předpoklady pro Poissonovo rozdělení, očekáváme tedy Poissonovo rozdělení se parametrem*

$$\lambda = \frac{\text{num_values}}{\text{num_bins}}, \quad (82)$$

který udává zároveň střední hodnotu (70) a zároveň rozptyl (71). Chceme-li toto rozdělení zobrazit, je potřeba jisté delikátnosti.

- Počty hodnot v okénkách (nenormovaného) histogramu jsou přirozená čísla. Pro zobrazení jejich rozdělení je tedy vhodné volit sekundární histogram se šířkou intervalu 1 (nebo s jinou celočíselnou šířkou).
- Hezčí rozdělení získáme pro malé parametry λ (pro λ velké se Poissonovo rozdělení blíží Gaussovskému rozdělení). V kódu volím hodnoty tak, aby $\lambda = 10$.

Srovnání příslušného histogramu s teoretickým rozdělením (69) je vypočteno funkcí `poisson` ze souboru `distributions.py` a je vykresleno na obrázku 13.



Obrázek 13: Poissonovo rozdělení, získané z fluktuací počtu hodnot histogramu s $H = 10^5$ intervaly a $N = 10^6$ rovnoměrně rozdělenými vstupními čísly. Střední hodnota a rozptyl takto získaného Poissonova rozdělení je $\lambda = 10$.

Úloha 8.6: Na základě známé hustoty pravděpodobnosti (66) vytvořte generátor čísel s Gaussovským normálním rozdělením. Porovnejte jeho rychlost s generátorem založeným na centrální limitní větě, který jste naprogramovali v úloze 8.4 a s generátorem z některé z knihoven.⁴²

Řešení 8.6: Generátor je naprogramován v souboru `gaussian.py`, funkce `generator_hm`, a porovnání histogramu takto nagenеровaných náhodných hodnot s teoretickou hustotou pravděpodobnosti je na obrázku 12(b). Nagenеровání 10^6 Gaussovsky rozdělených náhodných čísel trvá na mém PC⁴³

- 4s: funkce `generator.normal()` z knihovny `numpy`,
- 16s: součet 12 rovnoměrně rozdělených čísel,
- 111s: hit-and-miss metoda.

Úloha 8.7: Vytvořte generátor čísel z rozdělení daném distribuční funkcí

$$F(x) = \frac{1}{2} \left(1 + \frac{2}{\pi} \arctan x \right). \quad (83)$$

Jak vypadá analyticky hustota pravděpodobnosti? Nakreslete histogram a porovnejte.

⁴²Porovnání můžete provést tak, že nagenergusujete větší množství čísel, například $N = 10^6$, různými metodami a změříte dobu výpočtu.

⁴³Pokud k nagenеровání n hodnot použijeme knihovní funkci s počtem v argumentu, `generator.normal(1000000)`, výpočet trvá zlomek vteřiny. Velká část výpočetního času je tudíž způsobena voláním funkcí a prováděním cyklu.

Řešení 8.7: Hustota pravděpodobnosti je dána derivací distribuční funkce podle vztahu (50), tj.

$$\rho(x) = \frac{dF}{dx} = \frac{1}{\pi} \frac{d}{dx} \arctan x = \frac{1}{\pi} \frac{1}{1+x^2}. \quad (84)$$

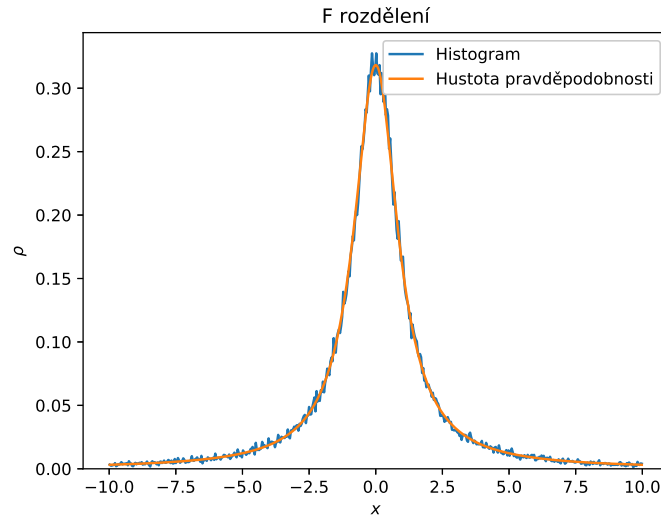
Toto rozdělení se nazývá Cauchyho či v kvantové fyzice Breit-Wignerovo. Popisuje šířku energetických hladin exponenciálně se rozpadajících systémů.

Ke generování čísel z tohoto rozdělení využijeme vztahu (76). Musíme tedy určit funkci inverzní k distribuční funkci (83), která je

$$F^{-1}(y) = \tan \left[\frac{\pi}{2} (2y - 1) \right], \quad (85)$$

a za y dosazovat čísla z rovnoměrného rozdělení na intervalu $\langle 0, 1 \rangle$.

Histogram čísel nagenерованých z Cauchyho rozdělení a srovnání s hustotou pravděpodobnosti (84) je počítán funkcí `cauchy` souboru `distributions.py` a je zobrazen na obrázku 14. Rozdělení má dlouhý dosah, s rostoucím x klesá jen polynomiálně k nule, pravděpodobnost nagenерování hodnoty daleko od maxima je tudíž velká. V obrázku proto zobrazují jen okno $x \in \langle -10, 10 \rangle$.



Obrázek 14: Cauchyho (Breit-Wignerovo) rozdělení nagenерované z distribuční funkce (83) a porovnané s teoretickou hustotou pravděpodobnosti (84). Počet hodnot pro histogram je $N = 10^5$, počet intervalů $H = 500$.

9 Monte-Carlo metoda

Pod Monte-Carlo⁴⁴ metodou se rozumí, že namísto systematického (a obvykle zdlouhavého) procházení nějakého parametrického prostoru využíváme náhodně generované body a hledané vlastnosti našeho systému určíme statisticky. Metodu poprvé navrhli a použili matematici Stanisław Ulam a John von Neumann ve 40. letech 20. století při výpočtu průchodu neutronů prostředím. V tomto cvičení zúročíme veškeré dosavadní zkušenosti s náhodnými čísly a budeme se zabývat zejména integrací Monte-Carlo.

Možná jste se již setkali se problémem tzv. **Buffonovy jehly**: pomocí náhodného házení jehly (či jakékoliv tyčky) na síť rovnoběžných čar nakreslenou na zemi lze určit číslo π ,

$$\pi \approx \frac{2l}{h} \frac{N_{\text{zásah}}}{N_{\text{celkem}}}, \quad (86)$$

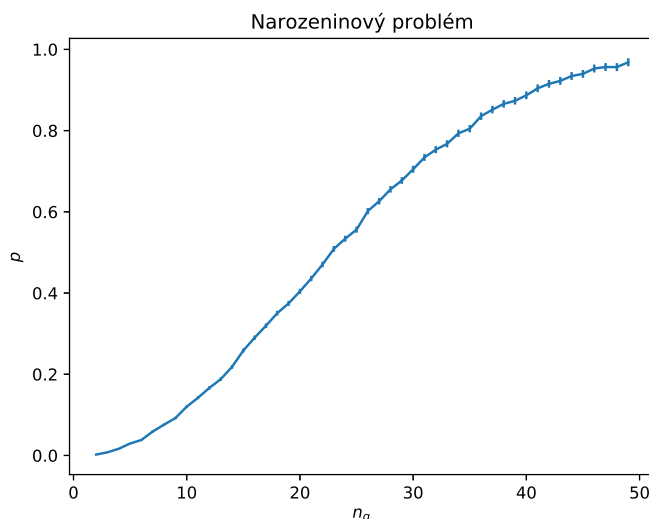
kde h je vzdálenost čar, $l \leq h$ délka jehly, N_{celkem} celkový počet hodů a $N_{\text{zásah}}$ počet hodů, při kterých jehla po dopadu kříží nějakou z čar. Buffonova jehla je názorné experimentální použití metody Monte-Carlo, konkrétně varianty nazývané hit-and-miss. Té jsme se již dotkli v sekci 8.3 a nyní si rozebereme podrobněji.

Úloha 9.1: Metodou Monte-Carlo vyřešte tzv. narozeninový problém: Uvažujte skupinu n lidí. Jaká je pravděpodobnost, že dva lidi ve skupině budou mít narozeniny ve stejný den? Úloha se samozřejmě dá vyřešit *exaktně*, ale zkuste si úlohu vyřešit metodou Monte-Carlo: Pokud nagenujete náhodně N_{celkem} -krát narozeniny n lidí a označíte $N_{\text{zásah}}$ případy, kdy alespoň dvoje narozeniny padnou na stejný den, bude podle zákona velkých čísel hledaná pravděpodobnost rovna

$$p \approx \frac{N_{\text{zásah}}}{N_{\text{celkem}}} \quad (87)$$

(rovnost by nastala pro $N_{\text{celkem}} \rightarrow \infty$). Naprogramujte tuto úlohu a určete,

1. jaká je pravděpodobnost pro skupinu 30 lidí a
2. jak velkou skupinu potřebujete, aby byla pravděpodobnost alespoň 30%.



Obrázek 15: Pravděpodobnost, že ve skupině n_g lidí budou mít alespoň dva lidé narozeniny ve stejný den v roce. Svislými čarami je odhad chyby $\pm 1\sigma$. Počet pokusů pro metodu Monte-Carlo je $N_{\text{celkem}} = 10^4$.

Řešení 9.1: Vzorové řešení je v souboru `birthday.py` a obsahuje dvě funkce:

⁴⁴Monte Carlo je oblast Monaka, ve kterém se nacházely a doposud nacházejí slavná kasina. Odtud název.

- `birthday_coincidence_probability` určí metodou Monte-Carlo pravděpodobnost, že ve skupině o velikosti $n_g = \text{group_size}$ bude alespoň jedna dvojice, která slaví narozeniny ve stejný den v roce. Řešení probíhá ve dvou krocích:

1. nageneryujeme řadu n_g čísel mezi 1 a 366, které udávají den narozenin,⁴⁵
2. zkontrolujeme, zda řada obsahuje dva stejné prvky (ve vzorovém řešení tak, že řadu seřadíme a zkontrolujeme, jestli obsahuje dvě či více stejných čísel v bezprostředně následujících prvcích).

Funkce vrací hodnotu pravděpodobnosti p a odhad chyby $\Delta p \approx 1\sigma_p$.

- `plot_birthday_problem` vykreslí graf pravděpodobností pro interval velikostí skupin. Graf včetně chyb je na obrázku 15.

Na základě těchto funkcí se jednoduše spočítá, že

1. Pro $n_g = 30$ je pravděpodobnost $p \approx 70\%$.
2. Pro $n_g = 17$ je pravděpodobnost $p \approx 31\%$.

Program lze triviálně rozšířit a počítat například pravděpodobnost, že

- alespoň d lidí bude mít narozeniny ve stejný den,
- alespoň 2 lidé budou mít narozeniny ve stejný den i rok.

9.1 Hit-And-Miss

Tato metoda spočívá v jednoduché aplikaci zákona velkých čísel. Její esence je načrtnuta na obrázku 16. Uvažujme nejprve, že chceme změřit plochu s černého obrazce složitěho tvaru znázorněného na panelu (a). Obrazec vepíšeme do jiného jednoduchého obrazce, jehož plochu S známe (nejčastěji obdélník, případně kruh). Poté do tohoto obrazce S náhodně „házíme“ body (křížky) a počítáme, kolikrát se trefíme do černého obrazce (červené křížky). Neznámá hledaná plocha s je pak při již použitém značení

$$s \approx S \frac{N_{\text{zásah}}}{N_{\text{celkem}}}. \quad (88)$$

Analogicky postupujeme při integrování, což je ukázáno na panelu (b). Integrál v mezích (a, b) je plocha pod křivkou funkce $f(x)$ (na obrázku černě vyšrafovaná plocha ohraničená zespodu osou x , shora funkcí $f(x)$ a ze stran modrými čarami $x = a$ a $x = b$). Uvedenou oblast vepíšeme do obdélníku o hranách $l = b - a$ a h s plochou $S = (b - a)h$ a stejnou metodou jako u panelu (a) a pomocí stejného vzorce jako je (88) vypočítáme hodnotu integrálu:

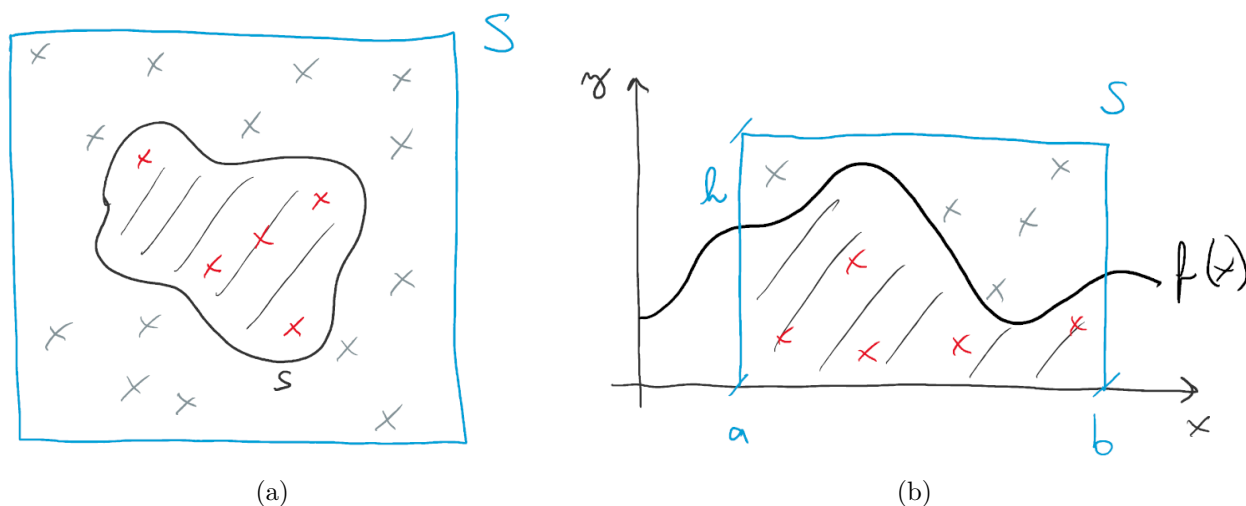
$$\int_a^b f(x)dx \approx S \frac{N_{\text{zásah}}}{N_{\text{celkem}}} = (b - a)h \frac{N_{\text{zásah}}}{N_{\text{celkem}}}. \quad (89)$$

9.1.1 Chyba

Počet zásahů je vlastně počet nezávislých „hodů“, kterými se trefíme do oblasti, jejíž plochu S hledáme. Pokud budeme opakovat metodu hit-and-miss s fixním N_{celkem} , bude mít náhodná veličina $N_{\text{zásah}}$ Poissonovo rozdělení se střední hodnotou (70) a rozptylem (71)

$$\lambda = E[N_{\text{zásah}}] = \sigma_{N_{\text{zásah}}}^2. \quad (90)$$

⁴⁵Řešení počítá i s datem narození 29. února, přičemž předpokládá, že pravděpodobnost všech dat narození je stejná. To však pro narozeniny 29. února neplatí, pravděpodobnost narození s tímto datem je menší, a to přibližně čtvrtinová.



Obrázek 16: Metoda hit-and-miss (a) pro výpočet plochy složitého obrazce a (b) pro výpočet integrálu jednorozměrné funkce. Podrobnosti k obrázku jsou uvedeny v textu.

Budeme-li mít jen jednu realizaci s dostatkem zásahů, můžeme střední hodnotu odhadnout pomocí této realizace, $E[N_{\text{zásah}}] \approx N_{\text{zásah}}$ a absolutní chybu odhadneme směrodatnou odchylkou

$$\Delta N_{\text{zásah}} = \sqrt{\sigma_{N_{\text{zásah}}}^2} \approx \sqrt{N_{\text{zásah}}}. \quad (91)$$

Relativní chyba pak je

$$\delta N_{\text{zásah}} = \frac{\Delta N_{\text{zásah}}}{N_{\text{zásah}}} \approx \frac{1}{\sqrt{N_{\text{zásah}}}}. \quad (92)$$

Tento vzorec platí i pro relativní chybu ve výpočtu plochy (88) či integrálu (89).

Jelikož $N_{\text{zásah}} \propto N_{\text{celkem}}$, z uvedených úvah vyplývá, že

- relativní chyba klesá jako převrácená hodnota odmocniny celkového počtu pokusů N_{celkem} a
- chceme-li zpřesnit výsledek získaný touto metodou desetkrát, musíme zetonásobit počet pokusů.

V praxi za použití běžných výpočetních prostředků lze dosáhnout nanejvýš $N_{\text{celkem}} \approx 10^{10}$, což dá výsledek s relativní chybou minimálně $\delta \approx 10^{-5}$, tj. pět desetinných míst.

9.1.2 Použití

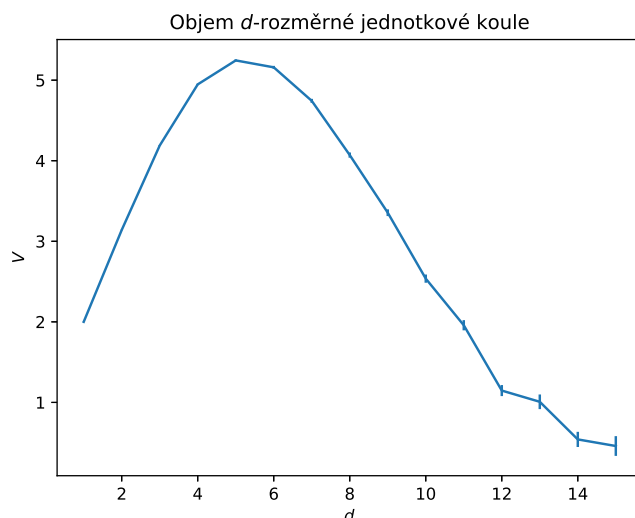
Integrace pomocí metody hit-and-miss se nepoužívá, jelikož je příliš neefektivní. K jejímu úspěšnému použití totiž musíme znát maximum funkce na zadaném intervalu, abychom efektivně určili výšku obdélníku h , a obecně je $N_{\text{zásah}}/N_{\text{celkem}}$ velmi malé číslo (funkce mají dlouhý chvost nebo jsou příliš vysoké), většina „hodů“ jde tedy mimo a i při jejich velkém množství získáme málo zásahů, a tudíž velkou chybu podle vztahu (92).

Na druhou stranu se tato metoda hodí k výpočtu povrchů, objemů či hyperobjemů v případě vícerozměrných objektů.

Úloha 9.2: Vytvořte program na výpočet objemu d -rozměrné jednotkové koule metodou Monte-Carlo.⁴⁶ Pro jakou dimenzi bude tento objem největší číslo?

⁴⁶Pod 1-rozměrnou jednotkovou koulí rozumíme úsečku délky 2, pod 2-rozměrnou jednotkovou koulí jednotkový kruh. Povrch jednotkového kruhu je $S = \pi$, výsledek lze tedy použít i k určení čísla π , aniž byste museli házet Buffonovou jehlou.

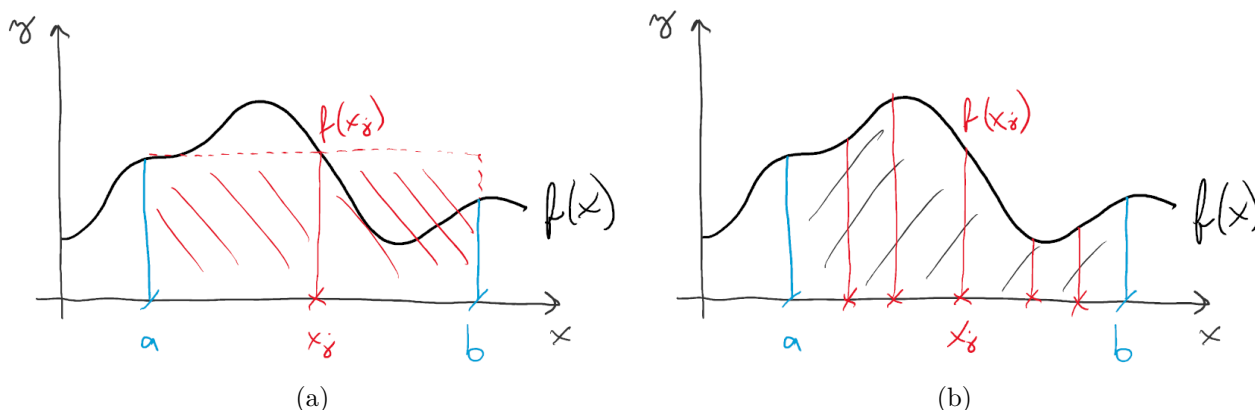
Řešení 9.2: Vzorový kód je v souboru `ball.py`. Funkce `volume` spočítá objem jednotkové koule metodou Monte-Carlo a vrátí současně odhad chyby. Funkce `plot_volumes` vykreslí graf objemů v závislosti na dimenzi, jak je ukázáno na obrázku 17. Největší objem $V \approx 5.27$ dostaneme pro $d = 5$. Zde je na místě jistá opatrnost: srovnáváme trochu „jablka s hruškami“, protože samozřejmě objem různěrozměrných koulí má různou dimenzionalitu. Pro jiné než jednotkové koule tato závislost platit nebude.



Obrázek 17: Závislost objemu jednotkové koule na dimenzionalitě prostoru. Svislémi čarami je zobrazena chyba $\Delta V \approx 1\sigma_V$. Počet pokusů pro metodu Monte-Carlo je pro každý bod $N_{\text{celkem}} = 10^6$.

Z obrázku je rovněž vidět, že s rostoucím d roste chyba výsledku. To je způsobeno tím, že s rostoucí dimenzionalitou se při metodě hit-and-miss čím dál častěji trefujeme mimo jednotkovou kouli, jinými slovy klesá při zadaném počtu pokusů počet zásahů.⁴⁷ Tento jev a jeho příčiny jsme již diskutovali v případě generování náhodného směru v d dimenzích v řešení úlohy 6.3.

9.2 Monte-Carlo integrace



Obrázek 18: Monte-Carlo integrace. Vysvětlení je v hlavním textu.

Pro hledání integrálu je mnohem efektivnější metoda, již lze vysvětlit pomocí obrázku 18. Uvažujme neprve, že známe pár hodnot $(x_j, f(x_j))$, nic víc, nic méně. Na základě těchto hodnot můžeme

⁴⁷Pro $d = 15$ je při $N_{\text{celkem}} = 10^6$ počet zásahů pouze řádově 10^1 , tj. z každých sto tisíc pokusů se jen jednou trefíme do jednotkové koule, zbytek pokusů padá do „rohů“ opsaných krychlí.

učinit pouze velmi hrubý odhad integrálu, a to jako plochu červeně vyšrafovaného obdélníku jako na panelu (a),

$$\int_a^b f(x)dx \approx (b-a)f(x_j). \quad (93)$$

Pokud budeme mít párů hodnot $(x_j, f(x_j))$ více, jak je znázorněno na panelu (b), vezmeme za odhad hodnoty integrálu průměr ploch takovýchto obdélníků,

$$\int_a^b f(x)dx \approx \frac{1}{N} \sum_{j=1}^N (b-a)f(x_j) = \frac{b-a}{N} \sum_{j=1}^N f(x_j). \quad (94)$$

V právě uvedeném postupu tkví je podstata integrace Monte-Carlo. Obecně platí, že pokud hodnoty x_j vybíráme z rozdělení s hustotou pravděpodobnosti $\rho(x)$, je integrál odhadnutý výrazem

$$\int_a^b f(x)dx \approx \frac{1}{N} \sum_{j=1}^N \frac{f(x_j)}{\rho(x_j)}, \quad (95)$$

přičemž nejvhodnější je volit takové pravděpodobnostní rozdělení, jehož hustota pravděpodobnosti co nejlépe kopíruje integrovanou funkci.⁴⁸ V praxi, jelikož na funkci nejčastěji pohlížíme jako na „černou skříňku“ a o jejím průběhu nic nevíme, se jako nevhodnější jeví volit rovnoměrné rozdělení s hustotou pravděpodobnosti (63), která po dosazení dá předchozí vzorec (94).

Chybu metody integrace lze odhadnout pomocí směrodatné odchylky

$$\begin{aligned} \Delta \equiv \sigma &= \sqrt{f^2 - \bar{f}^2}, \\ \bar{f}^2 &= \frac{1}{N} \sum_{j=1}^N f^2(x_j), \\ \bar{f}^2 &= \left[\frac{1}{N} \sum_{j=1}^N f(x_j) \right]^2. \end{aligned} \quad (96)$$

Úloha 9.3: Metodou Monte-Carlo spočítejte integrály

$$I_1 \equiv \int_0^{2\pi} e^{-x} \sin x \, dx, \quad (97)$$

$$I_2 \equiv \int_0^{\sqrt{10\pi}} \frac{\sin x^2}{\sqrt{1+x^4}} dx. \quad (98)$$

První integrál má analytické vyjádření, které si můžete odvodit a porovnat s hodnotou získanou Monte-Carlo integrací; druhý integrál lze spočítat pouze numericky. Metodu můžete otestovat i na jiných známých integrálech.

Řešení 9.3: Řešení je naprogramováno v souboru `integration.py` dvěma způsoby:

1. `integrate_1D` integruje postupným generováním $N = n$ náhodných bodů a výpočtem pomocí rovnice (94).
2. `integrate_1D_array` nagenereuje řadu N bodů z intervalu $\langle a, b \rangle$, pro tuto řadu spočítá řadu funkčních hodnot a tu pak sečte a vynásobí příslušným prefaktorem, rovněž dle rovnice (94). Tento postup je výrazně rychlejší, neboť obsluha cyklů a volání funkcí pro individuální body stojí v interpretovaných jazycích (jímž Python je) velké množství výpočetního času. Na druhou stranu tento postup vyžaduje mít v operační paměti uložená dlouhá pole bodů x_j a $f(x_j)$ a

⁴⁸Tento postup se nazývá *importance sampling*.

pro velké počty N nám dostupná paměť nemusí stačit. Řešením je samozřejmě oba postupy zkombinovat.

Výsledky integrálů jsou

$$I_1 = \frac{1}{2} (1 - e^{-2\pi}) \approx 0.499, \quad (99)$$

$$I_2 \approx 0.493. \quad (100)$$

Síla metody Monte-Carlo se naplno projeví při výpočtu vícerozměrných integrálů. Jak bylo ukázáno, chyba metody závisí jen na celkovému počtu pokusů $N = N_{\text{celkem}}$. Zatímco u jiných metod při požadování určité dané přesnosti výsledku drasticky narůstá časová složitost integrace s rostoucí dimenzí integrované funkce, u Monte-Carla časová složitost na dimenzi závisí jen nepatrně. Ve více rozměrech bývá navíc integrační oblast složitější, k čemuž lze využít metodu hit-and-miss. To se nejlépe ukáže na příkladu.

Úloha 9.4: Spočítejte čtyřrozměrný integrál

$$I_3 \equiv \int_{\Omega} \sin \sqrt{\ln(x+y+z+w+2)} \, dx \, dy \, dz \, dw, \quad (101)$$

kde integrační oblast je hyperkoule

$$\Omega : \left(x - \frac{1}{2}\right)^2 + \left(y - \frac{1}{2}\right)^2 + \left(z - \frac{1}{2}\right)^2 + \left(w - \frac{1}{2}\right)^2 \leq \frac{1}{4}. \quad (102)$$

Při výpočtu postupujte tak, že nejprve danou hyperkouli vepíšete do hyperkvádrů (či hyperkrychle) známých rozměrů, a tudíž známého objemu V . Poté pro náhodně zvolený bod v hyperkvádru určíte, zda se trefí do hyperkoule Ω či nikoliv. Pokud ano, spočítáte funkční hodnotu integrandu v tomto bodě. Jedná se tedy o kombinaci integrace (94) a metody hit-and-miss. Budete-li si uchovávat počet hodů N_{celkem} a počet zásahů $N_{\text{zásah}}$, získáte jako vedlejší produkt objem integrační hyperoblasti pomocí vzorce (88).

Řešení 9.4: Řešení pro tento konkrétní integrál je naprogramováno v souboru `integration.py` ve funkci `integral3`. Pro $N_{\text{celkem}} = 10^6$ tento integrál vychází přibližně

$$I_3 \approx 0.284. \quad (103)$$

Funkce `integral_ND` pak obsahuje obecnější řešení, které integruje libovolnou funkci v libovolněrozměrném prostoru na funkci `is_inside_region` zadané integrační oblasti.

10 Paralelizace

Rychlost výpočtu lze v zvyšovat dvěma základními způsoby:

1. Zvyšováním rychlosti procesoru.
2. Zvětšováním počtu procesorů.

Zatímco první způsob již narazil na fyzikální limity a kupředu postupuje jen zvolna,⁴⁹ druhý způsob lze použít takřka neomezeně. V dnešní době mají procesory osobních počítačů, ale i mobilních telefonů či dalších zařízení běžně dva až čtyři plnocenné podprocesory nazývané jádra, přičemž některé procesory navíc umožňují na každém jádře spustit dvě vlákna (technologie *hyperthreading*⁵⁰). To znamená, že můžeme na procesoru spustit několik výpočtů najednou a výpočty budou probíhat paralelně.

Jeden probíhající výpočet se běžně nazývá *vlákno* (thread) nebo *proces*.⁵¹ V moderních operačních systémech je počet současně běžících procesů neomezený. Pokud je procesů více, než dostupných procesorových jader, tak operační systém velmi rychle přepíná mezi prováděním procesu, čímž se zdá, že všechny procesy probíhají zároveň (tzv. *preemptivní multitasking*). Přepínání samozřejmě stojí určitý výpočetní čas. Optimální situace tedy je, pokud se počet procesů přesně rovná počtu procesorových jader.⁵² Pak jsou všechna jádra vytížena, a přitom operační systém není zatěžován nutností přepínat mezi jednotlivými vlákny. Toto je demonstrováno na obrázku 19.

Nejtriviálnější způsob paralelizace spočívá v tom, že spustíme nezávisle více programů najednou. Operační systém automaticky využije veškerá dostupná procesorová jádra. My jen počkáme na výsledky a ty pak zpracujeme. Tento způsob v podstatě vylučuje jakoukoliv komunikaci mezi jednotlivými vlákny, a úlohy proto musejí být zcela nezávislé. Z vnějšku lze také jen velmi omezeně řídit provádění vláken, například spustit nový výpočet po doběhnutí konkrétního vlákna.

Pokročilejší postup je ten, kdy danou úlohu rozsekáme na nezávislé samostatné úseky, ty pak z našeho programu (tzv. *hlavního vlákna*) pošleme ke zpracování na více procesorů, počkáme na výsledky a ty následně zpracujeme. Takto lze jednoduše paralelizovat metodu Monte-Carlo: chceme-li spočítat N Monte-Carlo bodů, spustíme výpočet N/p bodů současně v p vláknech, a poté všechny výpočty spojíme dohromady tak, že jednoduše spočítáme aritmetický průměr výsledků jednotlivých vláken. Vlákna se sebou nemusejí během výpočtu nijak komunikovat, nemusejí sdílet žádnou část paměti, není tudíž potřeba řešit jejich vzájemnou synchronizaci (v anglické terminologii se pro takovýto typ problémů používá označení „*embarrassingly parallel*“, trapně paralelní).⁵³

V úplně obecném případě je nutné řešit vzájemnou synchronizaci vláken, což přesahuje rámec tohoto kurzu. Pro ilustraci uvedu jen jeden z nejzákladnějších příkladů: Představte si, že dvě vlákna sdílejí některé proměnné, a tedy přistupují do stejné části paměti. Pak je nutné zabránit tomu, aby obě vlákna k jedné proměnné přistupovala zároveň (například jedno z proměnné četlo, zatímco

⁴⁹Maximální dosažitelná rychlost procesorů je dnes přibližně 5 GHz, tj. řádově miliardy strojových cyklů za vteřinu, a během posledních let se nemění. Strojová instrukce většinou trvá několik cyklů a jejich provádění je navíc zpomalováno přístupem programu do operační paměti, proto dnešní procesory zvládnou řádově nanejvýš stovky milionů instrukcí za vteřinu. Jeden jednoduchý příkaz v Pythonu může vyžadovat stovky až tisíce procesorových instrukcí, takže Python vykoná řádově maximálně stovky tisíc příkazů za vteřinu. Ke zrychlení procesoru se využívají nejrůznější sofistikované metody. Procesor například odhaduje, kam se program vydá, a instrukce se snaží předpočítávat dopředu. Pokud se v odhadu trefí, dojde ke zrychlení. Jiný způsob je vytváření nových strojových instrukcí, které zrychlují určitý často používaný typ úloh (například instrukce rychlá Fourierova transformace, která se intenzivně používá při dekodování videa a při práci s obrázky, speciální instrukce pro šifrování nebo instrukce pro speciální matematické operace).

⁵⁰Technologie spočívá v tom, že v každém procesorovém jádře jsou různé výpočetní jednotky, které zpracovávají různé typy procesorových instrukcí. Zatímco se tedy v jednom vlákne násobí dvě čísla s desetinnou čárkou, v jiném vlákne na tomtéž jádře se může zpracovávat například cyklus přes celočíselný index.

⁵¹Ve skutečnosti je to složitější, jeden proces může obsahovat i více vláken, ale v těchto zápiscích budu předpokládat, že v každém procesu běží jen jedno vlákno, a tudíž budu obě označení brát jako synonyma.

⁵²Počet logických jader se v Pythonu zjistí pomocí volání funkce `os.cpu_count()` nebo `multiprocessing.cpu_count()`.

⁵³Ve striktně funkcionálním programování není dovoleno funkcím měnit hodnoty proměnných, a proto lze čisté funkcionální programy triviálně paralelizovat.

druhé do ní zapisovalo). To by totiž vedlo k nejednoznačnému výsledku, protože nelze a priori říci, které vlákno bude operaci provádět dříve. K vyřešení kolize se používají tzv. *zámky* (lock). Než vlákno přistoupí ke sdílené proměnné, zamkne si ji pro sebe, provede svoji operaci a poté proměnnou odemkne. Pokud by mezitím k zamčené proměnné chtělo přistoupit jiné vlákno, jeho provádění se zastaví a vlákno čeká, než bude proměnná odemčena. Z toho ihned vyplývá, že při velkém počtu sdílených proměnných či při častém přístupu k nim dochází k významnému zpomalení paralelního zpracování, neboť velkou část výpočetního času vlákna čekají na přístup k momentálně zamčeným proměnným.

Zamykání proměnných s sebou nese problémy a potenciální obtížně odhalitelné chyby. Může se stát, že zapomeneme proměnnou odemknout, čímž zastavíme všechna ostatní vlákna, která chtějí k proměnné přistupovat. K zablokování programu může dojít i tak, že provádění vlákna skončí chybou ve chvíli, kdy má pro sebe nějakou proměnná zamčenou.

Jiný možný způsob zablokování paralelního výpočtu kvůli zámkům je tzv. *gridlock*: Jedno vlákno chce například sečíst hodnotu proměnných **a** a **b**. Zamkne proměnnou **a** a chce zamknout i proměnnou **b**, avšak tu má zrovna zamčenou druhé vlákno. Pokud v tu chvíli druhé vlákno potřebuje přistoupit k proměnné **a**, není mu to povoleno, protože tato proměnná je zamčena prvním vláknem. První vlákno tedy čeká na odemčení proměnné **b** aby mohlo odemknout proměnnou **a**, zatímco to druhé na odemčení proměnné **a**, bez čehož neuvolní proměnnou **b**. Obě vlákna jsou do sebe zakleslá, čekají a k uvolnění nedojde nikdy.

Pro pěkný podrobný úvod do paralelního programování doporučuji <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>.

V Pythonu existují dvě základní knihovny pro zpracování programu ve více vláknech: **threading** a **multiprocessing**. První z nich obsahuje více funkcionalit, avšak spouští všechna vlákna jen na jednom procesoru (jádre), k paralelnímu provádění výpočtů se tedy nehodí.⁵⁴

Paralelizace na více výpočetních jádrech je v Pythonu implementována v knihovně **multiprocessing**. Zde si ukážeme využití objektů **Pool**, **Process** a **Value** z této knihovny na příkladu paralelní integrace metodou Monte-Carlo. Vzorový kód je naprogramován v souboru **parallelization.py**.

- **integrate_1D_Pool**: Nejjednodušší způsob paralelizace je pomocí objektu **Pool**. Při vytváření instance tohoto objektu mu předáme parametr **processes** udávající maximální počet procesů, které bude objekt obsluhovat.⁵⁵ Následně zavoláme jeho metodu **starmap**, jejíž první parametr je funkce, kterou chceme spustit v jednotlivých procesech, a druhý parametr je iterovatelná, jehož každý element obsahuje *n*-tici s argumenty naší funkce. Metoda **starmap** spustí postupně naši funkci se všemi dostupnými *n*-ticemi parametrů z iterovatelné, přičemž použije všechny dostupné procesy objektu **Pool**, počká na výsledky z jednotlivých vláken a shromáždí je do seznamu, který přiřazujeme proměnné **results**.⁵⁶ Průměr ze všech hodnot dá finální hodnotu integrace Monte-Carlo.

Pokud by naše funkce měla jen jeden argument, namísto metody **starmap** bychom použili jednodušší metodu **map**. Ve vzorovém příkladu však spouštěné funkci **integrate_1D** musíme předat argumenty čtyři, proto volíme složitější **starmap**.⁵⁷

Při vytvoření instance objektu **Pool** jsme použili konstrukci s klíčovým slovem **with**.

- **integrate_1D_process**: Složitější, avšak univerzálnější je použití objektů **Process**.⁵⁸ Ty vyžadují vykonání tří kroků:

⁵⁴Použití knihovny **threading** je vhodné v případech, kdy provádíme úlohy, ve kterých se obvykle čeká na výsledek. Chceme například stáhnout webovou stránku z nějakého serveru, což může trvat nedefinovaně dlouho, a nechceme přitom, aby náš program přestal po dobu čekání reagovat.

⁵⁵Pokud parametr **processes** nezádáme, Python použije všechna dostupná logická jádra procesoru. Jejich počet lze získat voláním funkce **multiprocessing.cpu_count()**.

⁵⁶Pokud je *n*-tic s argumenty víc než počet dostupných procesů, bude objekt **Pool** spouštět volání postupně.

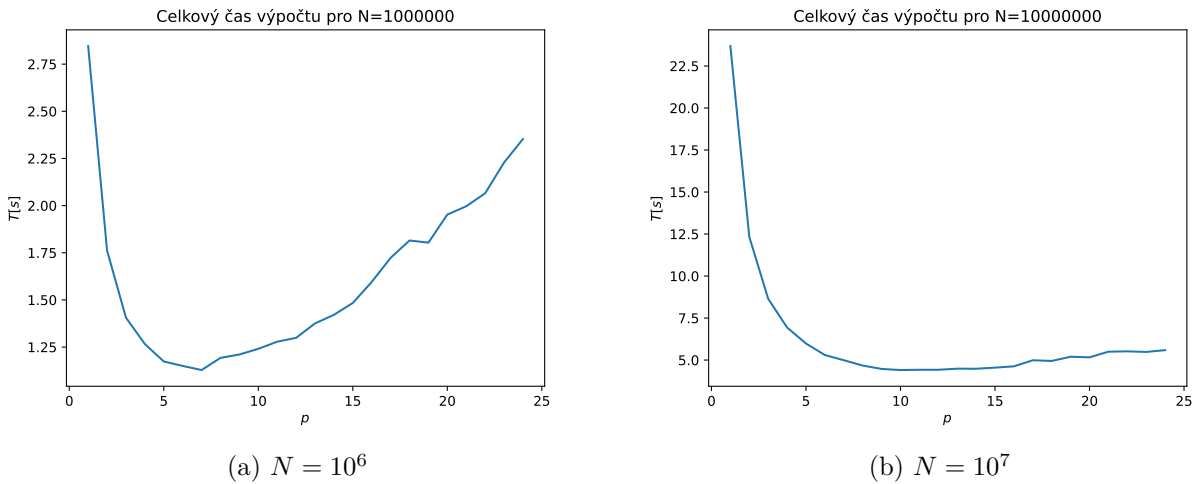
⁵⁷Kromě funkcí **starmap** a **map** existují ještě **starmap_async** a **map_async**, které spouštějí výpočet asynchronně. To zjednodušeně řečeno znamená, že hlavní vlákno programu.

⁵⁸Podobným způsobem se používá knihovna **pthread** v programovacím jazyce C++.

1. Vytvoříme instanci objektu typu `Process`. Při tom musíme specifikovat parametr `target`, jímž předáme funkci, kterou chceme v procesu spustit, a parametr `args`, který obsahuje její argumenty.
2. Výpočet v procesu spustíme pomocí metody `start`. Výpočet se spustí asynchronně, naše hlavní vlákno programu tedy nečeká, než se výpočet v novém procesu dokončí.
3. Chceme-li počkat na výsledek, využijeme metodu `join`. Zavoláme-li ji pro daný proces, bude hlavní vlákno programu čekat na dokončení příslušného procesu. My chceme počkat na dokončení všech procesů, proto si musíme po vytvoření a nastartování procesy uschovat (ve vzorovém kódu je uschováváme v proměnné `processes`) a poté zavolat `join` pro všechny.

Při použití objektu `Process` není možné získat návratovou hodnotu volané funkce. Proto musíme program ještě trochu zesložitit a volanou funkci vrátit ve sdílené proměnné — instanci objektu `Value`. Tento objekt slouží k přenosu hodnot mezi hlavním vláknem a ostatními procesy i mezi procesy navzájem. První parametr při vytváření instance objektu `Value` udává typ (`'d'` pro číslo s desetinnou čárkou, `'i'` pro celočíselný typ), druhý parametr počáteční hodnotu. Hodnota proměnné je uložena v atributu `value`. Vytvoříme takovouto proměnnou pro každý jednotlivý proces. Navíc musíme naprogramovat pomocnou funkci (*wrapper*), kterou pojmenujeme `integrate_1D_parallel` a která nám zavolá naši funkci `integrate_1D` a její návratovou hodnotu uloží do sdílené proměnné. Naši funkci `integrate_1D` chceme předat beze změny všechny parametry, které pomocná funkce `integrate_1D_parallel` dostane. K tomu slouží konstrukce `*args`, `**kwargs`.

Objekt `Value` má v sobě implementovaný zámek. Stačilo by tedy použít jen jednu instanci `Value` pro všechny procesy a výsledky dílčích integrací do ní přičítat. Průměr bychom nakonec získali vydělením počtem procesů.

(a) $N = 10^6$ (b) $N = 10^7$

Obrázek 19: Výpočetní čas integrace metodou Monte-Carlo při použití různého počtu paralelních procesů p a dvou hodnot N . V každém z p vláken je spuštěn výpočet integrálu I_1 (97). Výpočet byl prováděn na PC s 8-jádrovým procesorem AMD Ryzen 7 se zapnutou technologií hyperthreading. Pozorujeme, že celková doba výpočtu T je minimální pro $p \approx 8$, kdy jsou naplna vytížena všechna fyzická jádra. Pro $8 \leq p \leq 16$ je celkový čas výpočtu velmi plochý, a to zejména pro vyšší hodnotu N . Procesy se již musejí dělit o dostupná jádra, avšak hyperthreading úspěšně kompenzuje dodatečné výpočetní požadavky související s tvorbou nových vláken a jejich synchronizací po dokončení výpočtů. Pro $p > 16$ je již vláken více, než kolik je logických procesorů, a procesor je plně saturován. S rostoucím p pak dochází ke zpomalování výpočtu, protože si pro sebe více a více výpočetního času bere režie operačního systému a interpretu jazyka Python, aby všechna vlákna obhospodařila.

Důležité poznámky:

- Python postupuje tak, že v jednotlivých procesech spouští celé moduly obsahující spouštěnou funkci. V našem případě je v každém procesu spuštěn celý modul `parallelization.py`. Tento modul obsahuje globální část kódu, která v našem případě spouští funkce, které spouštějí víceprocesorové zpracování, a došlo by tedy k zacyklení programu. Abychom tomuto zabránili, je nutné tu část kódu, která smí být spuštěna jen z hlavního vlákna, vložit do podmínky `if __name__ == "__main__":`
- Ve Windows ve vývojovém prostředí IDLE nefunguje funkce `print`, pokud ji používáme z jiného než z hlavního vlákna (nic nevypíše). V jiných prostředích či operačních systémech by to mělo fungovat.

Efekty paralelizace lze vidět pomocí funkce `plot_integrate_1D_duration`. Tato funkce spouští postupně paralelní výpočet pro různé počty navzájem běžících paralelních vláken a výsledky zobrazí do grafů, které jsou vykresleny pro dvě hodnoty N v obrázku 19. Pozorujeme, že nejrychlejšího času výpočtu pro dané N dosáhneme, pokud vytížíme všechna dostupná fyzická jádra.

Úloha 10.1: *Prostudujte si vzorový příklad v souboru `parallelization.py` a na jeho základě vytvořte kód, který bude počítat ve více vláknech hodnotu integrálu I_3 (101) z úkolu 9.4. Zjistěte si, kolik výpočetních jader má procesor na vašem počítači, a spusťte výpočet tak, aby zaměstnal všechna jádra.*

Úloha 10.2: *Časově náročný, a přitom jednoduše paralelizovatelný je výpočet součinu dvou matic. Jelikož metoda `starmap` pracuje jednoduše jen s vektory, naprogramujte paralelní výpočet součinu matice a vektoru (výsledkem je vektor).*

11 LaTeX

Nedílnou součástí vědecké práce je prezentování výsledků, přičemž chceme, aby dokument bylo jednoduché napsat, a zároveň aby dobře vypadal. Jelikož fyzikální odborné texty obsahují velké množství rovnic a symbolů, není nejvhodnější volbou používat textové editory pro kancelářskou práci, jejichž možnosti psaní rovnic a použití vědeckých stylů jsou celkem omezené. Ve fyzice je nejběžnější psát texty v systému $\text{\LaTeX}$.

Dnes se jedná o nesmírně obsáhlý balík různých knihoven a doplňků, pomocí kterého lze

- psát dobře vypadající a typograficky správné odborné texty s matematickými rovnicemi, obrázky a tabulkami,
- vybírat z množství profesionálně připravených stylů obsahujících podporu tvorby obsahu, rejstříku, poznámek pod čarou, kapitol, seznamů, referencí a dalších vychytávek,
- stáhnout si přímo styl časopisu či knihy, pro které text připravujeme,
- používat tisíce matematických symbolů či druhů písem (lze psát třeba ve švabachu, gotickém písmu či elfím písmu, jste-li fanoušci díla J.R.R. Tolkiena),
- využívat různá makra a doplňky k jednoduchému vytváření speciálních objektů (například chemických vzorců nebo Feynmannových diagramů),
- vytvářet robustní prezentace (například pomocí knihovny Beamer),
- nebo sázet notové materiály a kdovíco dalšího (rajčata však zatím ne).

Systém $\text{\LaTeX}$ má dvě vrstvy:

1. $\text{\TeX}$: sazeč (zajišťuje, aby vše bylo na stránce tam, kde má být),
2. $\text{\LaTeX}$: typograf (zajišťuje, aby dokument dobře vypadal).

Program $\text{\TeX}$ začal vznikat v 70. letech 20. století a byl určen sázení textu a matematických rovnic při zachování vysoké typografické úrovně výsledného dokumentu. $\text{\LaTeX}$ je pak nadstavba maker, která psaní dokumentů velmi zjednodušuje a zcela zakrývá sazečskou práci. Příkazy v $\text{\TeX}$ u jsou však stále dostupné.

Dokument napsaný v $\text{\LaTeX}$ u striktně odděluje *text* (obsah) a *styl* (vzhled), přičemž my využijeme standardní $\text{\LaTeX}$ ovský styl a budeme se zabývat výhradně tím, jak napsat text. Jeho psaní se podobá programování: zdrojový soubor je textový dokument (nebo několik textových dokumentů), který sestává z psaného textu a doplňujících příkazů. Chceme-li vidět, jak bude výsledný dokument vypadat, musíme soubor „přeložit“.

Existují různé *distribuce* $\text{\LaTeX}$ u. Jedna z nejpoužívanějších je [MiKTeX](#) (všechny platformy), nebo [TeX Live](#) (Linux, Windows) nebo [MaxTeX](#) pro operační systém Max OS. Dále je nutné mít k dispozici textový editor, přičemž vhodný je ten, který rozumí $\text{\TeX}$ ovským příkazům a bude umět zdrojový text poslat k překladu a zobrazit. Pokročilé editory umějí navigovat mezi zdrojovým textem k přeloženému výsledku (většinou do PDF souboru) a naopak. Z volně dostupných editorů jsou nejčastěji používány:

- **TeXworks**: Editor se základními funkcemi. Je součástí distribuce MiKTeX.
- [TeXnicCenter](#): Pokročilejší editor, podpora použití projektů.
- [TeXstudio](#): Pokročilý editor, podpora projektů, automatického doplňování, jednoduchá práce s obrázky, možnost tvorby záložek, pomocníci po tvorbu rovnic a tabulek, zvýrazňování syntaktických chyb.

- **Visual Studio Code** s pluginem **LaTeX Workshop**: Méně funkcí co se týče samotného $\text{\LaTeX}$ u a komplikovanější instalace, avšak plná podpora všech funkcí tohoto rozšířeného vývojového prostředí (integrace s Gitem, pokročilé vyhledávání, programátorské možnosti editace). Pro správnou funkci je nutné kromě uvedeného doplňku nainstalovat ještě interpret jazyka **PERL**. Ve VS Code vznikají i tyto zápisky.
- **Overleaf**: Online editor, ideální pro práci v týmu. Nemusíte nic instalovat, potřebujete však připojení k internetu. Pro větší projekty je nutná placená verze.

Úloha 11.1: *Nainstalujte si na svůj počítač nějakou distribuci $\text{\LaTeX}$ u a editor.*

11.1 Formát $\text{\TeX}$ ovského souboru

- ASCII nebo UTF-8 kódování (lze použít i obskurnější kódování, ale v dnešní době je UTF-8 dostatečně univerzální).
- *Mezery*: více mezer je interpretováno jako jedna mezera, jednoduché zalomení řádku také jako jedna mezera.
- *Nový odstavec*: dvě zalomení řádku po sobě nebo příkaz `\par`.
- *Příkazy*: uvozeny znakem `\`, jejich povinné parametry ve složených závorkách `{...}`, volitelné parametry v hranatých závorkách `[...]`. U názvů příkazů záleží na velikosti písmen.
- *Rovnice v textu*: oddělena znaky `$...$`. Některé příkazy lze použít pouze uvnitř rovnic, jiné naopak uvnitř rovnic použít nelze.
- *Komentář*: uvozen znakem `%`. Vše za tímto znakem až do konce řádky je ignorováno.

11.2 Struktura $\text{\TeX}$ ovského souboru

11.2.1 Hlavička

Jedná se o první řádek souboru obsahující příkaz

- `\documentclass[a4paper,twoside,11pt,twocolumn]{article}`: Základní specifikace stylu dokumentu (v tomto případě papír velikosti A4, dvoustránkový tisk, základní velikost písma 11 bodů, dvousloupcová sazba). Kromě `article` se často použije `slide` pro tvorbu prezentací. Příkaz `\documentclass` je povinný, musí být vždy přítomen.

11.2.2 Preambule

Část zdrojového kódu obsahující výčet všech použitých dodatečných balíčků. Balíčky jednak obsahují styly, jednak rozšiřují funkce $\text{\LaTeX}$ u o nová makra. Dále se v preambuli udávají parametry, které se použijí pro styl textu, doplňující parametry balíčků a uživatelské definice maker.

- `\usepackage{epsfig}`: Použije se balíček `epsfig` pro vkládání vektorových obrázků ve formátu EPS.
- `\def\abs#1{\left|#1\right|}`: Definuje makro `\abs` s jedním parametrem (absolutní hodnota).
- `\pagestyle{headings}`: Na každé stránce bude záhlaví s názvem aktuální kapitoly, sekce a s číslem stránky.

Další nejčastěji používané balíčky jsou:

- `\usepackage{amsfonts,amsmath,amssymb}`:⁵⁹ Rozšiřuje množství použitelných písem, matematických symbolů a struktur (např. snazší psaní matic, víceřádkových rovnic atd.).
- `\usepackage[utf8]{inputenc}`: Specifikuje UTF-8 jako vstupní kódování (jinak je očekáváno ASCII).
- `\usepackage[czech]{babel}`: Udávájí české formátování (například české uvozovky) a české názvy (například Obsah, Rejstřík).
- `\usepackage{epsfig}`: Umožní vkládání vektorových EPS souborů.
- `\usepackage[unicode]{hyperref}`: Umožní vkládání hypertextových odkazů a učiní klikací i odkazy na rovnice, obrázky, stránky či kapitoly v textu pro snazší navigaci.

11.2.3 Tělo dokumentu

Tělo je uvozeno příkazy

```
\begin{document}
...
\end{document}
```

a do něj píšeme vlastní text dokumentu. Vše, co se nachází za příkazem `\end{document}`, je ignorováno.

Jednoduchý příklad L^AT_EXovského dokumentu s vysvětlením základů psaní textu je v repozitáři v adresáři LaTeX a jmenuje se `dokument.tex`. K jeho přeložení budete potřebovat i přiložený EPS soubor `kubik.eps`.

Pokročilejším příkladem je přímo tento soubor s poznámkami (i jeho zdroják `Poznamky.tex` je dostupný v repozitáři).

Úloha 11.2: *Prostudujte si vzorový L^AT_EXovský soubor `dokument.tex` a napište v L^AT_EXu pojednání o nějakém svém oblíbeném vědci či vědkyni nebo o fyzikálním (případně matematickém) problému. Dokument by měl obsahovat aspoň jednu rovnici a jeden obrázek. Textová část stačí na jednu stránku.*

11.3 Další návody a odkazy

- [Ne příliš stručný úvod do systému L^AT_EX2_ε](#): Vynikající srozumitelný, přehledný a čtivý návod v češtině. Doporučuji prostudovat.
- [Drsný úvod do L^AT_EXu](#): Trochu stručnější a zhuštěnější než předchozí návod.
- [Jak na LaTeX](#): Webový seriál, obsahuje i určité pokročilejší vychytávky.
- [The comprehensive L^AT_EX Symbol List](#): Několikasetstránkový dokument se všemi možnými použitelnými matematickými i nematematickými symboly.
- [L^AT_EX](#): Asi nejpodrobnější příručka dostupná na webu.

⁵⁹AMS = American Mathematical Society.

12 Symbolické výpočty

Symbolické výpočty znamenají výpočty s celými algebraickými výrazy: jejich upravování, integrování, derivování a podobně.⁶⁰ V Pythonu sice existuje knihovna `sympy`⁶¹ pro symbolické manipulace, avšak pro rozsáhlejší výpočty jsou optimalizovanější a jednodušší k použití komerční produkty `Mathematica` či `Maple`. My se v tomto cvičení seznámíme s nejjednoduššími základy práce v produktu `Mathematica`.⁶²

12.1 Mathematica

Program Wolfram Mathematica má za sebou již více než 30 let vývoje. Lze ho použít jako rozšířenou kalkulačku pro jednoduché výpočty a vykreslování grafů, ale jeho skutečná síla je v efektivním skloubení možnosti práce se symbolickými výrazy spolu s numerickými výpočty v libovolné přesnosti. Mathematica obsahuje nepřeberné množství funkcí, které zahrnují velkou část klasické i moderní matematiky (řešení algebraických i diferenciálních rovnic, teorie čísel, neuronové sítě, strojové učení, statistické nástroje, pokročilé metody vizualizace, interaktivní grafy, analýza časových řad a další). Umožňuje psát vlastní rozsáhlé programy v optimalizovaném (i když co se syntaxe týče trochu neobvyklém) programovacím jazyku. Všechny výpočty lze spouštět buď lokálně, nebo na velkých výpočetních clusterech.

Mathematica je sice placený produkt, avšak MFF UK má zakoupenou hromadnou licenci, kterou může používat každý náš student nebo zaměstnanec ([návod na instalaci](#)). Existuje verze zdarma pro počítače Raspberry Pi s operačním systémem Raspbian.

V [repozitáři](#) naleznete soubor `mathematica.nb`, který obsahuje jednoduchý úvod se základními funkcemi programu Mathematica. Projděte si ho příkaz po příkazu, pomůže vám naučit se syntaxi a zároveň vás seznámí s nejdůležitějšími a nejpoužívanějšími funkcemi a příkazy. Pro začátek je důležité vědět, že příkazy se spustí klávesovou kombinací **CTRL + Enter** (je to stejné jako v pythonovském programovacím prostředí Jupyter). Na příkladu tohoto vzorového souboru vypracujte následující úlohy.

Úloha 12.1: *Spočítejte limitu*

$$\lim_{x \rightarrow a} \left(\frac{\sin x}{\sin a} \right)^{\frac{1}{x-a}}. \quad (104)$$

Úloha 12.2: *Nalezněte primitivní funkci*

$$F(x) = \int \sqrt{1+x^4} \, dx \quad (105)$$

a vykreslete její graf pro $x \in \langle 0, 2 \rangle$.

Úloha 12.3: *Vyřešte diferenciální rovnici pro matematické kyvadlo s velkou výchylkou*

$$y''(t) = -\sin y(t) \quad (106)$$

s počátečními podmínkami

$$y(0) = 0, \quad y'(0) = 1 \quad (107)$$

a srovnajte v grafu s řešením linearizované rovnice

$$y''(t) = -y(t). \quad (108)$$

Řešení 12.1: *Řešení všech tří úloh je v souboru `solution.nb`.*

⁶⁰Programy pro symbolické výpočty se také nazývají *Computer Algebra Systems*, CAS.

⁶¹Tuto knihovnu začal vyvíjet v roce 2005 tehdy ještě student fyziky MFF UK Ondřej Čertík.

⁶²Pěkné srovnání funkcionalit a syntaxe knihovny `sympy` a programu Mathematica najdete na <https://github.com/sympy/sympy/wiki/SymPy-vs.-Mathematica>

Pokud se chcete s možnostmi programu Mathematica seznámit hlouběji a dozvědět se více o pokročilých možnostech programování, které nabízí, můžete navštěvovat dedikovanou přednášku Dr. Tomáše Ledvinky [Použití systémů počítačové algebry ve fyzice](#).

13 Fourierova transformace

Fourierova transformace je jednou z nejčastěji používaných metod ke zpracování signálu. Spočívá v převodu časově závislé funkce $h(t)$ na funkci spektra harmonických frekvencí $H(\omega)$. S Fourierovou transformací analytických funkcí se seznámíte v matematické analýze a v teorii distribucí. My se zde budeme zabývat diskrétní verzí této transformace, která se používá pro numerické zpracování naměřených signálů.

13.1 Diskrétní Fourierova transformace

Transformace a případná zpětná transformace $h_j \leftrightarrow H_k$ se provádějí předpisem

$$\begin{aligned} H_k &= \frac{1}{N} \sum_{j=0}^{N-1} h_j e^{-\frac{2\pi i j k}{N}}, \\ h_j &= \sum_{k=0}^{N-1} H_k e^{+\frac{2\pi i j k}{N}}, \end{aligned} \quad (109)$$

kde

- $h_j, j = 0, 1, \dots, N-1$ je *vstupní signál (časová řada)* délky N naměřený v ekvidistantních okamžicích odpovídajících časům

$$t_j = \frac{j}{f_s}. \quad (110)$$

- f_s je *vzorkovací frekvence* a odpovídá počtu měření za sekundu.
- $H_k, k = -\frac{N-1}{2}, \dots, 0, \frac{N-1}{2}$ (pro N liché) je spektrální rozklad vstupního signálu, kde

$$f_k = |k| \frac{f_s}{N-1} \quad (111)$$

je frekvence odpovídajícího příspěvku. Níže ukážeme, že k jedné frekvenci existují dvě komponenty H_k a H_{-k} , které se liší jen fází. Jelikož díky periodičnosti platí $H_k = H_{k+N}$, je při programování praktičtější uvažovat index $k = 0, \dots, N-1$. Pak H_k a H_{N-k} odpovídají stejné frekvenci.

- Nejvyšší frekvence, kterou jsme schopni ze vstupního signálu určit, nezávisí na délce signálu N a je určena pouze vzorkovací frekvencí: $f_{\max} = f_s/2$. Nazývá se *Nyquistova frekvence*.
- Nejnižší frekvence, kterou ze signálu vyextrahujeme, je $f_{\min} = \frac{f_s}{N-1}$. Pro signály velmi nízkých frekvencí potřebujeme tedy dlouhé vstupní časové řady.
- Frekvence f_0 odpovídá tzv. *stejnoseměrnému příspěvku* a odpovídající komponenta H_0 udává střední hodnotu signálu.

Přímá a zpětná transformace se liší jen znaménkem a normalizačním faktorem $1/N$, jehož umístění do zpětné Fourierovy transformace je věcí konvence.⁶³

Úloha 13.1: *Dokažte, že provedením Fourierovy transformace a zpětné Fourierovy transformace dostanete původní časovou řadu.*

⁶³Jiná často používaná konvence je tento faktor rozdělit symetricky mezi obě transformace: pak bude před oběma sumami ve vztazích (109) stát $\frac{1}{\sqrt{N}}$ a Fourierova transformace bude unitární transformací mezi vektory $\mathbf{h} = (h_j)$ a $\mathbf{H} = (H_k)$ (unitární transformace zachovává délku komplexního vektoru).

Řešení 13.1: Důkaz se provede přímočarým dosazením H_k do zpětné Fourierovy transformace:

$$\begin{aligned}
 \sum_{k=0}^{N-1} H_k e^{\frac{2\pi i j k}{N}} &= \sum_{k=0}^{N-1} \left[\frac{1}{N} \sum_{l=0}^{N-1} h_l e^{-\frac{2\pi i l k}{N}} \right] H_k e^{\frac{2\pi i j k}{N}} \\
 &= \frac{1}{N} \sum_{kl} h_l e^{\frac{2\pi i k}{N}(j-l)} = \frac{1}{N} \sum_{kl} h_l \left[e^{\frac{2\pi i}{N}(j-l)} \right]^k \quad \text{součet geometrické řady} \\
 &= \frac{1}{N} \sum_l h_l \frac{1 - \left[e^{\frac{2\pi i}{N}(j-l)} \right]^N}{1 - e^{\frac{2\pi i}{N}(j-l)}} = \frac{1}{N} \sum_l h_l \frac{1 - e^{2\pi i(j-l)}}{1 - e^{\frac{2\pi i}{N}(j-l)}} \\
 &= \frac{1}{N} \sum_l h_l N \delta_{jl} = h_j \quad (112)
 \end{aligned}$$

(δ_{jl} je Kroneckerovo delta), přičemž při úpravě na posledním řádku se využilo. toho, že

$$\begin{aligned}
 e^{2\pi i(j-l)} &= 1 \quad \text{a zároveň} \quad e^{\frac{2\pi i}{N}(j-l)} \neq 1 \quad \text{pro } j \neq l \\
 \lim_{x \rightarrow 0} \frac{1 - e^{2\pi i x}}{1 - e^{\frac{2\pi i}{N} x}} &= N \quad \text{pro } j = l. \quad (113)
 \end{aligned}$$

Z předpisu (109) vyplývá, že výsledek Fourierovy transformace je obecně sekvence *komplexních* čísel. Rozepsáním komplexní exponenciály dostaneme

$$\begin{aligned}
 H_k^R &= \frac{1}{N} \sum_{j=0}^{N-1} \left(h_j^R \cos \frac{2\pi j k}{N} + h_j^I \sin \frac{2\pi j k}{N} \right), \\
 H_k^I &= \frac{i}{N} \sum_{j=0}^{N-1} \left(-h_j^R \sin \frac{2\pi j k}{N} + h_j^I \cos \frac{2\pi j k}{N} \right), \quad (114)
 \end{aligned}$$

kde

$$\begin{aligned}
 h_j &= h_j^R + i h_j^I, \\
 H_k &= H_k^R + i H_k^I \quad (115)
 \end{aligned}$$

je rozklad na reálnou a imaginární část. Analogický vztah bychom získali pro zpětnou Fourierovu transformaci.

Za předpokladu, že vstupní časová řada h_j je reálná (obecně reálná být nemusí, ale v praxi obvykle bývá), můžeme pro reálnou a imaginární složku spektrálního rozkladu H_k psát

$$H_k = \frac{1}{N} \sum_{j=0}^{N-1} h_j \cos \frac{2\pi j k}{N} - \frac{i}{N} \sum_{j=0}^{N-1} h_j \sin \frac{2\pi j k}{N}. \quad (116)$$

Ze sudosti cosinu a lichosti sinu plyne, že $H_k^R = H_{-k}^R$ a $H_k^I = -H_{-k}^I$.

Komponenty H_k lze také rozepsat pomocí jiných dvou reálných čísel: velikosti $|H_k|$ a fáze ϕ_k ,

$$H_k = |H_k| e^{i\phi_k}, \quad (117)$$

$$\begin{aligned}
 |H_k| &= \sqrt{(H_k^R)^2 + (H_k^I)^2}, \\
 \phi_k &= \arctan \frac{H_k^I}{H_k^R}. \quad (118)
 \end{aligned}$$

V praxi je nejdůležitější údaj velikost příspěvku k -té frekvence

$$A_k = |H_k| + |H_{-k}| = 2|H_k|, \quad k = 1, \dots, \frac{N-1}{2}, \quad (119)$$

který udává amplitudu odpovídající harmonické vlny. Místo amplitudy se také často používá kvadrát

$$S_k \equiv |A_k|^2, \quad (120)$$

který se nazývá *výkonové spektrum* a udává, jak již název napovídá, výkon, který do signálu vnáší k -tá harmonická vlna.

Fourierova transformace tedy vyjadřuje signál jako součet prostých harmonických vln (sinů a cosinů) s frekvencemi f_k a amplitudami A_k . Hudební terminologií se jedná o rozklad souzvuku několika tónů na jednotlivé jednoduché tóny.

13.2 Použití Fourierovy transformace

- Získání dominantních frekvencí neznámého signálu (například délky slunečního cyklu, vlastních frekvencí kmitů složitěho objektu atd.).
- Rozpoznávání hlasu.
- Filtrování signálu, odstranění šumu (signál očekáváme na určitých frekvencích, šumu se zbavíme, pokud komponenty H_k od ostatních frekvencí vynulujeme).
- Komprese signálu: uchováme jen složky signálu s několika největšími příspěvky H_k . Toho se využívá v kompresi zvukového signálu (například MP3) i obrazového signálu (formát JPEG).

My si vyzkoušíme Fourierovu transformaci na zvukovém souboru. Práce se zvukovými soubory v Pythonu je popsána v sekci 3.2.7

Úloha 13.2: *Naprogramujte Fourierovu transformaci a zpětnou Fourierovu transformaci podle vztahů (109). Můžete počítat buď nezávisle reálnou a imaginární část, nebo využít toho, že funkce knihovny `numpy` rozumějí komplexním číslům. Imaginární jednotka v Pythonu je `1j`, komplexní číslo $3 + 4i$ se tedy zapíše ve tvaru `3 + 4j`.*

Řešení 13.2: *Vzorové řešení je v souboru `transform.py` a umí spočítat Fourierovu transformaci pro reálnou i komplexní časovou řadu. Jelikož samotné provádění cyklů v Pythonu zabere obrovské množství výpočetního času, je výhodné se cykly vyhnout. Jakékoliv zrychlení oceníme zejména v případech, kdy budeme chtít počítat transformaci z delší časové řady (již pro $N = 1000$ může výpočet trvat několik sekund).*

Úloha 13.3: *Vytvořte časovou řadu délky $N = 2000$ se vzorkovací frekvencí $f_s = 2000$ Hz (signál tedy bude trvat přesně 1 s) danou součtem tří harmonických funkcí:*

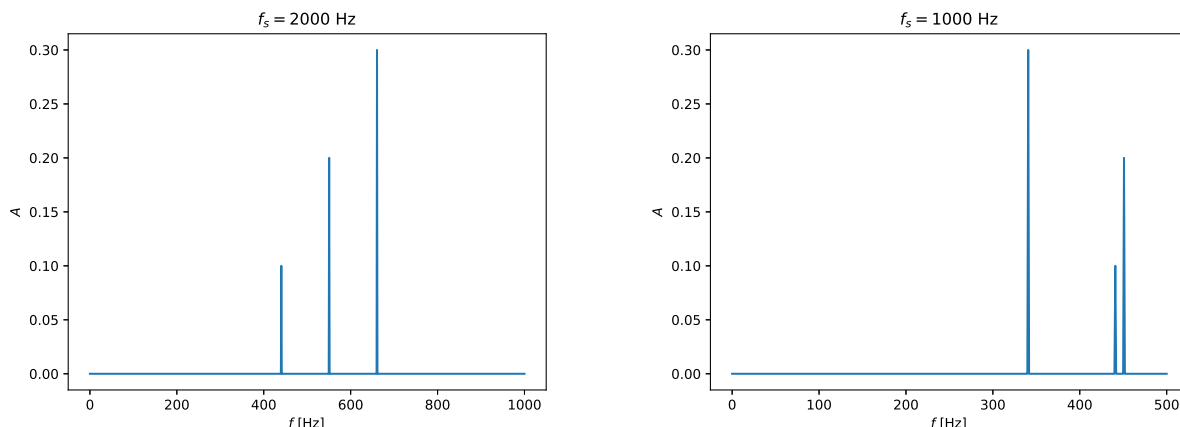
$$h_j = \sum_{n=1}^3 a_n \sin(2\pi F_n t_j), \quad (121)$$

kde

$$\begin{array}{lll} a_1 = 0.1, & a_2 = 0.2 & a_3 = 0.3, \\ F_1 = 440 \text{ Hz} & F_2 = \frac{5}{4}F_1 = 550 \text{ Hz} & F_3 = \frac{3}{2}F_1 = 660 \text{ Hz} \end{array}$$

a časy t_j jsou dány vztahem (110). Přehrajte si ji pomocí funkcí knihovny `sounddevice`.⁶⁴ Spočítejte Fourierovu transformaci a vykreslete do grafu amplitudy A_k na frekvencích f_k . Přesvědčte se, že graf bude mít tři vrcholy s výškami odpovídajícími zadaným amplitudám a_n a středy v místech zadaných frekvencí F_n .

Zopakujte výpočet pro $N = 1000$ a $f_s = 1000$ Hz. V tomto případě jsou již frekvence F_2, F_3 větší než Nyquistova frekvence $f_{\max}$. Zvuk bude zkreslený a vrcholy ve frekvenčním diagramu po provedení Fourierovy transformace se posunou.



Obrázek 20: Fourierovo spektrum pro testovací signál daný součtem tří harmonických funkcí (121) a pro dvě různé vzorkovací frekvence f_s . Pro f_s jsou již některé komponenty nad Nyquistovou frekvencí a projeví se jako „falešné“ frekvence $F' = f_{\max} - F$.

Řešení 13.3: Vzorové řešení je v souboru `analysis.py` ve funkci `test_signal`. Funkce vytvoří zadanou časovou řadu, přehraje ji a vykreslí graf s Fourierovými komponentami, který je pro $f_s = 2000$ Hz a $f_s = 1000$ Hz zakreslen v obrázku 20 a pro $f_s = 1842$ Hz v obrázku 21. Pro $f_s = 1000$ Hz jsou již frekvence vstupního signálu vyšší než Nyquistova frekvence $f_{\max} = f_s/2 = 500$ Hz. Ve spektru vidíme vrcholy na frekvencích

$$F'_{2,3} = f_{\max} - F_{2,3} = 450 \text{ Hz}, 340 \text{ Hz}. \quad (122)$$

Zvuk je při nevhodně malé vzorkovací frekvenci zkreslený. Abychom analýzu signálu neměli ovlivněnou těmito „falešnými“ frekvencemi, je nutné před digitalizací signálu použít filtr, který frekvence ležící nad Nyquistovou frekvencí odstraní.

V případě zvukového signálu se často používá vzorkovací frekvence $f_s = 44100$ Hz. Přestože odpovídající Nyquistova frekvence $f_{\max} = 22050$ Hz leží v ultrazvuku za oblastí slyšitelnosti, nefiltrované zvuky s frekvencí vyšší než $f_{\max}$ se klidně mohou projevit jako „falešné“ frekvence ve slyšitelné oblasti.

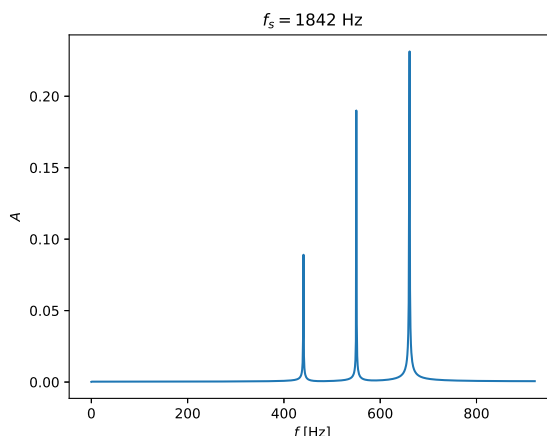
Díky tomu, že délka generované řady je jak pro $N = 2000$, $f_s = 2000$ Hz, tak pro $N = 1000$, $f_s = 1000$ Hz násobkem vzorkovací frekvence a všechny frekvence signálu jsou celočíselné, je signál přesně periodický. Ve Fourierově spektru zobrazeném na obrázku 20 jsou patrné jen tři nenulové komponenty, které mají výšku odpovídající přesně amplitudám harmonických komponent zadaného signálu a_1, a_2, a_3 .

Naopak na obrázku 21 je příklad, kdy délka řady není násobek vzorkovací frekvence. Periody komponent signálu nejsou soudělné s délkou časové řady, a tudíž analyzovaný signál není přesně periodický. Pozorujeme, že vrcholy sice sedí na zadaných frekvencích signálu F_1, F_2, F_3 , ale jsou nepatrně rozmazané (mají nenulovou šířku) a jsou nižší než zadané amplitudy. Zadané amplitudy bychom získali po zintegrování nenulového okolí maxim.

Úloha 13.4: Ve složce `sounds` v repozitáři jsou soubory `a.wav`, `e.wav`, `i.wav`, `o.wav`, `u.wav`, ve kterých jsou nahrané krátké vzorky samohlásek.⁶⁴ Vzorkovací frekvence nahrávek je $f_s = 16000$ Hz, nahrávky mají délku okolo $N \approx 8000$. Soubory načtěte pomocí knihovny `soundfile`, přehrajte si je, vyřízněte z nich kvůli rychlosti okno délky $N = 2000$ (vezměte například prostřední část příkazem `sound[3000:5000]`), vypočítejte na něm Fourierovu transformaci a zakreslete získané amplitudy A_k

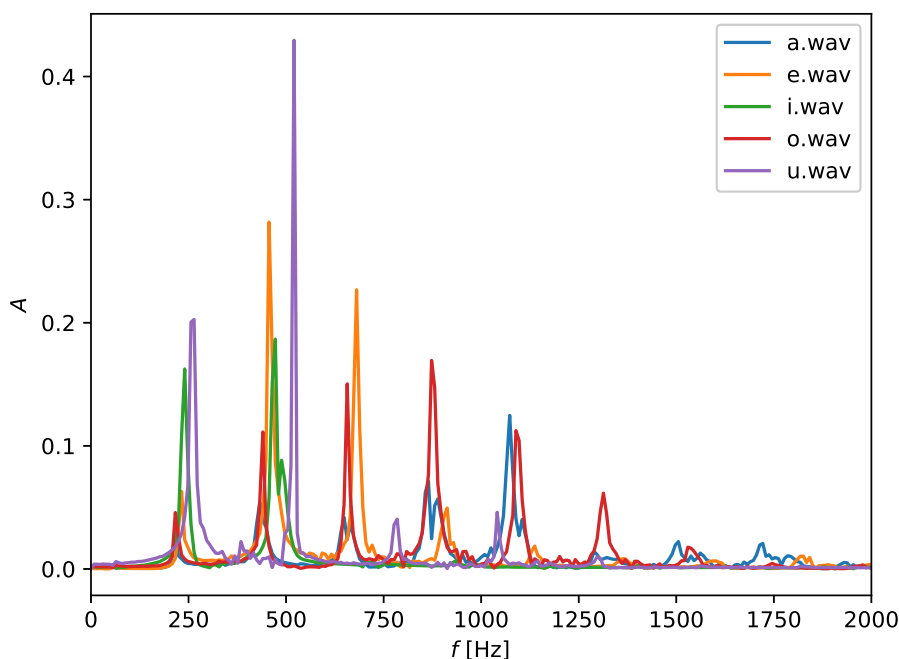
⁶⁴Uvedené frekvence odpovídají po řadě tónům a^1 , $c\#^2$, e^2 v přirozeném ladění. Zazníť by tedy měl durový kvintakord.

⁶⁵Nahrávky jsou staženy z <https://homepages.wmich.edu/~hillenbr/voweldata.html>.



Obrázek 21: Fourierovo spektrum pro testovací signál daný součtem tří harmonických funkcí (121) a pro vzorkovací frekvenci $f_s = 1842\text{s}^{-1}$, která je vyšší než Nyquistova frekvence, ale není soudělná s délkou řady. Signál tudíž není přesně periodický a vrcholy odpovídající frekvencím vstupního signálu se „rozmažou“.

do jednoho grafu pro všech pět samohlásek. Uvidíte, že každá samohláska má zcela jinak vysoké dominantní frekvence. Této skutečnosti se využívá pro strojovou analýzu hlasu.⁶⁶

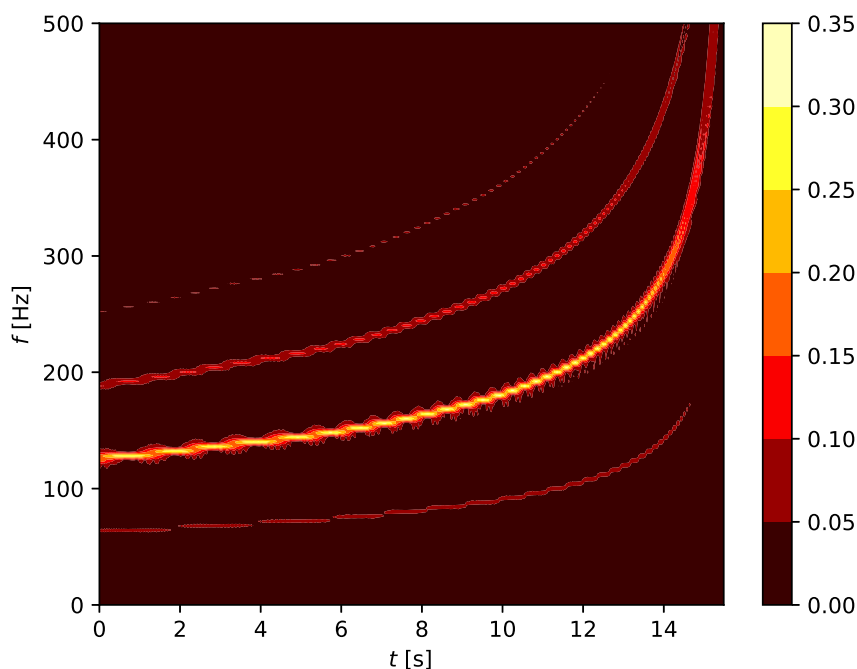


Obrázek 22: Fourierovo spektrum pro samohlásky z testovacích souborů.

Řešení 13.4: Fourierovo spektrum pro zadané samohlásky je vypočítáno pomocí funkce `vowels` ze souboru `analysis.py` a je zobrazeno na obrázku 22. Je vidět, že pro všechny samohlásky se skládá ze základní frekvence (okolo $f_0 = 240\text{ Hz}$) a jejích násobků (harmonické frekvence), přičemž často je dominantní první harmonická frekvence (pro samohlásku „o“ je to až třetí a pro samohlásku „a“ dokonce až čtvrtá harmonická frekvence). Právě poměry mezi amplitudami harmonických frekvencí jsou klíčové pro strojové rozpoznání, o kterou hlásku se jedná.

⁶⁶Pomocí funkce `rec` si můžete nahrát a zanalyzovat vlastní hlas.

Úloha 13.5: Ve složce `sounds` v repozitáři je soubor `BlackHolesCollision.wav`, ve kterém je nasimulovaný průběh gravitačních vln těsně před srážkou dvou černých děr.⁶⁷ Načtete tento soubor, přehrajte si ho (uslyšíte charakteristický tzv. chirp sound). Pozor, soubor má dva kanály, pro následující analýzu vyberte pouze jeden z nich příkazem `signal[:,0]`. Rozdělte časovou řadu na časová okna délky $N_W = 2000$ bodů a pro každé okno spočítejte Fourierovu transformaci a amplitudy A_k . Následně vykreslete konturový graf (spektrogram), kde na ose x bude čas (začátku nebo středu použitého časového okna), na ose y frekvence a na ose z (barevný kód) amplituda. Frekvence omezte pomocí příkazu `plt.ylim(0, 500)` na hodnoty $\langle 0 \text{ Hz}, 500 \text{ Hz} \rangle$.



Obrázek 23: Spektrogram gravitačních vln generovaný okolo sebe obíhajícími dvěma černými děrami těsně před srážkou. Ke srážce dojde v čase $t \approx 15$ s.

Řešení 13.5: Spektrogram je vypočítán pomocí funkce `black_holes` (soubor `analysis.py`) ze zadaného signálu pro plovoucí okno šířky 2000 bodů, které posouváme pro hladší graf po 100 bodech, a je vykreslen na obrázku 23. Je vidět dominantní frekvence stoupající od přibližně 130 Hz. V okamžiku srážky černých děr frekvence diverguje. Dominantní frekvence je první harmonická: základní frekvence začíná na 65 Hz. Podle tvaru křivky na spektrogramu se určí parametry srážejících se objektů: jejich hmotnosti a typ (černé díry, neutronové hvězdy).

Zvukový soubor byl podle zdrojové webové stránky „zrychlen“, aby se posunul do slyšitelných frekvencí. Frekvence ve spektrogramu 23 tedy neodpovídají počtu oběhů za sekundu černých děr těsně před srážkou. Objekty se ve skutečnosti pohybují pomaleji.

„Rázy“, které se objevují ve spektrogramu, jsou numerický artefakt způsobený konečností velikosti okna, ve kterém probíhá výpočet.

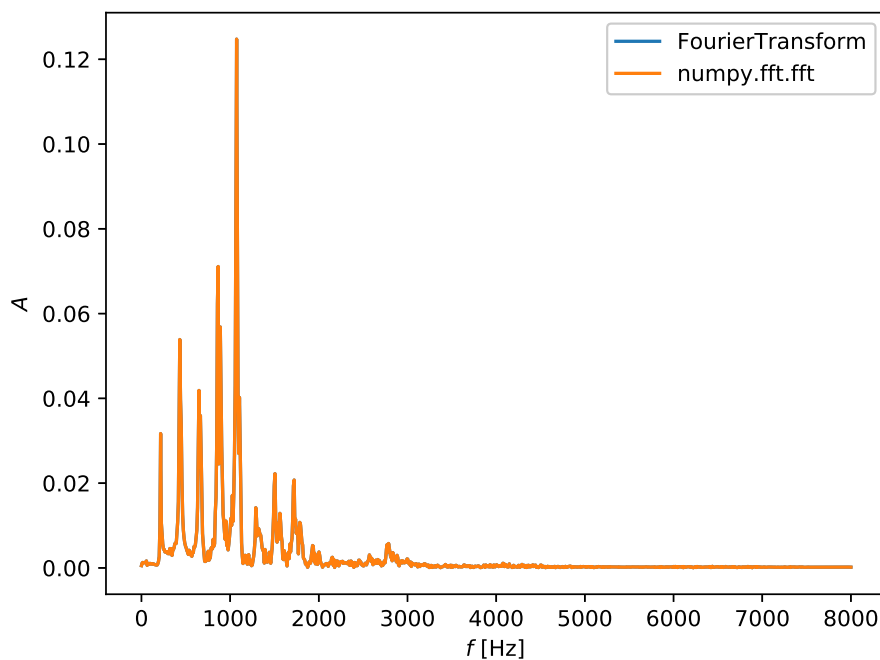
Srážka černých děr pomocí přímého měření gravitačních vln byla poprvé pozorována detektory LIGO a Virgo 14. září 2015. Analýza byla publikována o necelý půl rok později [4] (pozorovaný spektrogram je v tomto článku zobrazen v obrázku 1) a odstartovala nové odvětví fyziky: gravitační astronomii. K dnešnímu dni (květen 2020) bylo prostřednictvím gravitačních vln pozorováno několik desítek srážek velmi hmotných objektů (černých děr nebo neutronových hvězd).

⁶⁷Soubor pochází z <http://web.mit.edu/sahughes/www/sounds.html>.

13.3 Rychlá Fourierova transformace

Fourierova transformace naprogramovaná pomocí definičních vztahů (109) má časovou složitost $\mathcal{O}(N^2)$, pro delší časové řady tedy doba výpočtu prudce roste (již pro $N = 10000$ si počkáte několik minut). Existuje mnohem efektivnější algoritmus s časovou složitostí $\mathcal{O}(N \log N)$. V Pythonu naleznete funkci počítající rychlou Fourierovu transformaci ze zadané časové řady v balíku **numpy**. Jmenuje se `fft.fft`.

Úloha 13.6: Spočítejte Fourierovu transformaci (i) pomocí funkce naprogramované v úkolu 13.2 a (ii) pomocí knihovní funkce `numpy.fft.fft` buď pro signál z úkolu 13.3, nebo pro jednu z hlásek z úkolu 13.4. V grafu porovnejte získané amplitudy. Měly by být stejné.



Obrázek 24: Srovnání různých metod výpočtu Fourierovy transformace pro soubor `a.wav`. Výsledek získaný knihovní funkcí `numpy.fft.fft` byl vydělen délkou časové řady.

Řešení 13.6: Srovnání Fourierovského spektra pro hlásku „a“ je naprogramované ve funkci `FT_method_Comparison` v souboru `analysis.py` a graf je zobrazen na obrázku 24. Fourierova transformace z knihovny `numpy` má jinak definovanou normalizaci, proto je pro porovnání nutné vydělit Fourierovy komponenty délkou časové řady N . Pak jsou komponenty spočítané oběma metodami stejné.

14 Kafe

Toto cvičení se bude týkat zpracování naměřených dat od načtení dat přes vytvoření modelu, nafitování jeho parametrů, vykreslení dat a uložení výsledků. Vše bude demonstrováno na časové řadě teplot během chlazení kávy. Data naměřil již před více než 15 lety zakladatel tohoto cvičení [doc. Jiří Dolejší](#) a jsou dostupná v repozitáři v souboru `kafe.txt`. Vzorový kód v Pythonu je pak v souboru `kafe.py`.

14.1 Načtení dat

Umět načíst externí data do vašeho programu je velmi důležité, jelikož zdroj dat může být úplně jiný systém než ten, ve kterém data zpracováváte (data můžete získat měřením, můžete je stáhnout z internetu, může vám je poslat kolega z druhého konce světa nebo je získáte výpočtem nějakém specializovaném programu). My se budeme zabývat načtením textového souboru.

V Pythonu existuje nespočet možností, jak textová data načíst: od jednoduchého otevření souboru, načítání znaku po znaku nebo řádky po řádce a zpracovávání přes různá automatizovaná řešení, například funkcí `genfromtxt` z knihovny `numpy` až po sofistikovaný balík `pandas`. V příloženém vzorovém souboru je použita jednoduchá možnost pomocí knihovny `csv`,⁶⁸ která je součástí instalace jazyka Python. Pokud neznáme formát dat předem, je dobré si datový soubor nejprve otevřít v nějakém textovém editoru (například ve Visual Studiu Code) a zjistit, jak dlouhá je hlavička a v jakém sloupci nebo řádce je jaký údaj. V souboru `kafe.txt` vidíme, že hlavička je dlouhá 10 řádků a je následována daty ve čtyřech sloupcích, přičemž v prvním sloupci je to nejdůležitější — měřená teplota —, ve druhém jednotka (stupně Celsia), ve třetím označení TE (jako teploměr) a ve čtvrtém čas. Měření probíhala striktně po 10 sekundách, nemusíme tudíž složitě převádět formát času na číslo, ale stačí začít s časem $t_0 = 0$ s a postupně přičítat 10 s. Data z hlavičky potřebovat nebudeme, proto celou hlavičku přeskóčíme. Z celého souboru stačí tedy načíst první sloupec.

Měřená data jsou dvojice čas–teplota

$$(t_j^{\text{exp}}, T_j^{\text{exp}}), \quad j = 0, \dots, N-1, \quad (123)$$

kde N je počet měření. Data jsou vykreslena na obrázku 25. Teplota byla měřena s přesností $\Delta T^{\text{exp}} = 1^\circ\text{C}$.

14.2 Model

Chlazení kapaliny vedením tepla popisuje [Newtonův zákon chlazení](#). Odevzdané teplo ΔQ za krátký okamžik Δt je tím větší, čím je větší rozdíl teploty tělesa T a teploty okolí $T_{\text{okolí}}$,

$$\Delta Q = -hS(T - T_{\text{okolí}})\Delta t, \quad (124)$$

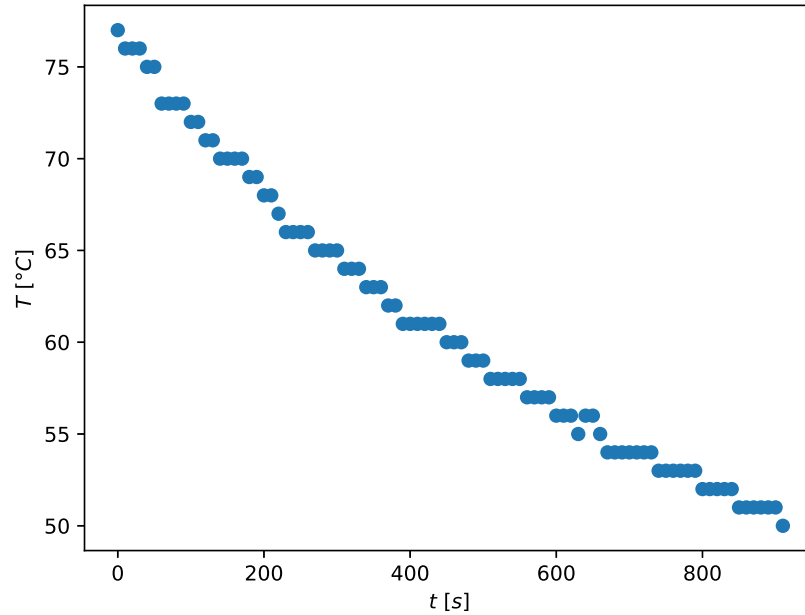
kde h je koeficient vedení tepla a S aktivní povrch, skrz který teplo z tělesa uniká. Na druhou stranu tím, že těleso teplo odevzdá, dojde k jeho ochlazení o teplotu ΔT ,

$$\Delta Q = cm\Delta T, \quad (125)$$

kde c je měrná tepelná kapacita a m hmotnost tělesa. Z těchto rovnic získáváme pro infinitezimální změny dQ , dT , dt diferenciální rovnici

$$\frac{dT}{dt} = - \underbrace{\frac{hS}{cm}}_{\alpha} (T - T_{\text{okolí}}), \quad (126)$$

⁶⁸**CSV** = **C**omma-**S**eparated **V**alues, hodnoty oddělené čárkami. Ve skutečnosti může být jako oddělovač libovolný znak, běžně třeba mezera nebo tabulátor. Oddělovač se specifikuje pomocí parametru `delimiter`.



Obrázek 25: Naměřená data chlazení kávy.

jejímž řešením je zmiňovaný Newtonův zákon chlazení⁶⁹

$$T(t; \alpha) = (T_0 - T_{\text{okolí}}) e^{-\alpha(t-t_0)} + T_{\text{okolí}} \quad (127)$$

s počáteční podmínkou $T(t_0) = T_0$. Explicitně je navíc uveden parametr modelu α .

14.3 Fitování dat

Máme model (127) s jedním parametrem α . Dalším krokem je nalézt optimální hodnotu tohoto parametru. K tomu se používá *metoda nejmenších čtverců* odchylek, která obecně spočívá v minimalizaci funkce

$$R^2(\alpha, \beta, \dots) = \sum_{j=0}^{N-1} \left[T_j^{\text{exp}} - T(t_j^{\text{exp}}; \alpha, \beta, \dots) \right]^2, \quad (128)$$

kde minimum hledáme přes hodnoty parametrů $\alpha, \beta, \dots$. K fitování slouží v Pythonu dvě funkce z knihovny `scipy.optimize`, a to `least_squares` a `curve_fit`, které fungují podobně.⁷⁰ My použijeme druhou z nich, neboť nám poskytne i odhady chyb fitovaných parametrů, což se ve fyzice hodí.

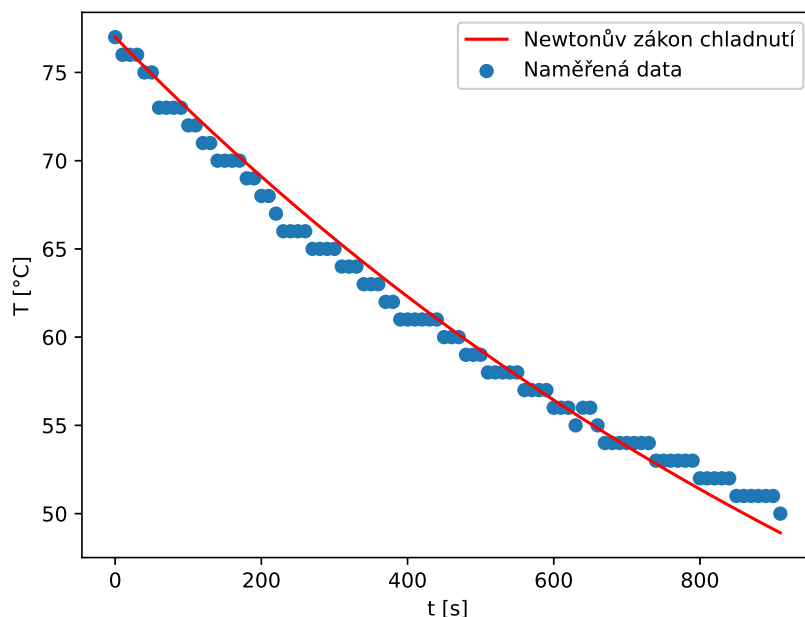
Funkce `curve_fit` očekává na vstupu funkci modelu a dvojici naměřených dat. Dále je vhodné udat počáteční odhad fitovaných parametrů (argument `p0`). Zejména pokud jsou optimální hodnoty parametrů o několik řádů jinde než 1, optimalizační procedura může selhat, jak se lze přesvědčit na vzorovém příkladu. Další parametr je `sigma`, ve kterém můžeme specifikovat absolutní chyby jednotlivých měření. To se hodí zejména v případě, kdy každý naměřený údaj má jinou chybu.

Fitovací funkce vrací dvě pole: v prvním jsou nafitované hodnoty všech parametrů modelu, v druhém je tzv. *korelační matice*, jejíž diagonální prvky udávají odhady chyb parametrů modelu. Funkce nevrací součet kvadrátů reziduí R_{min}^2 , který může sloužit k porovnání výkonností jednotlivých modelů. Tuto veličinu však není obtížné vypočítat.

Použití fitovací funkce je ukázáno ve vzorovém kódu `kafe.py`. Za teplotu okolí byla použita hodnota $T_{\text{okolí}} = 20$ °C. Experimentální data a optimální teoretická křivka jsou na obrázku 26.

⁶⁹Diferenciální rovnici vyřešíme například jednoduše separací proměnných.

⁷⁰Obě funkce volají jednu a tutéž minimalizační proceduru.



Obrázek 26: Fitování Newtonova zákona chladnutí. Optimální hodnota parametru modelu jsou $\alpha = (746 \pm 5) \cdot 10^{-6} \text{ s}^{-1}$. Součet reziduí je $R_{\min} = 8.64 \text{ °C}$.

Je vidět, že Newtonův zákon nepopisuje dobře skutečnost. Otevírá se tedy prostor pro nás fyziky, abychom se pokusili model vylepšit.

14.4 Vylepšení modelu

Jednoduchý model založený na ztrátě tepla vedením neobstojí. Ukažme si několik možností, jak model rozšířit. Rozšíření modelu mimo jiné znamená, že přidáme další parametry, jejichž hodnotu budeme fitovat. Platí samozřejmě, že bez ohledu na kvalitu modelu čím více parametrů, tím lepší fit. Je vždy dobré se vyvarovat extrémů, kdy model má více parametrů, než kolik máme experimentálních hodnot N . Dobrý model je takový, který popisuje pomocí málo parametrů velké množství experimentálních dat.

14.4.1 Neurčitá teplota okolí

Teplotu okolí $T_{\text{okolí}}$ neznáme. Předpokládali jsme jen, že experiment byl prováděn za běžné pokojové teploty, a proto jsme volili $T_{\text{okolí}} = 20 \text{ °C}$. Co však, pokud byla teplota jiná? Můžeme zvolit teplotu okolí jako druhý fitovaný parametr modelu.

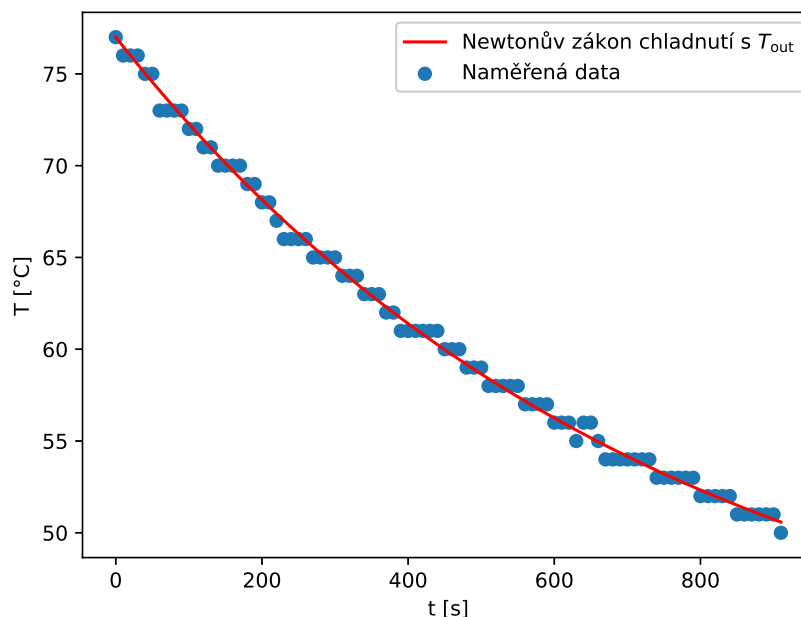
Výsledek fitování dává mnohem lepší souhlas modelu s naměřenými daty (obrázek 27), avšak nabitovaná teplota okolí je $T_{\text{okolí}} \approx 40 \text{ °C}$, což neodpovídá laboratorním podmínkám.⁷¹ Tento model, ač popisuje velmi dobře teoretická data, zavrhneme.⁷²

14.4.2 Model s vyzařováním

Kapalina chladne čtyřmi základními procesy: vedením, prouděním, vyzařováním a vypařováním. Newtonův zákon popisuje jen první z nich. Druhý proces (proudění) závisí významně na tvaru tělesa

⁷¹Doc. Jiří Dolejší se dušuje, že na takovou teplotu si netopí.

⁷²Při hlubším zamyšlení však lze i tento model přijmout. Jak je známo, vzduch je celkem špatný vodič tepla. Okolo tělesa chladnoucího na vzduchu se tedy vytvoří teplejší vrstva, která může odpovídat „efektivní“ teplotě okolí vyšší, než je skutečná teplota okolí daleko od chladnoucího tělesa.



Obrázek 27: Fitování Newtonova zákona chladnutí s volnou teplotou okolí. Optimální hodnoty parametrů jsou $\alpha = (1360 \pm 30) \cdot 10^{-6} \text{ s}^{-1}$, $T_{\text{okolí}} = (39.8 \pm 0.6) \text{ }^{\circ}\text{C}$. Součet reziduí je $R_{\text{min}} = 3.76 \text{ }^{\circ}\text{C}$.

a na okolním prostředí (foukáme do kávy? máme puštěný větrák? jsme na vrcholu větrné hory?) a je obtížné ho modelovat. Můžeme předpokládat, že experiment byl prováděn v laboratoři (kanceláři) a vliv proudění byl eliminován. Zbývají tedy procesy vyzařování a vypařování. Zaměříme se na první z nich.

Těleso⁷³ vyzařuje podle [Stefanova-Boltzmannova zákona](#), podle kterého je odevzdané teplo za časový okamžik Δt za předpokladu, že teplota povrchu tělesa je T a teplota okolí $T_{\text{okolí}}$,⁷⁴

$$\Delta Q = -\sigma S (T^4 - T_{\text{okolí}}^4) \Delta t, \quad (129)$$

kde σ je Stefanova-Boltzmannova konstanta a S je povrch tělesa. To v kombinaci s (124) vede na diferenciální rovnici

$$\frac{dT}{dt} = -\alpha (T - T_{\text{okolí}}) - \underbrace{\beta}_{\frac{\sigma S}{cm}} (T^4 - T_{\text{okolí}}^4). \quad (130)$$

Tato rovnice není exaktně řešitelná, je tedy potřeba ji řešit numericky. Jedná se o diferenciální rovnici 1. řádu, což je téma, se kterým jsme tento kurz začínali (sekce 4). Nové je zde to, že musíme zkombinovat řešení diferenciální rovnice s hledáním optimální hodnoty parametrů pro diferenciální rovnici.

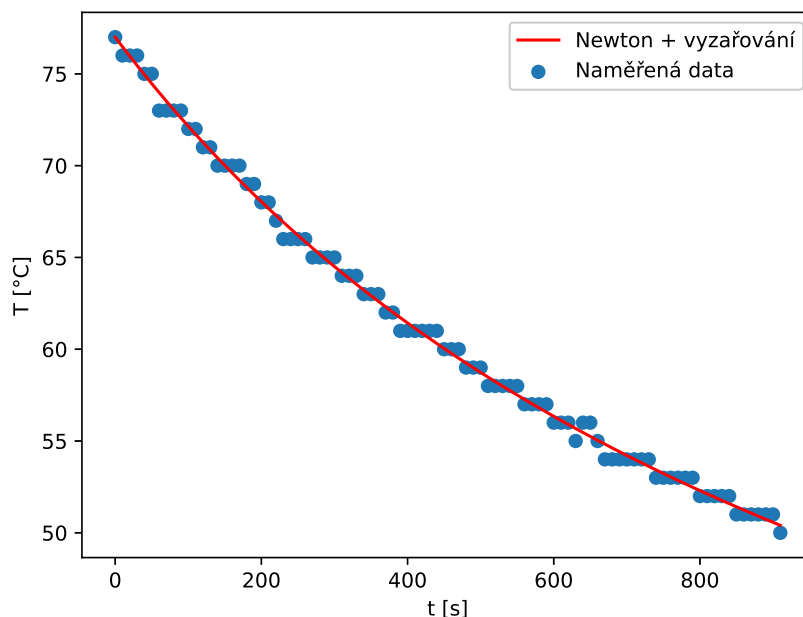
Řešení je opět ve vzorovém kódu [kafe.py](#) a výsledek je na obrázku 28. Optimální hodnota parametru β je řádu 10^{-9} , je tedy nutné při volání funkce `curve_fit` upřesnit počáteční hodnotu pro hledání tohoto parametru v `p0`, jinak optimalizační funkce selže.

Opět dostáváme mnohem lepší souhlas mezi teorií a experimentem, ale fitovací funkce najde optimální hodnotu $\alpha < 0$, což odpovídá nefyzikální situaci, kdy by hrnek s kávou teplo sice vyzařoval, ale naopak přijímal vedením.⁷⁵

⁷³Přesněji tzv. *absolutně černé těleso*, což je ideální těleso pohlcující a vyzařující záření všech vlnových délek.

⁷⁴Ve Stefanově-Boltzmannově zákoně se musí počítat s absolutními teplotami, tj. k teplotám ve $^{\circ}\text{C}$ je nutné připočíst $T_{\text{abs}} = 273.15 \text{ K}$.

⁷⁵Funkci `curve_fit` lze parametrem `bounds` předat intervaly, ve kterých má parametry hledat. V případě tohoto modelu ale optimalizace s požadavkem $\alpha > 0$ nekonverguje.



Obrázek 28: Fitování Newtonova modelu chladnutí rozšířeného o ztrátu tepla vyzařováním (130). Optimální hodnoty parametrů jsou $\alpha = -(2.2 \pm 0.2) \cdot 10^{-3} \text{ s}^{-1}$, $\beta = (24 \pm 1) \cdot 10^{-12} \text{ s}^{-1} \text{ T}^{-3}$. Součet reziduí je $R = 3.69 \text{ °C}$. Hodnota $\alpha < 0$ je nefyzikální.

14.4.3 Další člen Taylorova rozvoje

Můžeme se ocitnout v situaci, kdy vyčerpáme všechny možnosti namodelovat jev chladnutí, které nám naše fyzikální znalosti poskytují. Přesto však chceme popsat naměřená data s větší přesností, než zvládne nejlepší fyzikální model, na který jsme schopni přijít. Řešení, které se často používá, je následující:

- Budeme předpokládat, že (126) je ve skutečnosti složitější funkce rozdílu teplot,

$$\frac{dT}{dt} = -F(T - T_{\text{okolí}}), \quad (131)$$

a že (126) je vlastně jen první člen Taylorova rozvoje (nultý člen jsme zavrhlí, ten odpovídá proměnné teplotě okolí T_{out}).

- Model rozšíříme o druhý Taylorův člen, tj.

$$\frac{dT}{dt} = -\alpha (T - T_{\text{okolí}}) - \beta (T - T_{\text{okolí}})^2. \quad (132)$$

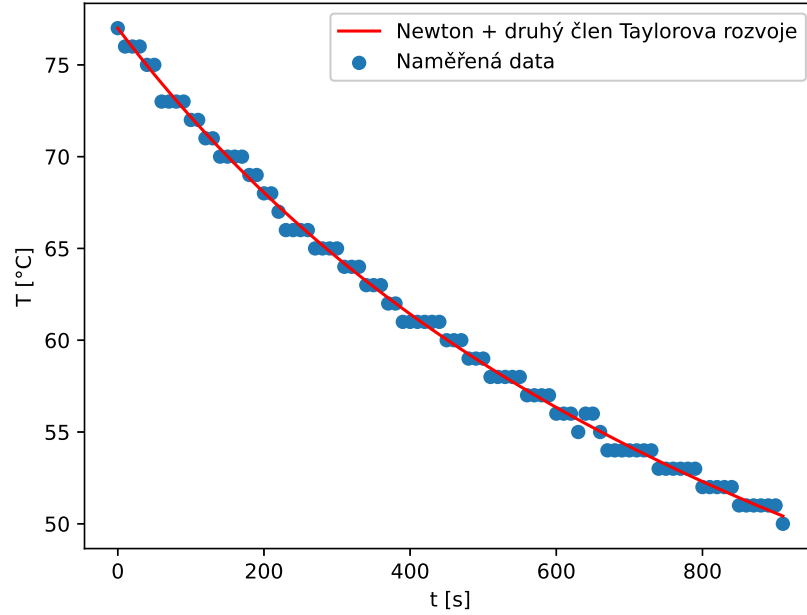
- Pokud ani v tomto případě nedosáhneme požadované přesnosti, přidáme další člen. Takto lze postupovat dál a dál.

Řešení s druhým členem Taylorova rozvoje je ve vzorovém kódu [kafe.py](#) a výsledek je na obrázku 29.

14.5 Shrnutí

Výsledky pro všechny modely jsou v tabulce 1.

O chladnutí kávy a čaje byly napsány i odborné publikace, např. [5]. Pokud Vás toto téma zaujalo, doporučuji do článku nahlédnout.



Obrázek 29: Fitování Newtonova modelu chladnutí rozšířeného o další člen Taylorova rozvoje (132) Optimální hodnoty parametrů jsou $\alpha = (80 \pm 30) \cdot 10^{-6} \text{ s}^{-1}$, $\beta = (14.7 \pm 0.8) \cdot 10^{-6} \text{ s}^{-1} \text{ T}^{-1}$. Součet reziduí je $R = 3.69 \text{ }^\circ\text{C}$.

Název	Parametry	Označení v kódu	Hodnota	Jednotka	$R_{\min} [^\circ\text{C}]$
Newton	α	alpha_fit	746 ± 5	10^{-6} s^{-1}	8.64
Newton s teplotou okolí	α	alpha_out_fit	1360 ± 30	10^{-6} s^{-1}	3.76
	$T_{\text{okolí}}$	T_out_fit	39.8 ± 0.6	$^\circ\text{C}$	
Newton s vyzařováním	α	alpha_rad_fit	-2.2 ± 0.2	10^{-3} s^{-1}	3.69
	β	beta_rad_fit	24 ± 1	$10^{-12} \text{ s}^{-1} \text{ T}^{-3}$	
Newton + Taylorův rozvoj do 2. řádu	α	alpha_2_fit	80 ± 30	10^{-6} s^{-1}	3.69
	β	beta_2_fit	14.7 ± 0.8	$10^{-6} \text{ s}^{-1} \text{ T}^{-1}$	

Tabulka 1: Shrnutí modelů. Červeně označené jsou nepřijatelné hodnoty.

14.6 Uložení dat

Po provedení všech výpočtů uložíme výsledky pro příští použití. Existuje opět hromada možností, jak toho docílit, od jednoduchého zápisu do souboru po sofistikované knihovny pro serializaci a deserializaci dat, jakou je například `pickle`. My využijeme opět jednoduché knihovny `csv`. Uložení všech namodelovaných teoretických křivek je ve vzorovém kódu `kafe.py`.

Reference

- [1] Kevlin Henney, <https://twitter.com/KevlinHenney/status/381021802941906944>.
- [2] M.E. Muller, *A note on a method for generating points uniformly on n-dimensional spheres*, Communications of the Association for Computing Machinery **2**, 19 (1959).
- [3] G. Marsaglia, *Choosing a Point from the Surface of a Sphere*, The Annals of Mathematical Statistics **43**, 645 (1972).
- [4] B. P. Abbott *et al.* (LIGO Scientific Collaboration and Virgo Collaboration), *Observation of Gravitational Waves from a Binary Black Hole Merger*, Physical Review Letters **116**, 061102 (2016).
- [5] W. G. Rees, C. Viney, *On cooling tea and coffee*, American Journal of Physics **56**, 434 (1988).